Universidad de Valladolid

Facultad de Ciencias de Valladolid

Grado en Matemáticas

Polinomios de Conway: Implementación en Singular

Alumno: Miguel Sánchez-Beato Díaz-Hellín

Tutores: José Ignacio Farrán Martín

Fecha: 18 de marzo de 2026

Resumen

Los polinomios de Conway proporcionan una elección canónica de polinomio irreducible para representar cada cuerpo finito $\mathbb{F}_{p^n}$, garantizando la compatibilidad entre extensiones de cuerpos. Calcularlos es un problema computacionalmente costoso que combina álgebra de cuerpos finitos, teoría de números y álgebra computacional.

En este trabajo presentamos los fundamentos teóricos de los polinomios de Conway, incluyendo la demostración de su existencia, y desarrollamos una librería para Singular (sistema de álgebra computacional desarrollado en Kaiserslautern) que implementa el algoritmo de Heath-Loehr (2004), motivados por el trabajo previo de Ruano y Laplagne (2007). La implementación incorpora además optimizaciones propias que permiten extender el cálculo a grados no abordados anteriormente y aumentar la velocidad de cálculo de estos polinomios.

Hemos calculado polinomios de Conway para los cuerpos finitos $\mathbb{F}_2$, $\mathbb{F}_3$, $\mathbb{F}_5$, $\mathbb{F}_7$, $\mathbb{F}_{11}$ y $\mathbb{F}_{13}$ principalmente, y realizado pruebas para otros primos mayores. El resultado más importante ha sido el cálculo de ocho polinomios de Conway que no estaban registrados, correspondientes a los exponentes de Mersenne 521, 607, 1279, 2203, 2281, 3217, 4253 y 4423, gracias a una optimización para este caso que antes no se tenía en cuenta.

Palabras clave: polinomios de Conway, cuerpos finitos, polinomios primitivos, algoritmo de Heath-Loehr, Singular, primos de Mersenne.

Abstract

Conway polynomials provide a canonical choice of irreducible polynomial to represent each finite field $\mathbb{F}_{p^n}$, guaranteeing compatibility between field extensions. Computing them is a computationally expensive problem that combines finite field algebra, number theory and computational algebra.

In this work we present the theoretical foundations of Conway polynomials, including the proof of their existence, and develop a library for Singular (a computer algebra system developed in Kaiserslautern) that implements the Heath-Loehr algorithm (2004), motivated by the previous work of Ruano and Laplagne (2007). The implementation additionally incorporates original optimizations that allow extending the computation to degrees not approached previously and increasing the speed of computation of these polynomials.

We have computed Conway polynomials for the finite fields $\mathbb{F}_2$, $\mathbb{F}_3$, $\mathbb{F}_5$, $\mathbb{F}_7$, $\mathbb{F}_{11}$ and $\mathbb{F}_{13}$ primarily, and tested other larger primes. The most important result has been the computation of eight Conway polynomials that were not previously recorded, corresponding to the Mersenne exponents 521, 607, 1279, 2203, 2281, 3217, 4253 and 4423, thanks to an optimization for this particular case that had not been considered before.

Keywords: Conway polynomials, finite fields, primitive polynomials, Heath-Loehr algorithm, Singular, Mersenne primes.

Índice general

1. Introducción **1**
1.1. Motivación . . . 1
1.2. Objetivos del trabajo . . . 2
1.3. Estructura del trabajo . . . 2

2. Preliminares **5**
2.1. Estructuras algebraicas . . . 5
2.2. Anillos de polinomios . . . 8
2.3. Cuerpos finitos y sus propiedades . . . 9
2.4. Polinomios primitivos e irreducibilidad . . . 11
2.5. Teoría de Galois en cuerpos finitos . . . 13
2.6. Resultados algorítmicos sobre cuerpos finitos . . . 13
2.6.1. Test de irreducibilidad de Ben-Or . . . 13
2.7. Extensiones, Normas y Compatibilidad . . . 14
2.7.1. Inclusión de Subcuerpos . . . 14
2.7.2. Función Norma Relativa . . . 15
2.7.3. Compatibilidad de Norma . . . 17
2.7.4. Función Traza Relativa . . . 18
2.7.5. Compatibilidad de Traza . . . 19

3. Polinomios de Conway **21**
3.1. Definición y propiedades . . . 21
3.2. Existencia de los polinomios de Conway . . . 22
3.3. Reducción a divisores maximales . . . 24
3.4. El parámetro g: número de candidatos compatibles . . . 24
3.5. Ejemplo: cálculo de $C_{2,6}$ . . . 26
3.6. Descripción del algoritmo . . . 27
3.7. Tabla de polinomios de Conway conocidos . . . 28

4. Algoritmo e implementación **31**
4.1. Cuatro caminos según la estructura de n . . . 31
4.2. Caso compuesto ($s \geq 2$): el algoritmo de Heath-Loehr . . . 31
4.2.1. Construcción de $r(z)$ . . . 32
4.2.2. Iteración sobre los g candidatos . . . 32
4.2.3. Limitación por el orden lexicográfico . . . 32
4.3. Caso potencia de primo ($s = 1$, $e \geq 2$) . . . 32

4.4. Caso primo ($s = 1$, $e = 1$): búsqueda exhaustiva . . . 33
4.4.1. ¿Por qué funciona bien? . . . 33
4.4.2. El cuello de botella: factorización de $p^n - 1$ . . . 33
4.5. Primos de Mersenne: un caso excepcional . . . 34
4.5.1. Test de irreducibilidad de Ben-Or . . . 34
4.5.2. La función `conwayPolyMersenne` . . . 34
4.6. Funciones auxiliares . . . 35
4.6.1. Test de primitividad . . . 35
4.6.2. Comparación en orden de Parker . . . 35
4.6.3. Extracción de raíces g-ésimas . . . 36
4.6.4. Búsqueda de irreducibles . . . 36
4.7. Implementación en Singular . . . 36
4.7.1. Estructura de la librería . . . 36
4.7.2. Representación de $\mathbb{F}_{p^n}$ . . . 37
4.7.3. Exponenciación modular . . . 37
4.7.4. Comparación con la implementación original de Heath y Loehr . . . 37
4.8. Verificación de resultados . . . 38
4.8.1. Comprobación de primitividad . . . 38
4.8.2. Comprobación de compatibilidad . . . 38
4.8.3. Comparación con tablas conocidas . . . 39

5. Resultados computacionales **41**
5.1. Polinomios de Conway sobre $\mathbb{F}_2$ . . . 41
5.1.1. Grados $n = 1, \ldots, 41$ . . . 41
5.1.2. Grados compuestos mayores . . . 42
5.1.3. Grados primos hasta 211 . . . 43
5.1.4. Polinomios de Conway para exponentes de Mersenne . . . 43
5.2. Polinomios de Conway sobre $\mathbb{F}_3$, $\mathbb{F}_5$, $\mathbb{F}_7$, $\mathbb{F}_{11}$ y $\mathbb{F}_{13}$ . . . 44
5.2.1. Característica $p = 3$ . . . 44
5.2.2. Característica $p = 5$ . . . 45
5.2.3. Característica $p = 7$ . . . 45
5.2.4. Características $p = 11$ y $p = 13$ . . . 46
5.3. Análisis del rendimiento . . . 47
5.3.1. El caso compuesto: dependencia con g . . . 47
5.3.2. El caso primo: el papel de la factorización de $p^n - 1$ . . . 47
5.3.3. Comparación con la búsqueda exhaustiva . . . 47
5.3.4. Los exponentes de Mersenne: un caso límite favorable . . . 48
5.4. Resumen . . . 48

6. Conclusiones **49**
6.1. Resumen de aportaciones . . . 49
6.2. Reflexiones . . . 50
6.3. Líneas de trabajo futuro . . . 51

A. Instalación de Singular **53**
A.1. Linux / Unix 53
A.1.1. Ubuntu (20.04, 22.04, 24.04) 53
A.1.2. Debian (11, 12, 13) 54
A.1.3. Fedora y otras distribuciones 54
A.1.4. Gentoo 54
A.2. Windows 54
A.2.1. Opción A: Cygwin (método clásico) 54
A.2.2. Opción B: WSL (más cómodo, recomendado) 55
A.3. macOS 55
A.3.1. Homebrew (recomendado) 55
A.3.2. Archivo DMG (instalación manual) 55
A.3.3. Fink (alternativa) 56
A.4. Primer uso y carga de conway.lib 56
A.5. Recursos adicionales 56

B. Código fuente de conway.lib **57**

Bibliografía **89**

Capítulo 1

Introducción

1.1. Motivación

Todo cuerpo finito con p^n elementos es isomorfo a cualquier otro con el mismo número de elementos. Sin embargo, para trabajar computacionalmente con un cuerpo finito $\mathbb{F}_{p^n}$ es necesario elegir una representación concreta, lo que en la práctica significa elegir un polinomio irreducible de grado n sobre $\mathbb{F}_p$ para construir el cociente $\mathbb{F}_p[x]/(f(x))$. Distintas elecciones de f producen representaciones distintas: los mismos elementos se expresan con coordenadas diferentes, y las tablas de multiplicación cambian.

Cuando se trabaja con un solo cuerpo finito, esta ambigüedad es irrelevante. El problema aparece cuando se necesitan *varios* cuerpos finitos a la vez, relacionados por inclusiones entre subcuerpos. Si m divide a n, el cuerpo $\mathbb{F}_{p^m}$ es un subcuerpo de $\mathbb{F}_{p^n}$, pero para que esta inclusión sea computacionalmente manejable, los polinomios irreducibles elegidos para representar ambos cuerpos deben ser *compatibles*: la raíz del polinomio del cuerpo grande, elevada a la potencia adecuada, debe ser raíz del polinomio del cuerpo pequeño.

Los **polinomios de Conway**, introducidos por R. Parker en 1990 [8] y formalizados posteriormente en [4], resuelven este problema fijando, para cada primo p y cada grado n, un único polinomio irreducible canónico que es primitivo, compatible con los de todos los grados divisores, y mínimo en un cierto orden lexicográfico. Esta estandarización permite que sistemas de álgebra computacional como GAP, Magma y Singular utilicen la misma representación de los cuerpos finitos, facilitando la portabilidad de datos y la interoperabilidad entre sistemas.

El cálculo de estos polinomios es, no obstante, un problema computacionalmente costoso. El método más directo, que es recorrer todos los polinomios mónicos de grado n hasta encontrar el primero que cumpla las condiciones, tiene un coste que crece exponencialmente con n y se vuelve impracticable para grados moderados. Heath y Loehr [4] propusieron en 2004 un algoritmo alternativo que, para grados compuestos, reduce drásticamente el espacio de búsqueda aprovechando la estructura algebraica de los cuerpos finitos.

Hay que tener en cuenta que se han utilizado **ideas de la implementación de Ruano y Laplagne** en Singular en 2007 [10], aunque el código ha sido reconstruido desde cero y ampliado significativamente para superar cuellos de botella que hacían impracticable su uso en grados altos, además de que había quedado en desuso.

1.2. Objetivos del trabajo

El objetivo principal de este trabajo es implementar el algoritmo de Heath-Loehr en Singular y utilizarlo para calcular polinomios de Conway, incluyendo algunas optimizaciones que permitan calcular algunos que, hasta donde sabemos, no han sido calculados previamente.

Los objetivos concretos son:

1. **Estudio teórico.** Presentar los fundamentos algebraicos necesarios: cuerpos finitos, polinomios primitivos e irreducibles, la norma en extensiones de cuerpos finitos, y el criterio de irreducibilidad de Rabin/Ben-Or. Definir formalmente los polinomios de Conway, demostrar su existencia mediante la teoría de sistemas compatibles de generadores en grupos cíclicos, y analizar el parámetro g que determina el número de candidatos compatibles.

2. **Implementación.** Desarrollar una librería completa para Singular (`conway.lib`) que implemente el algoritmo de Heath-Loehr junto con las optimizaciones necesarias para ser más eficiente: búsqueda exhaustiva en orden de Parker para grados primos, test de irreducibilidad de Ben-Or, poda lexicográfica, detección de primos de Mersenne mediante Miller-Rabin, y factorización delegada al sistema operativo Linux, que está muy optimizada.

3. **Cálculo de resultados.** Calcular polinomios de Conway para un rango amplio de parámetros: todos los grados hasta 41 y todos los primos hasta 211 sobre $\mathbb{F}_2$, grados selectos sobre $\mathbb{F}_3$, $\mathbb{F}_5$, $\mathbb{F}_7$, $\mathbb{F}_{11}$ y $\mathbb{F}_{13}$ y primos superiores, y como conclusión final, los polinomios de Conway $C_{2,n}$ para los exponentes de Mersenne $n = 521, 607, 1279, 2203, 2281, 3217, 4253$ y 4423, que no figuran en las bases de datos existentes.

4. **Verificación.** Comprobar que son correctos los polinomios calculados contrastándolos con las tablas de Frank Lübeck [6] (para los grados ya conocidos).

1.3. Estructura del trabajo

El trabajo se organiza en cinco capítulos, además de esta introducción, y dos apéndices.

Capítulo 2: Preliminares. Se recogen los resultados de álgebra necesarios para el resto del trabajo: aritmética de polinomios, cuerpos finitos y sus propiedades, polinomios primitivos e irreducibles, criterios de irreducibilidad (Eisenstein, Rabin, Ben-Or), Teoría de Galois en cuerpos finitos (automorfismo de Frobenius), y las funciones norma y traza con sus propiedades.

Capítulo 3: Polinomios de Conway. Se define el orden de Parker y los polinomios de Conway, se demuestra su existencia mediante la teoría de sistemas compatibles de generadores, se establece la reducción a divisores maximales, y se introduce el parámetro g. Se incluye un ejemplo detallado (el cálculo de $C_{2,6}$) y una descripción general del algoritmo de Heath-Loehr, distinguiendo los distintos casos según la factorización de n.

Capítulo 4: Algoritmo e implementación. Se describen en detalle los cuatro caminos que sigue nuestra implementación según la estructura de n (compuesto, potencia de primo, primo, primo de Mersenne), las optimizaciones incorporadas (multiplicación incremental, poda lexicográfica, test de Ben-Or, detección de Mersenne por Miller-Rabin, factorización externa) y la estructura de la librería `conway.lib`.

Capítulo 5: Resultados computacionales. Se presentan los polinomios calculados, organizados por característica, con tablas de tiempos y análisis de rendimiento. Se destacan los cinco

polinomios nuevos para exponentes de Mersenne y se discute el alcance y las limitaciones de la implementación.

Capítulo 6: Conclusiones. Se resumen las aportaciones del trabajo y se sugieren líneas de trabajo futuro.

Apéndice A: Instalación de Singular. Instrucciones de instalación para Linux, Windows y macOS.

Apéndice B: Código fuente de `conway.lib`. Listado completo de la librería con comentarios.

Capítulo 2

Preliminares: Teoría de Galois, Polinomios en Cuerpos Finitos y Compatibilidad de Norma y de Traza

En este capítulo se presentan los fundamentos algebraicos necesarios para la comprensión y construcción de cuerpos finitos, así como las herramientas teóricas que permiten definir y caracterizar los polinomios de Conway.

2.1. Estructuras algebraicas

Definición 2.1.1. *Un conjunto G con una operación binaria interna $*$ en él definida se dice que es un* ***grupo*** *si se cumplen las siguientes condiciones:*

(G1) La operación binaria $*$ es asociativa, es decir,

$$g_1 * (g_2 * g_3) = (g_1 * g_2) * g_3, \quad \forall g_1, g_2, g_3 \in G.$$

(G2) Existe un elemento neutro $e \in G$ tal que

$$e * g = g * e = g, \quad \forall g \in G.$$

(G3) Para todo elemento $g \in G$ existe un elemento $g' \in G$, denominado ***inverso*** *de g, tal que*

$$g * g' = g' * g = e.$$

Si además la operación binaria $*$ cumple la propiedad conmutativa, es decir,

$$g_1 * g_2 = g_2 * g_1, \quad \forall g_1, g_2 \in G,$$

entonces se dice que G es un ***grupo conmutativo*** *o* ***abeliano****.*

Definición 2.1.2. *Sea $(G, *)$ un grupo. Un subconjunto $H \subseteq G$ se dice que es un* ***subgrupo*** *de G si H es un grupo con la misma operación $*$ inducida de G. Es decir, H satisface las siguientes condiciones:*

(S1) H es cerrado bajo la operación $*$, es decir,

$$\forall h_1, h_2 \in H, \quad h_1 * h_2 \in H.$$

(S2) H contiene el elemento neutro de G, es decir,

$$e \in H.$$

(S3) H es cerrado bajo la operación de tomar inversos, es decir,

$$\forall h \in H, \quad h^{-1} \in H.$$

En otras palabras, H hereda la estructura de grupo de G, por lo que también es un grupo bajo la misma operación $*$.

Definición 2.1.3. *Un conjunto A dotado de dos operaciones binarias cerradas que escribiremos $+$ (suma) y $\cdot$ (producto) se llama* ***anillo*** *si se cumplen las siguientes propiedades:*

(A1) $(A, +)$ es un grupo abeliano.

(A2) El producto $\cdot$ es asociativo, es decir,

$$a \cdot (b \cdot c) = (a \cdot b) \cdot c, \quad \forall a, b, c \in A.$$

(A3) El producto $\cdot$ es distributivo respecto a la suma $+$, es decir,

$$a \cdot (b + c) = a \cdot b + a \cdot c, \quad (b + c) \cdot a = b \cdot a + c \cdot a, \quad \forall a, b, c \in A.$$

Si además el producto es conmutativo, es decir,

$$a \cdot b = b \cdot a, \quad \forall a, b \in A,$$

entonces A se llama un ***anillo conmutativo****.*

Si existe un elemento $1 \in A$ tal que

$$1 \cdot a = a \cdot 1 = a, \quad \forall a \in A,$$

entonces A se llama un ***anillo con unidad*** *o* ***anillo unitario****.*

Definición 2.1.4. *Sea $(A, +, \cdot)$ un anillo. Un subconjunto $B \subseteq A$ se dice que es un* ***subanillo*** *de A si B es un anillo con las operaciones inducidas de A. Es decir, B satisface:*

(SA1) $(B, +)$ es un subgrupo de $(A, +)$.

(SA2) B es cerrado bajo el producto, es decir,

$$\forall b_1, b_2 \in B, \quad b_1 \cdot b_2 \in B.$$

(SA3) B contiene el elemento neutro multiplicativo de A (si existe).

Definición 2.1.5. *Un **ideal** I en un anillo R es un subgrupo de $(R,+)$ que satisface $xr, rx \in I$ para todo $x \in I$ y $r \in R$.*

*Un **ideal izquierdo** I de R es un subgrupo de $(R,+)$ tal que $rx \in I$ siempre que $r \in R$ y $x \in I$. Un **ideal derecho** se define de forma análoga cumpliendo que $xr \in I$ siempre que $x \in I$ y $r \in R$.*

Para anillos conmutativos, las tres definiciones implican que el subgrupo es un ideal.

Definición 2.1.6. *El ideal más pequeño que contiene un subconjunto S se llama el **ideal generado por** S, y se denota por (S).*

*Un ideal generado por un único elemento $x \in R$ se denomina **ideal principal** y se denota por (x).*

Definición 2.1.7. *Un ideal P de un anillo conmutativo A se dice **primo** si $P \neq A$ y*

$$ab \in P \Rightarrow a \in P \text{ o } b \in P.$$

Definición 2.1.8. *Un ideal M en un anillo R se llama **maximal** si $M \neq R$ y no existen ideales estrictamente entre M y R; es decir, los únicos ideales que contienen a M son M y R.*

Definición 2.1.9. *Dado un anillo A y un ideal I, el conjunto de clases laterales $A/I = \{a + I : a \in A\}$ forma un **anillo cociente** bajo las operaciones:*

$$(a + I) + (b + I) = (a + b) + I$$

$$(a + I) \cdot (b + I) = (a \cdot b) + I$$

Definición 2.1.10. *Un anillo conmutativo con unidad A se llama **dominio de integridad** si no tiene divisores de cero; es decir, si*

$$ab = 0 \Rightarrow a = 0 \text{ o } b = 0.$$

Teorema 2.1.1. *Sea A un anillo conmutativo con unidad e I un ideal. Entonces:*

$$I \text{ es primo} \iff A/I \text{ es un dominio de integridad.}$$

Definición 2.1.11. *Un dominio de integridad A se llama **dominio de ideales principales** (PID) si todo ideal de A es principal.*

Proposición 2.1.1. *En un dominio de ideales principales, un ideal es primo si y solo si es maximal.*

Definición 2.1.12. *Un **cuerpo** es un anillo con unidad $(A,+,\cdot)$ tal que el conjunto*

$$A^* = A \setminus \{0\}$$

formado por todos los elementos de A excepto el neutro de la suma, es un grupo con respecto a la segunda operación $\cdot$. Si también es conmutativo se dice que es un cuerpo conmutativo.

Definición 2.1.13. *Sea $(A,+,\cdot)$ un cuerpo. Un subconjunto $K \subseteq A$ se dice que es un **subcuerpo** de A si K es un cuerpo con las operaciones inducidas de A. En particular, K debe cumplir:*

(SC1) $(K, +, \cdot)$ *es un subanillo con unidad de* A.

(SC2) Todo elemento no nulo de K *es invertible en* K.

Definición 2.1.14. *Sean* R *y* S *dos anillos con identidad. Un* ***homomorfismo de anillos*** *es una aplicación* $\varphi : R \to S$ *que satisface las siguientes propiedades para cualesquiera* $a, b \in R$*:*

1. $\varphi(a + b) = \varphi(a) + \varphi(b)$,
2. $\varphi(a \cdot b) = \varphi(a) \cdot \varphi(b)$,
3. $\varphi(1_R) = 1_S$, *donde* 1_R *y* 1_S *son los elementos identidad de* R *y* S, *respectivamente.*

Observación 1. *Dado un homomorfismo de anillos* $\varphi : R \to S$, *su* ***núcleo****, definido como el conjunto de elementos que se anulan mediante* φ, *es decir,*

$$\ker(\varphi) = \{x \in R \mid \varphi(x) = 0_S\},$$

forma siempre un ideal (bilateral) del anillo R.

Teorema 2.1.2. *Si* A *es un anillo conmutativo con unidad y* M *es un ideal maximal de* A, *entonces el anillo cociente* A/M *es un cuerpo.*

2.2. Anillos de polinomios

En esta sección recordamos algunos resultados básicos sobre anillos de polinomios que serán necesarios para la construcción de extensiones finitas.

Definición 2.2.1. *El* <u>*anillo de polinomios*</u> $K[x]$ *se define como el conjunto de expresiones de la forma*

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0,$$

donde n *es un entero no negativo, los coeficientes* a_i *pertenecen a* K, *y* x *es una variable formal.*

El conjunto $K[x]$ *forma un anillo conmutativo con unidad bajo las operaciones de suma y multiplicación de polinomios definidas a continuación.*

Definición 2.2.2. *Sean* $f(x), g(x) \in K[x]$ *dados por*

$$f(x) = a_n x^n + \cdots + a_1 x + a_0, \quad g(x) = b_m x^m + \cdots + b_1 x + b_0.$$

Definimos la ***suma*** *de polinomios como*

$$(f + g)(x) = (a_n + b_n)x^n + \cdots + (a_1 + b_1)x + (a_0 + b_0).$$

La operación se realiza coeficiente a coeficiente en K, *extendiendo los coeficientes faltantes con ceros si es necesario.*

De manera similar, la ***resta*** *de polinomios se define como*

$$(f - g)(x) = (a_n - b_n)x^n + \cdots + (a_1 - b_1)x + (a_0 - b_0).$$

Definición 2.2.3. *La **multiplicación** de polinomios en $K[x]$ se define como*

$$(f \cdot g)(x) = \sum_{i=0}^{n+m} c_i x^i,$$

donde los coeficientes c_i se obtienen mediante

$$c_i = \sum_{j=0}^{i} a_j b_{i-j}, \quad \text{con } a_j = 0 \text{ si } j > n, \text{ y } b_{i-j} = 0 \text{ si } i - j > m.$$

En otras palabras, cada coeficiente de $f(x)$ se multiplica con cada coeficiente de $g(x)$ y se suman los términos de igual grado.

Proposición 2.2.1. *Si K es un cuerpo, entonces $K[x]$ es un dominio de ideales principales.*

Teorema 2.2.1 (División euclidiana de polinomios)**.** *Dados $f(x), g(x) \in K[x]$ con $g(x) \neq 0$, existen únicos $q(x), r(x) \in K[x]$ tales que*

$$f(x) = g(x)q(x) + r(x), \quad \deg(r) < \deg(g).$$

Definición 2.2.4. *El **máximo común divisor** de $f, g \in K[x]$ es el polinomio mónico de mayor grado que divide a ambos. Lo denotamos por MCD(f,g).*

Teorema 2.2.2 (Algoritmo de Euclides para polinomios)**.** *Se puede calcular el MCD de dos polinomios usando la división con resto iterativamente.*

Teorema 2.2.3 (Identidad de Bézout)**.** *Para todo $f, g \in K[x]$ existen $a, b \in K[x]$ tales que*

$$af + bg = MCD(f, g).$$

Teorema 2.2.4 (Teorema de los restos chinos para polinomios)**.** *Si $f_1(x), \ldots, f_n(x)$ son polinomios coprimos dos a dos en $K[x]$, entonces el sistema*

$$F(x) \equiv a_i \mod f_i(x), \quad i = 1, \ldots, n$$

tiene una solución única en $K[x]$ módulo $f_1(x) \ldots f_n(x)$.

Definición 2.2.5. *Un polinomio $f(x) \in K[x]$ es **irreducible** en K si no puede factorizarse en polinomios de menor grado en $K[x]$.*

2.3. Cuerpos finitos y sus propiedades

A diferencia de los cuerpos infinitos (como $\mathbb{Q}$ o $\mathbb{R}$), la estructura de un cuerpo finito está completamente determinada por su cardinalidad.

Definición 2.3.1. *Un **cuerpo finito** es un cuerpo con un número finito de elementos.*

Definición 2.3.2. *Sea K un cuerpo. Consideremos el homomorfismo de anillos*

$$\varphi : \mathbb{Z} \to K, \quad n \mapsto n \cdot 1_K = \underbrace{1 + \cdots + 1}_{n \text{ veces}}.$$

*La **característica de** K se define como el generador del ideal* $\ker(\varphi)$.

Proposición 2.3.1. *La característica de un cuerpo K es 0 o un número primo p.*

Proposición 2.3.2. *Sea F un cuerpo finito. Entonces existe un primo p y un entero $m \geq 1$ tales que*

$$|F| = p^m.$$

En particular, $\mathrm{char}(F) = p$.

Corolario 2.3.1. *Si un cuerpo finito F tiene $q = p^m$ elementos con p primo, entonces su característica es p.*

El ejemplo más básico de cuerpo finito proviene de la aritmética modular.

Proposición 2.3.3. *Para todo número primo p, el anillo $\mathbb{F}_p$ es un cuerpo finito.*

Construcción de cuerpos finitos

Sea $f(x) \in \mathbb{F}_p[x]$ un polinomio irreducible de grado n. El ideal $(f(x))$ es primo en $\mathbb{F}_p[x]$. Como $\mathbb{F}_p[x]$ es un dominio de ideales principales, este ideal es también maximal. Por tanto,

$$\mathbb{F}_p[x]/(f(x))$$

es un cuerpo con p^n elementos.

En general, un cuerpo (o anillo de división) no tiene por qué ser conmutativo. Sin embargo, todo cuerpo finito es conmutativo, por lo que en este trabajo consideraremos únicamente cuerpos conmutativos.

Observación 2 (Representación en base polinómica)**.** *La construcción como anillo cociente $\mathbb{F}_p[x]/(f(x))$ implica que $\mathbb{F}_{p^n}$ puede estructurarse como un espacio vectorial de dimensión n sobre $\mathbb{F}_p$.*

Si denotamos por β a una raíz del polinomio irreducible $f(x)$ de grado n (lo cual equivale a tomar la clase de equivalencia de x en el cociente), el conjunto

$$\{1, \beta, \beta^2, \ldots, \beta^{n-1}\}$$

forma una base de $\mathbb{F}_{p^n}$ sobre $\mathbb{F}_p$. En consecuencia, cualquier elemento $\alpha \in \mathbb{F}_{p^n}$ puede representarse de forma única como una combinación lineal de la forma:

$$\alpha = a_{n-1}\beta^{n-1} + \cdots + a_1\beta + a_0,$$

donde los coeficientes $a_j \in \mathbb{F}_p$. Esta es la representación computacional habitual de los cuerpos finitos.

Teorema 2.3.1 (Existencia y Unicidad)**.** *Sea p un número primo y $n \in \mathbb{N}$.*

1. *Existe un cuerpo finito con exactamente $q = p^n$ elementos, denotado por $\mathbb{F}_q$ (o $GF(q)$).*

2. *Cualesquiera dos cuerpos finitos con el mismo número de elementos son isomorfos (se dicen ."equivalentes").*

Grupo multiplicativo

Definición 2.3.3. *El conjunto de elementos no nulos de $\mathbb{F}_q$, denotado por $\mathbb{F}_q^*$, forma un grupo abeliano bajo la multiplicación.*

Teorema 2.3.2 (Ciclicidad de $\mathbb{F}_q^*$)**.** *El grupo multiplicativo $\mathbb{F}_q^*$ es cíclico. Es decir, existe un elemento $\alpha \in \mathbb{F}_q^*$ tal que*

$$\mathbb{F}_q^* = \{1, \alpha, \alpha^2, \ldots, \alpha^{q-2}\}.$$

Definición 2.3.4. *Un generador del grupo cíclico $\mathbb{F}_q^*$ se denomina **elemento primitivo** de $\mathbb{F}_q$.*

La existencia de elementos primitivos permite transformar costosas multiplicaciones entre elementos de $\mathbb{F}_q$ en simples sumas de exponentes módulo $q-1$ si expresamos cada elemento como una potencia del elemento primitivo. De este modo se simplifican las multiplicaciones de polinomios en cuerpos finitos.

Definición 2.3.5. *Sea $\alpha \in \mathbb{F}_q^*$. El **orden** de α es el menor entero $d > 0$ tal que $\alpha^d = 1$.*

Definición 2.3.6. *Si F es un subcuerpo de K, se dice que K/F es una **extensión de cuerpos** y su grado es $[K : F]$.*

2.4. Polinomios primitivos e irreducibilidad

Como el grupo multiplicativo de un cuerpo finito es cíclico, tiene sentido estudiar polinomios irreducibles cuyas raíces generen dicho grupo.

Definición 2.4.1. *Un polinomio irreducible es **primitivo** si alguna de sus raíces es un generador de $\mathbb{F}_{q^n}^*$.*

Definición 2.4.2. *Para un elemento $\alpha \in \mathbb{F}_{q^n}$, el **polinomio mínimo** sobre $\mathbb{F}_q$ es el polinomio irreducible de menor grado que tiene α como raíz.*

El siguiente criterio da una condición suficiente para comprobar la irreducibilidad sobre $\mathbb{Z}$ de un polinomio con coeficientes enteros:

Teorema 2.4.1 (Criterio de Eisenstein)**.** *Sea*

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0 \in \mathbb{Z}[x].$$

Supongamos que existe un número primo p tal que se cumplen las siguientes condiciones:

1. $p \mid a_i$ *para todo* $i = 0, 1, \ldots, n-1$,
2. $p \nmid a_n$,
3. $p^2 \nmid a_0$.

Entonces el polinomio $f(x)$ es irreducible en $\mathbb{Z}[x]$ (y por tanto también en $\mathbb{Q}[x]$).

Sin embargo, nosotros necesitaremos un criterio de irreducibilidad para el caso de cuerpos finitos, ya que por ejemplo en característica $p = 2$ el polinomio $x^2 + 1 = (x + 1)^2$ pero sobre $\mathbb{Z}$ es irreducible.

Así, para comprobar si un polinomio sobre un cuerpo finito es irreducible, necesitamos el siguiente criterio, publicado en [9] para cuerpos primos, pero que según [7] funciona en general para cualquier cuerpo finito. Nosotros solo necesitaremos la versión original:

Teorema 2.4.2 (Criterio de Rabin)**.** *Sea $f(x)$ un polinomio de grado n sobre $\mathbb{F}_p$. Entonces $f(x)$ es irreducible sobre $\mathbb{F}_p$ si y solo si:*

(1) $f(x)$ es un divisor de $x^{p^n} - x$, y

(2) $mcd(f(x), x^{p^{n/q}} - x) = 1$, para todo divisor primo q de n.

Demostración. Si $f(x)$ es irreducible y de grado n, todas sus raíces α tienen multiplicidad 1, porque las extensiones de cuerpos finitos son separables, y su orden es un divisor de n. Por lo tanto, las raíces están todas en $\mathbb{F}_{p^n}$, y verifican la ecuación

$$\alpha^{p^n} - \alpha = 0$$

En consecuencia, $(x - \alpha)$ divide a $x^{p^n} - x$ para toda raíz de $f(x)$ y se tiene el apartado (1). El apartado (2) es consecuencia de que si $f(x)$ es irreducible de grado n, entonces no tiene ninguna raíz en una extensión $\mathbb{F}_{2^m}$ con $m < n$.

Recíprocamente, supongamos que se verifican las condiciones (1) y (2). La condición (1) implica que todas las raíces de $f(x)$ están en la extensión $\mathbb{F}_{p^n}$.

Supongamos, por reducción al absurdo, que $f(x)$ tiene un factor irreducible $g(x)$, de grado $m < n$. Entonces toda las raíces de $g(x)$ están en la extensión $\mathbb{F}_{p^m}$, y una de esas raíces genera dicha extensión sobre el cuerpo primo $\mathbb{F}_p$. En consecuencia $\mathbb{F}_{p^m}$ es un subcuerpo de $\mathbb{F}_{p^n}$, y en particular $m|n$. Pero entonces m divide a n/q para algún factor primo q de n, con lo que $mcd(f(x), x^{p^{n/q}} - x)$ es divisible por $g(x)$, contradiciendo la condición (2).

□

Teorema 2.4.3 (Número de polinomios irreducibles y primitivos)**.** *La cantidad de polinomios primitivos de grado n en $\mathbb{F}_q[x]$ está dada por*

$$\frac{\varphi(q^n - 1)}{n},$$

donde φ es la función de Euler.

Teorema 2.4.4 (Propiedad de los polinomios primitivos)**.** *Si $f(x)$ es un polinomio primitivo, cualquier raíz α de $f(x)$ es un generador del grupo multiplicativo $\mathbb{F}_{q^n}^*$*

Sabemos, por tanto, que los polinomios primitivos existen y que sus raíces generan el grupo multiplicativo. Pero para verificar computacionalmente si un polinomio es primitivo, necesitamos un criterio práctico. El siguiente resultado es un test de primitividad que nos proporciona:

Proposición 2.4.1 (Test de primitividad)**.** *Sea $f(x) \in \mathbb{F}_p[x]$ un polinomio irreducible de grado n, y sea α una raíz de f en $\mathbb{F}_{p^n}$. Entonces f es primitivo (es decir, α genera $\mathbb{F}_{p^n}^*$) si y solo si*

$$\alpha^{(p^n-1)/q} \neq 1$$

para todo primo q que divida a $p^n - 1$.

Demostración. El elemento α genera $\mathbb{F}_{p^n}^*$ si y solo si su orden es $p^n - 1$. Por propiedades de grupos cíclicos, un elemento de orden d en un grupo cíclico de orden m satisface $d = m$ si y solo si $\alpha^{m/q} \neq 1$ para todo primo $q \mid m$. □

Este test requiere conocer los factores primos de $p^n - 1$, lo cual puede ser muy laborioso para valores grandes (y no tan grandes si p es grande) de n. En el Capítulo 4 discutimos las estrategias empleadas para obtener esta factorización.

2.5. Teoría de Galois en cuerpos finitos

Definición 2.5.1. *El **cuerpo de descomposición** es el mínimo cuerpo donde un polinomio se descompone completamente en factores lineales.*

Teorema 2.5.1. *Sea p un número primo y $n \in \mathbb{N}$. El cuerpo finito $\mathbb{F}_q$ es el **cuerpo de descomposición** del polinomio $x^{p^n} - x$ sobre $\mathbb{F}_p$.*

Este teorema es fundamental para nuestro trabajo, pues garantiza que el cuerpo finito $\mathbb{F}_q$ es único salvo isomorfismo. Esto significa que la estructura algebraica subyacente es siempre la misma, independientemente de cómo se construya. Sin embargo, desde el punto de vista computacional, existen diferentes representaciones explícitas (tablas de operaciones) dependiendo del polinomio irreducible elegido para definir el cuerpo. Los **polinomios de Conway** surgen precisamente para estandarizar esta elección, fijando una representación canónica común para todos los sistemas.

Definición 2.5.2. *Sea $\mathbb{F}_{q^n}$ una extensión de $\mathbb{F}_q$. Se llama **automorfismo de Frobenius** a la aplicación:*

$$\sigma : \mathbb{F}_{q^n} \to \mathbb{F}_{q^n}, \quad \sigma(x) = x^q.$$

*Esta aplicación es un elemento del **grupo de Galois** $\mathrm{Gal}(\mathbb{F}_{q^n}/\mathbb{F}_q)$ y, de hecho, es un generador del mismo; es decir:*

$$\mathrm{Gal}(\mathbb{F}_{q^n}/\mathbb{F}_q) = \langle \sigma \rangle \cong \mathbb{Z}/n\mathbb{Z}.$$

Teorema 2.5.2 (Acción transitiva del grupo de Galois sobre las raíces de polinomios irreducibles)**.** *Si $f(x)$ es irreducible en $F_q[x]$ y tiene n raíces en $\mathbb{F}_{q^n}$, el grupo de Galois actúa transitivamente sobre ellas.*

2.6. Resultados algorítmicos sobre cuerpos finitos

Existen algoritmos fundamentales para la implementación del cálculo de polinomios de Conway. Recogemos aquí los tres que utilizaremos directamente.

2.6.1. Test de irreducibilidad de Ben-Or

El criterio de Rabin proporciona una condición necesaria y suficiente para la irreducibilidad, pero su formulación directa requiere calcular $\gcd(f(x), x^{p^{n/q}} - x)$ para cada divisor primo q de n. El polinomio $x^{p^{n/q}} - x$ tiene grado $p^{n/q}$, que puede ser extremadamente grande, lo que hace inviable construirlo explícitamente.

Ben-Or [1] observó que no es necesario construir estos polinomios: basta con calcular la *clase* de x^{p^d} en el anillo cociente $\mathbb{F}_p[x]/(f(x))$ mediante exponenciación modular, y comprobar la condición de coprimalidad directamente en el cociente. Esto da lugar al siguiente algoritmo:

Teorema 2.6.1 (Test de irreducibilidad de Ben-Or [1])**.** *Sea $f(x) \in \mathbb{F}_p[x]$ un polinomio mónico de grado n con $f(0) \neq 0$. Entonces f es irreducible sobre $\mathbb{F}_p$ si y solo si se cumplen las dos condiciones siguientes:*

1. $x^{p^n} \equiv x \pmod{f(x)}$,
2. $\gcd\Big(x^{p^{n/q}} - x,\ f(x)\Big) = 1$ *para todo primo q que divida a n.*

Ambas condiciones se evalúan en el anillo $\mathbb{F}_p[x]/(f(x))$, donde los polinomios tienen grado menor que n.

Demostración. Las condiciones son equivalentes a las del Criterio de Rabin. La condición (1) equivale a que $f(x)$ divida a $x^{p^n} - x$, y la condición (2) es idéntica. La aportación de Ben-Or [1] no es una nueva caracterización de la irreducibilidad, sino la observación de que ambas condiciones pueden evaluarse en el anillo cociente $\mathbb{F}_p[x]/(f(x))$, donde todos los polinomios tienen grado menor que n, evitando construir el polinomio $x^{p^n} - x$ de grado p^n. □

La ventaja computacional es notable: la condición (1) se verifica calculando $x^{p^n} \bmod f(x)$ mediante n aplicaciones sucesivas del Frobenius $x \mapsto x^p \bmod f(x)$, cada una de las cuales es una exponenciación modular de coste $O(n^2 \log p)$ operaciones en $\mathbb{F}_p$. La condición (2) requiere s cálculos análogos (uno por cada primo que divida a n) más un GCD de polinomios de grado menor que n. El coste total es $O(s \cdot n^2 \log p + n^2)$, frente al coste de una factorización completa que puede ser $O(n^3)$ o superior.

Observación 3. *Cuando n es primo, la condición (2) se reduce a comprobar que $\gcd(x^p - x, f(x)) = 1$, lo cual es trivialmente cierto si $\deg(f) > 1$ y $f(0) \neq 0$. En este caso,* ***basta con verificar la condición (1)****, lo que reduce el test a n aplicaciones del Frobenius.*

En nuestra implementación (Capítulo 4), utilizamos este test en las búsquedas exhaustivas en orden de Parker, donde se evalúan miles de candidatos y la diferencia de velocidad respecto a una factorización completa es significativa.

2.7. Extensiones, Normas y Compatibilidad

Esta sección explica la motivación principal de los polinomios de Conway: la compatibilidad entre subcuerpos.

2.7.1. Inclusión de Subcuerpos

Teorema 2.7.1 (Criterio de Subcuerpo)**.** *El cuerpo $\mathbb{F}_{p^m}$ es isomorfo a un subcuerpo de $\mathbb{F}_{p^n}$ si y solo si m divide a n ($m \mid n$).*

Lema 2.7.1. *Sean $a \geq 2$ un entero y m, n enteros positivos. Entonces si m divide a n se cumple que $(a^m - 1)$ divide a $(a^n - 1)$. En particular, para $a = p$, esto garantiza que el orden del grupo multiplicativo de un subcuerpo divide al orden del grupo multiplicativo del cuerpo que lo contiene.*

Demostración. Supongamos que $m \mid n$. Entonces existe un entero $k \geq 1$ tal que $n = mk$. Utilizando la identidad algebraica fundamental para la diferencia de potencias, podemos reescribir $a^n - 1$ como:

$$a^n - 1 = a^{mk} - 1 = (a^m)^k - 1.$$

Haciendo el cambio de variable $y = a^m$, obtenemos $y^k - 1$, que se factoriza de forma exacta como:

$$y^k - 1 = (y-1)(y^{k-1} + y^{k-2} + \cdots + y + 1).$$

Sustituyendo y por su valor original, llegamos a:

$$a^n - 1 = (a^m - 1)\left(a^{m(k-1)} + a^{m(k-2)} + \cdots + a^m + 1\right).$$

Como el segundo factor es un número entero, queda demostrado que $(a^m - 1)$ divide a $(a^n - 1)$. □

Ejemplo 2.7.1. *Para $n = 128 = 2^7$, la cadena de subcuerpos es lineal:*

$$\mathbb{F}_2 \subset \mathbb{F}_{2^2} \subset \mathbb{F}_{2^4} \subset \mathbb{F}_{2^8} \subset \mathbb{F}_{2^{16}} \subset \mathbb{F}_{2^{32}} \subset \mathbb{F}_{2^{64}} \subset \mathbb{F}_{2^{128}}.$$

El lema anterior nos asegura que esta torre de subcuerpos cumple también que los órdenes de sus respectivos grupos multiplicativos se dividen de manera exacta. Por ejemplo, al considerar la inclusión $\mathbb{F}_{2^4} \subset \mathbb{F}_{2^8}$, el lema garantiza que $(2^4 - 1)$ divide a $(2^8 - 1)$. En efecto, el grupo pequeño tiene 15 elementos y el grande 255, y se cumple que $255/15 = 17$.

Esta estructura simplifica la verificación de compatibilidad. Además, la divisibilidad exacta garantiza que el cálculo de la norma relativa entre estos cuerpos siempre tendrá exponentes enteros bien definidos.

2.7.2. Función Norma Relativa

Para conectar un cuerpo finito con sus subcuerpos utilizamos la función norma, definida a partir de la estructura de extensión de Galois.

Definición 2.7.1 (Norma relativa)**.** *Sea $E = \mathbb{F}_{q^n}$ y $K = \mathbb{F}_q$ con $K \subset E$. La **norma relativa** de un elemento $\alpha \in E$ sobre K se define como*

$$N_{E/K}(\alpha) = \prod_{\sigma \in \mathrm{Gal}(E/K)} \sigma(\alpha).$$

En el caso de cuerpos finitos, el grupo de Galois es cíclico y está generado por el automorfismo de Frobenius $\sigma(x) = x^q$. Por tanto, $\mathrm{Gal}(E/K) = \langle \sigma \rangle$ y tiene orden n, lo que permite obtener una expresión explícita para la norma.

Proposición 2.7.1 (Fórmula explícita de la norma en cuerpos finitos)**.** *Sea $E = \mathbb{F}_{q^n}$ una extensión de $K = \mathbb{F}_q$. Para todo $\alpha \in E$, la norma relativa viene dada por*

$$N_{E/K}(\alpha) = \alpha \cdot \alpha^q \cdot \alpha^{q^2} \cdots \alpha^{q^{n-1}} = \alpha^{1+q+q^2+\cdots+q^{n-1}} = \alpha^{\frac{q^n-1}{q-1}}.$$

Demostración. Los elementos de $\mathrm{Gal}(E/K) = \langle\sigma\rangle$ son $\{\mathrm{id}, \sigma, \sigma^2, \ldots, \sigma^{n-1}\}$, donde $\sigma^i(x) = x^{q^i}$. Sustituyendo en la definición:

$$N_{E/K}(\alpha) = \prod_{i=0}^{n-1} \sigma^i(\alpha) = \prod_{i=0}^{n-1} \alpha^{q^i} = \alpha^{\sum_{i=0}^{n-1} q^i}.$$

La suma geométrica vale $\sum_{i=0}^{n-1} q^i = \frac{q^n-1}{q-1}$, lo que completa la prueba. □

Proposición 2.7.2 (Propiedades de la norma)**.** *Sea $E = \mathbb{F}_{q^n}$ una extensión de $K = \mathbb{F}_q$. La norma relativa $N_{E/K}$ satisface las siguientes propiedades [5]:*

1. ***Multiplicatividad:*** *Para todo $\alpha, \beta \in E$,*

$$N_{E/K}(\alpha\beta) = N_{E/K}(\alpha) \cdot N_{E/K}(\beta).$$

2. ***Acción sobre el cuerpo base:*** *Para todo $a \in K$,*

$$N_{E/K}(a) = a^n.$$

3. ***Sobreyectividad:*** *La aplicación $N_{E/K} : E^* \to K^*$ es un homomorfismo de grupos sobreyectivo.*

4. ***Norma del cero:*** $N_{E/K}(0) = 0$.

5. ***Transitividad:*** *Si $K \subset L \subset E$ son extensiones finitas, entonces*

$$N_{E/K} = N_{L/K} \circ N_{E/L}.$$

Demostración.

1. Se sigue de que cada $\sigma \in \mathrm{Gal}(E/K)$ es un homomorfismo de anillos:

$$N_{E/K}(\alpha\beta) = \prod_\sigma \sigma(\alpha\beta) = \prod_\sigma \sigma(\alpha)\sigma(\beta) = \left(\prod_\sigma \sigma(\alpha)\right)\left(\prod_\sigma \sigma(\beta)\right).$$

2. Si $a \in K$, entonces $\sigma(a) = a$ para todo $\sigma \in \mathrm{Gal}(E/K)$, de donde $N_{E/K}(a) = a^n$.

3. La multiplicatividad implica que $N_{E/K}\big|_{E^*}$ es un homomorfismo de grupos. Para la sobreyectividad: sea α un generador de E^* (grupo cíclico de orden $q^n - 1$). Entonces $N_{E/K}(\alpha) = \alpha^{(q^n-1)/(q-1)}$ tiene orden

$$\frac{q^n-1}{\gcd\left(\frac{q^n-1}{q-1},\ q^n-1\right)} = \frac{q^n-1}{\frac{q^n-1}{q-1}} = q-1,$$

   por lo que genera todo K^*.

4. Inmediato: $N_{E/K}(0) = 0^{(q^n-1)/(q-1)} = 0$.

5. Sean $[E : L] = m$ y $[L : K] = r$, con $n = mr$. Para $\alpha \in E$:

$$N_{L/K}\big(N_{E/L}(\alpha)\big) = N_{L/K}\left(\alpha^{\frac{q^n-1}{q^r-1}}\right) = \left(\alpha^{\frac{q^n-1}{q^r-1}}\right)^{\frac{q^r-1}{q-1}} = \alpha^{\frac{q^n-1}{q-1}} = N_{E/K}(\alpha).$$

□

Ejemplo 2.7.2. *Consideremos la extensión $\mathbb{F}_{2^4}/\mathbb{F}_{2^2}$, donde $q = 4$ y $n = 2$. La norma de un elemento $\alpha \in \mathbb{F}_{2^4}$ sobre $\mathbb{F}_{2^2}$ es:*

$$N_{\mathbb{F}_{2^4}/\mathbb{F}_{2^2}}(\alpha) = \alpha^{\frac{4^2-1}{4-1}} = \alpha^5.$$

Si α es un elemento primitivo de $\mathbb{F}_{2^4}^*$ (de orden 15), su norma es α^5, que tiene orden

$$15/\gcd(5, 15) = 3 = 4 - 1$$

es decir, un elemento primitivo de $\mathbb{F}_{2^2}^*$.

2.7.3. Compatibilidad de Norma

La norma proporciona la conexión algebraica entre un cuerpo finito y sus subcuerpos que fundamenta la definición de los polinomios de Conway. La noción de compatibilidad por norma fue introducida por Conway [8] y formalizada por Scheerhorn [11], quien la denomina *norm-compatible polynomials*. Heath y Loehr [4] la reformulan en el marco de sistemas compatibles en grupos cíclicos.

Definición 2.7.2 (Compatibilidad de norma)**.** *Sea p primo. Un sistema de polinomios $\{f_n(x) \in \mathbb{F}_p[x] : n \geq 1\}$, donde cada f_n es un polinomio irreducible de grado n, se dice* ***compatible por norma*** *(o simplemente* ***compatible****) si para toda pareja de enteros positivos $m \mid n$ se cumple la siguiente condición:*

Si α_n es una raíz de $f_n(x)$ en $\mathbb{F}_{p^n}$, entonces

$$N_{\mathbb{F}_{p^n}/\mathbb{F}_{p^m}}(\alpha_n) \text{ es una raíz de } f_m(x).$$

De forma equivalente, si denotamos por α_m una raíz fija de f_m en $\mathbb{F}_{p^m}$, la condición de compatibilidad establece que

$$\alpha_n^{\frac{p^n-1}{p^m-1}} = \alpha_m^{p^j}$$

para algún $j \in \{0, 1, \ldots, m-1\}$, es decir, la norma es un conjugado de Galois de α_m.

Observación 4. *La condición de compatibilidad por norma tiene una interpretación natural en términos de los grupos multiplicativos. El exponente $\frac{p^n-1}{p^m-1}$ es exactamente el que proyecta el grupo cíclico $\mathbb{F}_{p^n}^*$ de orden $p^n - 1$ sobre su único subgrupo de orden $p^m - 1$, que se identifica con $\mathbb{F}_{p^m}^*$ [4]. La compatibilidad exige que las raíces de los polinomios del sistema se correspondan bajo estas proyecciones de manera coherente a lo largo de toda la torre de subcuerpos.*

Observación 5. *Por la transitividad de la norma (Proposición 2.7.2), basta verificar la condición de compatibilidad para los divisores propios maximales de n. Es decir, si $n = q_1^{e_1} \cdots q_s^{e_s}$ y $d_i = n/q_i$, entonces f_n es compatible con todos los f_m ($m \mid n$, $m < n$) si y solo si lo es con $f_{d_1}, \ldots, f_{d_s}$ [4, 10].*

Ejemplo 2.7.3. *Consideremos la torre $\mathbb{F}_2 \subset \mathbb{F}_{2^2} \subset \mathbb{F}_{2^4}$ y un sistema compatible $\{f_1, f_2, f_4\}$. Si α_4 es una raíz de f_4 en $\mathbb{F}_{2^4}$, la compatibilidad exige:*

1. $N_{\mathbb{F}_{2^4}/\mathbb{F}_{2^2}}(\alpha_4) = \alpha_4^{\frac{2^4-1}{2^2-1}} = \alpha_4^5$ *es raíz de* f_2.
2. $N_{\mathbb{F}_{2^4}/\mathbb{F}_2}(\alpha_4) = \alpha_4^{\frac{2^4-1}{2^1-1}} = \alpha_4^{15} = 1$ *es raíz de* $f_1(x) = x + 1$ *en* $\mathbb{F}_2$.
3. *Por la transitividad de la norma,* $N_{\mathbb{F}_{2^2}/\mathbb{F}_2}(\alpha_4^5) = (\alpha_4^5)^3 = \alpha_4^{15} = 1$ *también es raíz de* f_1*, lo cual es consistente.*

2.7.4. Función Traza Relativa

La traza es el análogo aditivo de la norma: mientras la norma es el producto de los conjugados de Galois, la traza es la suma. Junto con la norma, constituye una de las dos funciones fundamentales para describir extensiones de cuerpos finitos.

Definición 2.7.3 (Traza relativa)**.** *Sea $E = \mathbb{F}_{q^n}$ una extensión de $K = \mathbb{F}_q$. La* ***traza relativa*** *de un elemento $\alpha \in E$ sobre K se define como*

$$T_{E/K}(\alpha) = \sum_{\sigma \in \mathrm{Gal}(E/K)} \sigma(\alpha).$$

Proposición 2.7.3 (Fórmula explícita de la traza en cuerpos finitos)**.** *Para todo $\alpha \in \mathbb{F}_{q^n}$, la traza relativa sobre $\mathbb{F}_q$ viene dada por*

$$T_{\mathbb{F}_{q^n}/\mathbb{F}_q}(\alpha) = \alpha + \alpha^q + \alpha^{q^2} + \cdots + \alpha^{q^{n-1}}.$$

Demostración. Los elementos de $\mathrm{Gal}(\mathbb{F}_{q^n}/\mathbb{F}_q)$ son $\sigma^i(x) = x^{q^i}$ para $i = 0, \ldots, n-1$. Sustituyendo en la definición:

$$T_{E/K}(\alpha) = \sum_{i=0}^{n-1} \sigma^i(\alpha) = \sum_{i=0}^{n-1} \alpha^{q^i} = \alpha + \alpha^q + \alpha^{q^2} + \cdots + \alpha^{q^{n-1}}.$$

□

Proposición 2.7.4 (Propiedades de la traza)**.** *Sea $E = \mathbb{F}_{q^n}$ una extensión de $K = \mathbb{F}_q$. La traza relativa $T_{E/K}$ satisface [5]:*

1. ***$\mathbb{F}_q$-linealidad:*** *Para todo $\alpha, \beta \in E$ y todo $a \in K$,*

$$T_{E/K}(\alpha + \beta) = T_{E/K}(\alpha) + T_{E/K}(\beta), \qquad T_{E/K}(a\alpha) = a \cdot T_{E/K}(\alpha).$$

2. ***Acción sobre el cuerpo base:*** *Para todo $a \in K$,*

$$T_{E/K}(a) = n \cdot a.$$

   En particular, si $\mathrm{char}(K) = p$ *y $p \mid n$, entonces $T_{E/K}(a) = 0$ para todo $a \in K$.*

3. ***Sobreyectividad:*** *La aplicación $T_{E/K} : E \to K$ es sobreyectiva.*

4. ***Transitividad:*** *Si $K \subset L \subset E$ son extensiones finitas, entonces*

$$T_{E/K} = T_{L/K} \circ T_{E/L}.$$

Demostración. 1. Los automorfismos de Galois son homomorfismos de anillos (preservan la suma), y para $a \in K$ se tiene $\sigma(a) = a$:

$$T_{E/K}(a\alpha) = \sum_{\sigma} \sigma(a\alpha) = \sum_{\sigma} a\,\sigma(\alpha) = a \sum_{\sigma} \sigma(\alpha) = a\,T_{E/K}(\alpha).$$

La aditividad se demuestra de forma análoga.

2. Si $a \in K$, entonces $\sigma(a) = a$ para todo σ, luego $T_{E/K}(a) = \sum_{i=0}^{n-1} a = n \cdot a$.

3. La $\mathbb{F}_q$-linealidad implica que $T_{E/K}$ es una forma lineal del espacio vectorial E (dimensión n) sobre K. Su núcleo tiene dimensión a lo sumo $n-1$ (pues $T_{E/K}$ no es idénticamente nula: basta tomar un generador α de E sobre K, cuyos conjugados $\alpha, \alpha^q, \ldots, \alpha^{q^{n-1}}$ son las n raíces distintas del polinomio mínimo de α y no pueden sumarse a cero todos). Por tanto la imagen tiene dimensión al menos 1 sobre K, es decir, $\mathrm{Im}(T_{E/K}) = K$.

4. Sean $[E : L] = m$ y $[L : K] = r$, con $n = mr$. Para $\alpha \in E$:

$$T_{L/K}\big(T_{E/L}(\alpha)\big) = T_{L/K}\left(\sum_{i=0}^{m-1} \alpha^{q^{ri}}\right) = \sum_{j=0}^{r-1}\sum_{i=0}^{m-1} \alpha^{q^{ri+j}} = \sum_{k=0}^{n-1} \alpha^{q^k} = T_{E/K}(\alpha).$$

□

2.7.5. Compatibilidad de Traza

De manera análoga a la compatibilidad de norma (que opera en el grupo multiplicativo), se puede definir una noción de compatibilidad que opera en la estructura aditiva del cuerpo finito, utilizando la función traza en lugar de la norma.

Definición 2.7.4 (Compatibilidad de traza)**.** *Sea p primo. Un sistema de polinomios $\{g_n(x) \in \mathbb{F}_p[x] : n \geq 1\}$, donde cada g_n es un polinomio irreducible de grado n, se dice* ***compatible por traza*** *si para toda pareja de enteros positivos $m \mid n$ se cumple:*

Si β_n es una raíz de $g_n(x)$ en $\mathbb{F}_{p^n}$, entonces

$$T_{\mathbb{F}_{p^n}/\mathbb{F}_{p^m}}(\beta_n) \text{ es una raíz de } g_m(x).$$

Observación 6 (Dualidad multiplicativa-aditiva)**.** *La compatibilidad de norma y la compatibilidad de traza reflejan un paralelismo entre las estructuras multiplicativa y aditiva del cuerpo finito [11], que resumimos en la siguiente tabla:*

	Multiplicativa	***Aditiva***
Función de enlace	*Norma $N_{E/K}$ (producto)*	*Traza $T_{E/K}$ (suma)*
Generadores	*Elementos primitivos*	*Elementos normales*
Compatibilidad	*$N(\alpha_n)$ raíz de f_m*	*$T(\beta_n)$ raíz de g_m*

Un ***elemento normal*** *de $\mathbb{F}_{p^n}$ sobre $\mathbb{F}_p$ es un elemento β tal que $\{\beta, \beta^p, \beta^{p^2}, \ldots, \beta^{p^{n-1}}\}$ forma una base de $\mathbb{F}_{p^n}$ como espacio vectorial sobre $\mathbb{F}_p$, llamada* ***base normal****.*

Observación 7. *La compatibilidad de norma es la condición que aparece en la definición de los polinomios de Conway y la que emplearemos en el algoritmo de cálculo [4, 10]. La compatibilidad de traza, en cambio, resulta útil cuando se trabaja con bases normales en lugar de bases polinómicas [11], pero no interviene en la construcción de los polinomios de Conway.*

Capítulo 3

Polinomios de Conway

3.1. Definición y propiedades

Distintos polinomios irreducibles de grado n sobre $\mathbb{F}_p$ producen representaciones diferentes del cuerpo. Esto complica la compatibilidad entre extensiones. Los **polinomios de Conway** proporcionan una elección estandarizada de polinomio irreducible para cada par (p, n), garantizando compatibilidad entre torres de cuerpos finitos.

Definición 3.1.1 (Orden lexicográfico de Parker)**.** *Sean $f, g \in \mathbb{F}_p[x]$ dos polinomios mónicos de grado n,*

$$f(x) = x^n + a_{n-1}x^{n-1} + \cdots + a_1x + a_0, \qquad g(x) = x^n + b_{n-1}x^{n-1} + \cdots + b_1x + b_0.$$

Decimos que $f <_{\text{lex}} g$ si existe un índice i con $n-1 \geq i \geq 0$ tal que $a_j = b_j$ para todo $j > i$ y

$$(-1)^{n-i}\, a_i \;<\; (-1)^{n-i}\, b_i,$$

donde el orden en $\mathbb{F}_p$ es $0 < 1 < \cdots < p-1$.

En la práctica, este orden compara coeficientes de mayor a menor grado, alternando el sentido de la comparación: en las posiciones donde $n-i$ es par se usa el orden natural $(0 < 1 < \cdots < p-1)$, y donde $n-i$ es impar se usa el orden invertido. En característica 2, como $-1 = 1$, el factor $(-1)^{n-i}$ es siempre 1 y el orden de Parker coincide con el orden lexicográfico estándar.

Definición 3.1.2 (Polinomio de Conway)**.** *El **polinomio de Conway** de grado n sobre $\mathbb{F}_p$, denotado $C_{p,n}(x)$, es el polinomio mónico de grado n que satisface:*

1. ***Primitividad:*** *$C_{p,n}(x)$ es primitivo sobre $\mathbb{F}_p$, es decir, cualquier raíz suya genera el grupo multiplicativo $\mathbb{F}_{p^n}^*$.*

2. ***Compatibilidad:*** *Para todo divisor propio $m \mid n$, si α es una raíz de $C_{p,n}(x)$ en $\mathbb{F}_{p^n}$, entonces*

$$N_{\mathbb{F}_{p^n}/\mathbb{F}_{p^m}}(\alpha) = \alpha^{\frac{p^n-1}{p^m-1}}$$

es raíz de $C_{p,m}(x)$. Equivalentemente,

$$C_{p,n}(x) \mid C_{p,m}\left(x^{\frac{p^n-1}{p^m-1}}\right).$$

3. ***Minimalidad lexicográfica:*** *Entre todos los polinomios mónicos de grado n que satisfacen las condiciones anteriores, $C_{p,n}$ es el mínimo respecto al orden $<_{\text{lex}}$ de Parker.*

Observación 8. *Las dos primeras condiciones garantizan que los subcuerpos encajen de forma coherente. La tercera es puramente convencional y solo sirve para que la elección sea única, de modo que distintos sistemas de álgebra computacional utilicen el mismo polinomio [4, 10].*

3.2. Existencia de los polinomios de Conway

No es evidente que exista un polinomio que sea simultáneamente primitivo y compatible con todos los polinomios de Conway de grados inferiores. La demostración de existencia se basa en la teoría de sistemas compatibles en grupos cíclicos desarrollada por Heath y Loehr [4], que a su vez utiliza el Teorema de los Restos Chinos.

Definición 3.2.1 (Sistema compatible de generadores)**.** *Sea C_k un grupo cíclico de orden k. Un* ***sistema de generadores compatibles*** *para C_k es una función parcial*

$$\Gamma : \operatorname{div}(k) \to C_k,$$

definida en un subconjunto $\operatorname{def}(\Gamma) \subseteq \operatorname{div}(k)$*, que satisface:*

1. $1 \in \operatorname{def}(\Gamma)$,
2. *si* $i \in \operatorname{def}(\Gamma)$*, entonces* $\operatorname{ord}(\Gamma(i)) = i$,
3. *si* $i \in \operatorname{def}(\Gamma)$ *y* $j \mid i$*, entonces* $j \in \operatorname{def}(\Gamma)$ *y* $\Gamma(i)^{i/j} = \Gamma(j)$.

Si $\operatorname{def}(\Gamma) = \operatorname{div}(k)$*, el sistema se dice* ***completo****.*

Teorema 3.2.1 (Extensión de sistemas compatibles [4])**.** *Todo sistema de generadores compatibles para un grupo cíclico C_k puede extenderse a un sistema completo.*

La demostración de este teorema está basada en el Teorema 3 y Lema 4 de [4].

Demostración. Procedemos por inducción sobre el número de divisores de k que faltan por definir. Si $\operatorname{def}(\Gamma) = \operatorname{div}(k)$, no hay nada que demostrar. En caso contrario, sea s el menor divisor de k tal que $s \notin \operatorname{def}(\Gamma)$. Demostraremos que Γ puede extenderse a s, es decir, que existe $g \in C_k$ con $\operatorname{ord}(g) = s$ y $g^{s/j} = \Gamma(j)$ para todo $j \in \operatorname{def}(\Gamma)$ con $j \mid s$.

Escribimos $s = q_1^{a_1} \cdots q_r^{a_r}$ y distinguimos tres casos.

Caso 1: s es primo. Como s es el menor divisor no definido y $1 \in \operatorname{def}(\Gamma)$, los únicos divisores de s en $\operatorname{def}(\Gamma)$ son $\{1\}$. La condición de compatibilidad se reduce a $g^s = \Gamma(1) = e$ (la identidad), es decir, g debe tener orden que divida a s. Como s es primo, cualquier elemento de orden s en el subgrupo $C_s \leq C_k$ sirve, y hay exactamente $s - 1$ tales elementos.

Caso 2: $s = q^a$ es potencia de un solo primo. Por la minimalidad de s, todos los divisores propios de s (que son $1, q, q^2, \ldots, q^{a-1}$) están en $\operatorname{def}(\Gamma)$. En particular, $\Gamma(q^{a-1})$ está definido y tiene orden q^{a-1}. Necesitamos g con $\operatorname{ord}(g) = q^a$ y $g^q = \Gamma(q^{a-1})$. El subgrupo $C_{q^a} \leq C_k$ es cíclico de orden q^a, y en él el homomorfismo $\varphi : x \mapsto x^q$ envía los generadores de C_{q^a} (elementos de orden q^a) sobre los elementos de orden q^{a-1}. Como $\Gamma(q^{a-1})$ tiene orden q^{a-1}, es imagen de algún elemento de orden q^a bajo φ. Concretamente, si γ es un generador de C_{q^a} y $\Gamma(q^{a-1}) = \gamma^{q \cdot t}$

para algún t coprimo con q^{a-1}, entonces $g = \gamma^t$ satisface $g^q = \gamma^{qt} = \Gamma(q^{a-1})$ y $\mathrm{ord}(g) = q^a$ (pues $\mathrm{mcd}(t, q^a) = 1$). Hay exactamente q preimágenes, correspondientes a multiplicar g por las q raíces q-ésimas de la unidad en C_{q^a}.

Caso 3: s tiene al menos dos primos distintos. Escribimos $s = s_1 \cdot s_2$ con $s_1 = q_1^{a_1}$ y $s_2 = s/s_1$, de modo que $\mathrm{mcd}(s_1, s_2) = 1$. Por minimalidad de s, tanto s_1 como s_2 están en $\mathrm{def}(\Gamma)$. Aplicamos el Teorema de los Restos Chinos al grupo cíclico: como $\mathrm{mcd}(s_1, s_2) = 1$, el subgrupo $C_s \leq C_k$ se descompone como

$$C_s \cong C_{s_1} \times C_{s_2}.$$

Bajo este isomorfismo, definimos g como el único elemento cuya proyección sobre C_{s_1} es

$$\Gamma(s_1)^{s_1/s_1} = \Gamma(s_1)$$

elevado adecuadamente, y cuya proyección sobre C_{s_2} es $\Gamma(s_2)$ elevado adecuadamente. Formalmente, sean $u, v \in \mathbb{Z}$ tales que $u \cdot s_1 + v \cdot s_2 = 1$ (existen por Bézout). Definimos

$$g = \Gamma(s_1)^{v \cdot s_2} \cdot \Gamma(s_2)^{u \cdot s_1}.$$

Entonces $g^{s/s_1} = g^{s_2} = \Gamma(s_1)^{v \cdot s_2^2} \cdot \Gamma(s_2)^{u \cdot s_1 \cdot s_2}$. Como $\Gamma(s_2)^{s_2} = \Gamma(1) = e$ y $v \cdot s_2 \equiv 1 \pmod{s_1}$, se obtiene $g^{s_2} = \Gamma(s_1)$. Análogamente, $g^{s_1} = \Gamma(s_2)$. Para cualquier otro divisor $j \mid s$ con $j \in \mathrm{def}(\Gamma)$, la compatibilidad $g^{s/j} = \Gamma(j)$ se deduce por transitividad: $j \mid s_1$ o $j \mid s_2$ (o ambos), y la condición se hereda de la compatibilidad de $\Gamma(s_1)$ y $\Gamma(s_2)$ con $\Gamma(j)$.

Además, $\mathrm{ord}(g) = \mathrm{mcm}(\mathrm{ord}(\mathrm{proy}_{s_1}(g)), \mathrm{ord}(\mathrm{proy}_{s_2}(g))) = \mathrm{mcm}(s_1, s_2) = s$, puesto que $\mathrm{mcd}(s_1, s_2) = 1$.

La unicidad en este caso se sigue de la unicidad en el Teorema de los Restos Chinos.

En los tres casos, definimos $\Gamma(s) = g$. El sistema extendido sigue siendo compatible, pues g tiene orden s y cumple $g^{s/j} = \Gamma(j)$ para todo $j \mid s$ con $j \in \mathrm{def}(\Gamma)$. Iterando, se llega a un sistema completo. □

La conexión con los cuerpos finitos es directa: el grupo multiplicativo $\mathbb{F}_{p^n}^*$ es cíclico de orden $M_{p,n} = p^n - 1$, y los órdenes de los subgrupos correspondientes a los subcuerpos $\mathbb{F}_{p^m}$ $(m \mid n)$ son los divisores $M_{p,m} = p^m - 1$ de $M_{p,n}$.

Teorema 3.2.2 (Existencia de los polinomios de Conway [4])**.** *Para todo primo p y todo entero positivo n, el polinomio de Conway $C_{p,n}$ existe.*

Demostración. Por inducción sobre n. Para $n = 1$, el grupo $\mathbb{F}_p^*$ es cíclico de orden $p-1$; se define $C_{p,1}(x) = x - \gamma$, donde γ es el menor elemento primitivo de $\mathbb{F}_p$ (que existe porque $\mathbb{F}_p^*$ es cíclico).

Supongamos $n > 1$ y que $C_{p,i}$ está definido para todo $i < n$. Para cada divisor m de n con $m < n$, sea α_m una raíz de $C_{p,m}$ en $\mathbb{F}_{p^n}$. Estas raíces determinan un sistema (parcial) de generadores compatibles para $\mathbb{F}_{p^n}^*$, ya que la compatibilidad de los polinomios de Conway de grado menor garantiza la condición $\alpha_j^{M_{p,j}/M_{p,i}} = \alpha_i$ cuando $i \mid j$.

Por el Teorema 3.2.1, este sistema parcial se extiende a uno completo, lo que proporciona al menos un elemento primitivo α_n de $\mathbb{F}_{p^n}^*$ compatible con todas las raíces anteriores. Entre todos los polinomios mínimos de tales elementos, elegimos el lexicográficamente menor. Este es $C_{p,n}$. □

3.3. Reducción a divisores maximales

En la práctica, no es necesario verificar la compatibilidad con todos los divisores propios de n. Gracias a la transitividad de la norma (Proposición 2.7.2), basta comprobarla con los **divisores propios maximales**.

Proposición 3.3.1 (Reducción a divisores maximales)**.** *Sea $n = q_1^{e_1} \cdots q_s^{e_s}$ la factorización en primos de n, y sean $d_i = n/q_i$ los divisores propios maximales de n para $1 \leq i \leq s$. Entonces $C_{p,n}$ es compatible con todos los $C_{p,m}$ ($m \mid n$, $m < n$) si y solo si es compatible con $C_{p,d_1}, \ldots, C_{p,d_s}$.*

Demostración. Sea m un divisor propio cualquiera de n. Entonces m divide a algún d_i. Por definición, C_{p,d_i} es compatible con $C_{p,m}$ (hipótesis inductiva). Si además $C_{p,n}$ es compatible con C_{p,d_i}, es decir,

$$C_{p,n}(x) \mid C_{p,d_i}\left(x^{M_{p,n}/M_{p,d_i}}\right),$$

entonces, como $C_{p,d_i}(x) \mid C_{p,m}\left(x^{M_{p,d_i}/M_{p,m}}\right)$, se obtiene por sustitución:

$$C_{p,d_i}\left(x^{M_{p,n}/M_{p,d_i}}\right) \mid C_{p,m}\left(x^{M_{p,n}/M_{p,m}}\right).$$

Por transitividad de la divisibilidad, $C_{p,n}(x) \mid C_{p,m}\left(x^{M_{p,n}/M_{p,m}}\right)$. □

Esta reducción es fundamental para la eficiencia del algoritmo: en lugar de verificar la compatibilidad con todos los divisores de n (que pueden ser muchos), basta comprobarla con los s divisores maximales, donde s es el número de primos distintos que dividen a n.

3.4. El parámetro g: número de candidatos compatibles

Una vez reducida la condición de compatibilidad a los divisores maximales, el polinomio de Conway $C_{p,n}$ debe satisfacer

$$C_{p,n}(x) \mid \operatorname{mcd}_{1 \leq i \leq s} \left\{C_{p,d_i}(x^{m_i})\right\},$$

donde $m_i = M_{p,n}/M_{p,d_i} = (p^n - 1)/(p^{d_i} - 1)$ para cada i.

Definición 3.4.1. *Definimos el **parámetro de compatibilidad** como*

$$g = \operatorname{mcd}_{1 \leq i \leq s}\{m_i\} = \operatorname{mcd}_{1 \leq i \leq s}\left\{\frac{p^n - 1}{p^{d_i} - 1}\right\}.$$

Teorema 3.4.1 (Número de candidatos compatibles [4])**.** *El número de polinomios mónicos irreducibles de grado n sobre $\mathbb{F}_p$ que son compatibles con los polinomios de Conway $C_{p,d_1}, \ldots, C_{p,d_s}$ es exactamente g.*

La demostración de este teorema está basada en el Teorema 9 de [4].

Demostración. Sea $M = p^n - 1$ el orden del grupo cíclico $\mathbb{F}_{p^n}^*$, y sea γ un generador fijo de $\mathbb{F}_{p^n}^*$. Para cada $i = 1, \ldots, s$, la compatibilidad con C_{p,d_i} exige que α^{m_i} sea raíz de C_{p,d_i}, donde $m_i = (p^n - 1)/(p^{d_i} - 1)$.

Escribimos $\alpha = \gamma^a$ para algún $a \in \{1, \ldots, M\}$. Entonces $\alpha^{m_i} = \gamma^{a \cdot m_i}$. La raíz de C_{p,d_i} en $\mathbb{F}_{p^n}$ es un elemento de orden $p^{d_i} - 1 = M/m_i$, que podemos escribir como $\gamma^{m_i \cdot c_i}$ para algún c_i coprimo con M/m_i (fijado por el polinomio C_{p,d_i} ya conocido). La condición de compatibilidad se traduce en

$$\gamma^{a \cdot m_i} = \gamma^{m_i \cdot c_i},$$

es decir, $a \cdot m_i \equiv m_i \cdot c_i \pmod{M}$, lo que equivale a

$$a \equiv c_i \pmod{M/m_i}.$$

Por tanto, el conjunto de a compatibles con *todos* los divisores maximales es la intersección

$$S = \bigcap_{i=1}^{s} \{a \in \mathbb{Z}/M\mathbb{Z} : a \equiv c_i \pmod{M/m_i}\}.$$

Cada congruencia individual tiene exactamente m_i soluciones en $\mathbb{Z}/M\mathbb{Z}$ (pues $M/(M/m_i) = m_i$). Para contar las soluciones de la intersección, observamos que el sistema es compatible (por el Teorema 3.2.1, existe al menos un generador compatible) y que el conjunto de soluciones forma una clase lateral del subgrupo

$$H = \bigcap_{i=1}^{s} (M/m_i)\mathbb{Z}/M\mathbb{Z}.$$

El orden de H es $\mathrm{mcd}\{m_1, \ldots, m_s\} = g$ (pues en $\mathbb{Z}/M\mathbb{Z}$, la intersección de subgrupos $(M/m_i)\mathbb{Z}$ tiene índice $\mathrm{mcm}\{M/m_i\}$ y por tanto orden $M/\mathrm{mcm}\{M/m_i\} = \mathrm{mcd}\{m_i\} = g$). Por consiguiente, $|S| = |H| = g$.

Cada $a \in S$ determina un elemento $\alpha = \gamma^a$ compatible. Dos elementos α y α' dan el mismo polinomio mínimo si y solo si son conjugados bajo el Frobenius, es decir, si $\alpha' = \alpha^{p^j}$ para algún j. Esto equivale a $a' \equiv a \cdot p^j \pmod{M}$. Las órbitas del Frobenius sobre los a de orden M (que corresponden a polinomios primitivos) tienen tamaño exactamente n. Por el Teorema 7 de [4], cada una de las g soluciones produce un elemento primitivo (de orden M), y estas g soluciones se agrupan en g/n órbitas de Frobenius de tamaño n... pero esto contaría polinomios, no elementos.

Sin embargo, el enunciado cuenta los polinomios irreducibles compatibles, no los elementos. Como cada polinomio irreducible de grado n tiene exactamente n raíces (conjugadas bajo el Frobenius), y las g soluciones de S que son primitivas se reparten en órbitas de tamaño n, el número de polinomios es g si contamos cada factor irreducible del polinomio $r(x^g) = \mathrm{mcd}\{C_{p,d_i}(x^{m_i})\}$ que tiene grado n.

En efecto, el polinomio $r(z) = \mathrm{mcd}\{C_{p,d_i}(z^{m_i/g})\}$ tiene grado n y es irreducible (Teorema 8 de [4]). Cada raíz z_0 de $r(z)$ tiene exactamente g raíces g-ésimas en$\mathbb{F}_{p^n}$ (pues $g \mid M$ y el grupo es cíclico). Cada raíz g-ésima α satisface $\alpha^{m_i} = z_0^{m_i/g}$, que es raízde C_{p,d_i} por construcción. Los polinomios mínimos de estas g raíces son exactamente los g polinomios irreducibles compatibles. □

Este resultado es la clave de la eficiencia del algoritmo: la búsqueda exhaustiva tendría que recorrer p^n polinomios, mientras que el algoritmo de Heath-Loehr solo necesita examinar g candidatos. Cuando n es compuesto, g suele ser mucho menor que p^n.

Ejemplo 3.4.1. *Algunos valores de g para $p = 2$:*

n	*Factorización de* n	g	2^n
6	$2 \cdot 3$	3	64
10	$2 \cdot 5$	11	1 024
12	$2^2 \cdot 3$	13	4 096
24	$2^3 \cdot 3$	241	16 777 216
30	$2 \cdot 3 \cdot 5$	331	1 073 741 824

En todos los casos, $g \ll 2^n$*, lo que evidencia la ventaja del algoritmo frente a la fuerza bruta.*

3.5. Ejemplo: cálculo de $C_{2,6}$

Ilustramos el proceso completo con el cálculo de $C_{2,6}$, siguiendo el ejemplo de Heath y Loehr en [4].

Datos iniciales. $p = 2$, $n = 6 = 2 \cdot 3$, con $s = 2$ primos distintos. Los divisores propios maximales son $d_1 = 6/2 = 3$ y $d_2 = 6/3 = 2$, con polinomios de Conway conocidos:

$$C_{2,3}(x) = x^3 + x + 1, \qquad C_{2,2}(x) = x^2 + x + 1.$$

Exponentes y parámetro g**.** Calculamos

$$m_1 = \frac{2^6 - 1}{2^3 - 1} = \frac{63}{7} = 9, \qquad m_2 = \frac{2^6 - 1}{2^2 - 1} = \frac{63}{3} = 21, \qquad g = \mathrm{mcd}(9, 21) = 3.$$

Solo hay 3 candidatos compatibles.

Cambio de variable. Con la sustitución $z = x^g = x^3$, escribimos los polinomios de compatibilidad como

$$C_{2,3}(z^{m_1/g}) = C_{2,3}(z^3) = z^9 + z^3 + 1, \qquad C_{2,2}(z^{m_2/g}) = C_{2,2}(z^7) = z^{14} + z^7 + 1.$$

Cálculo del mcd. El máximo común divisor de estos dos polinomios es

$$r(z) = \mathrm{mcd}(z^9 + z^3 + 1,\ z^{14} + z^7 + 1) = z^6 + z^5 + z^4 + z^2 + 1,$$

que es irreducible de grado 6 sobre $\mathbb{F}_2$.

Factorización en la variable original. En la variable x, el polinomio $r(z) = r(x^3)$ se factoriza como

$$r(x^3) = (x^6 + x^4 + x^3 + x + 1)(x^6 + x^5 + 1)(x^6 + x^5 + x^2 + x + 1).$$

Cada raíz z_0 de $r(z)$ tiene exactamente tres raíces cúbicas en $\mathbb{F}_{2^6}^*$, lo que produce los tres polinomios candidatos.

Verificación de primitividad. Para $p = 2$ y $n = 6$, el grupo multiplicativo $\mathbb{F}_{2^6}^*$ tiene orden $63 = 3^2 \cdot 7$. Un polinomio de grado 6 es primitivo si y solo si para su raíz α se cumple $\alpha^{63/3} = \alpha^{21} \neq 1$ y $\alpha^{63/7} = \alpha^9 \neq 1$. Los tres candidatos resultan ser primitivos.

Selección lexicográfica. En característica 2, el orden de Parker coincide con el lexicográfico estándar. Comparando los vectores de coeficientes (de x^5 a x^0):

$$\begin{aligned} x^6 + x^4 + x^3 + x + 1 &\longleftrightarrow (0,\ 1,\ 1,\ 0,\ 1,\ 1), \\ x^6 + x^5 + 1 &\longleftrightarrow (1,\ 0,\ 0,\ 0,\ 0,\ 1), \\ x^6 + x^5 + x^2 + x + 1 &\longleftrightarrow (1,\ 0,\ 0,\ 1,\ 1,\ 1). \end{aligned}$$

El primero tiene coeficiente 0 en x^5, mientras que los otros dos tienen 1. Como $0 < 1$, el primero es el menor. Por tanto,

$$\boxed{C_{2,6}(x) = x^6 + x^4 + x^3 + x + 1.}$$

3.6. Descripción del algoritmo

El ejemplo anterior ilustra los pasos del **algoritmo basado en polinomios** de Heath y Loehr [4], que es el implementado en la librería `conway.lib`, en el apéndice B. Describimos aquí el procedimiento general.

Sea $n = q_1^{e_1} \cdots q_s^{e_s}$ la factorización de n, con divisores maximales $d_i = n/q_i$, exponentes $m_i = M_{p,n}/M_{p,d_i}$ y parámetro $g = \text{mcd}\{m_i\}$. El algoritmo distingue tres casos.

Caso $s \geq 2$ (el más favorable). Se introduce la variable $z = x^g$ y se calcula

$$r(z) = \text{mcd}_{1 \leq i \leq s} \left\{ C_{p,d_i}(z^{m_i/g}) \right\}.$$

Por el Teorema 9 de [4], $r(z)$ es un polinomio irreducible de grado n sobre $\mathbb{F}_p$. Se define $\mathbb{F}_{p^n} \cong \mathbb{F}_p[z]/(r(z))$ y se denota z_0 a la clase de z en este cociente (que es raíz de r). Se calculan las g raíces g-ésimas de z_0 en $\mathbb{F}_{p^n}$: si α es una de ellas y ζ es una raíz g-ésima primitiva de la unidad, todas son de la forma $\alpha\zeta^k$ con $0 \leq k < g$. Para cada raíz que sea primitiva, se calcula su polinomio mínimo sobre $\mathbb{F}_p$ y se selecciona el lexicográficamente menor.

Caso $s = 1$ ($n = q^e$ potencia de primo). Aquí $r(z) = C_{p,n/q}(z)$, que tiene grado $n/q < n$ y no proporciona directamente una representación de $\mathbb{F}_{p^n}$. Se busca un polinomio irreducible auxiliar de grado n, se encuentra una raíz de $C_{p,n/q}$ en $\mathbb{F}_{p^n}$ y se procede de forma análoga con las raíces g-ésimas.

Caso $n = 1$ (caso base). Se busca el menor $\gamma \in \{2, 3, \ldots, p-1\}$ que sea generador de $\mathbb{F}_p^*$ y se devuelve $C_{p,1}(x) = x - \gamma$.

Observación 9 (Complementariedad de los enfoques)**.** *El coste del algoritmo de Heath-Loehr está determinado por la necesidad de iterar sobre g candidatos, lo que lo hace extremadamente eficiente para grados compuestos con $s \geq 2$ (donde el cálculo del MCD desploma el valor de g). Sin embargo, cuando n es primo ($s = 1$), $g = (p^n - 1)/(p - 1) \approx p^{n-1}$, perdiendo toda su ventaja frente al tamaño total del espacio.*

En estos escenarios con grados primos, la búsqueda exhaustiva en orden de Parker resulta superior. Al no requerir la exploración de un subespacio predeterminado, la búsqueda de Parker solamente busca polinomios primitivos irreducibles y puede pausar en el primer candidato válido. Ambos enfoques son, por tanto, complementarios [4].

El caso de las potencias de primo ($n = q^e$) presenta un escenario intermedio. La reducción del espacio mediante la fórmula $g = (p^n - 1)/(p^{n/q} - 1)$ es drástica y permite resolver casos pequeños de forma eficiente; por ejemplo, para $C_{2,16}$ el espacio se reduce de $65\,536$ a solo $g = 257$ candidatos. No obstante, para potencias mayores (como $n = 49$), el valor de g vuelve a ser computacionalmente inmanejable, y más sin computación paralela.

Por este motivo, nuestra implementación establece un umbral de decisión basado en el orden de magnitud de g. Si el número de candidatos es inferior a 20 millones, el algoritmo procede mediante el método de raíces g-ésimas de Heath-Loehr. Si se supera este límite, el sistema aborta dicho procedimiento y realiza una búsqueda exhaustiva en orden lexicográfico de Parker. En este último caso, el algoritmo opera de forma análoga al caso primo, pero con la exigencia adicional de

verificar explícitamente la condición de compatibilidad con el polinomio de Conway del subcuerpo maximal, $C_{p,q^{e-1}}$, para cada candidato irreducible localizado. La mayoría de veces no encontrará una solución porque es una "lotería", que cuanto más tiempo esté ejecutando más posible es que lo encuentres, y esto ocurrirá si el polinomio de Conway resulta ser "pequeño"lexicográficamente.

Observación 10 (Primos de Mersenne)**.** *Un caso especialmente favorable se da cuando $p = 2$ y $2^n - 1$ es un primo de Mersenne (es decir $n = 2, 3, 5, 7, 13, 17, 19, 31, 61, 89, 107, 127, 521 \dots$). En este caso, $|\mathbb{F}_{2^n}^*| = 2^n - 1$ es primo, lo que implica que* todo *polinomio irreducible de grado n sobre $\mathbb{F}_2$ es automáticamente primitivo. El polinomio de Conway se reduce entonces al menor polinomio irreducible con término constante* 1 *en el orden de Parker, eliminando por completo la necesidad de un test de primitividad. Esto permite calcular polinomios de Conway de grados muy grandes, como veremos en el Capítulo 5. Es una idea ingeniosa que no se había utilizado anteriormente hasta donde hemos visto que permite encontrar los polinomios de grados más altos nunca calculados con un coste computacional realizable por ordenadores personales.*

3.7. Tabla de polinomios de Conway conocidos

Para referencia, recogemos los primeros polinomios de Conway sobre $\mathbb{F}_2$ y $\mathbb{F}_3$, tomados de la base de datos de Frank Lübeck [6].

n	$C_{2,n}(x)$
1	$x+1$
2	x^2+x+1
3	x^3+x+1
4	x^4+x+1
5	x^5+x^2+1
6	$x^6+x^4+x^3+x+1$
7	x^7+x+1
8	$x^8+x^4+x^3+x^2+1$
9	x^9+x+1
10	$x^{10}+x^6+x^5+x^3+x^2+x+1$

n	$C_{3,n}(x)$
1	$x+2$
2	x^2+2x+2
3	x^3+2x+1
4	x^4+2x^3+2
5	x^5+2x+1
6	$x^6+2x^4+x^2+2x+2$
7	x^7+2x^2+1
8	$x^8+2x^5+x^4+2x^2+2x+2$
9	$x^9+2x^3+2x^2+x+1$
10	$x^{10}+2x^6+2x^5+x^4+x+2$

Observación 11 (Convención de coeficientes)**.** *En la literatura, los polinomios de Conway sobre $\mathbb{F}_3$ aparecen a veces escritos con coeficientes negativos (por ejemplo, $x^2 - x - 1$ en [4]). Estos son equivalentes a los de la tabla anterior tras reducir módulo* 3*: así, $x^2 - x - 1 \equiv x^2 + 2x + 2$* (mód 3)*. Nuestra implementación en Singular [6] utiliza la convención con coeficientes negativos y la base de datos de Lübeck la de coeficientes positivos.*

Nota 3.7.1 (Aplicaciones)**.** *Los polinomios de Conway tienen varias aplicaciones:*

- *Fijan un polinomio irreducible* canónico *para cada cuerpo finito $\mathbb{F}_{p^n}$, lo que permite que distintos sistemas de álgebra computacional (GAP, Magma, Singular, SageMath) trabajen con la misma representación.*
- *Simplifican la construcción de embeddings entre subcuerpos: si $m \mid n$, la inclusión $\mathbb{F}_{p^m} \hookrightarrow \mathbb{F}_{p^n}$ viene dada directamente por la condición de compatibilidad, sin necesidad de resolver sistemas de ecuaciones.*

- *Son útiles en aplicaciones donde se necesitan extensiones sucesivas de cuerpos finitos, como en teoría de códigos algebraico-geométricos [10] o en el cálculo de caracteres de Brauer en teoría de representaciones [6].*

Capítulo 4

Algoritmo e implementación

En el capítulo anterior hemos definido los polinomios de Conway, demostrado su existencia y descrito el algoritmo de Heath-Loehr a nivel teórico. En este capítulo presentamos los detalles de nuestra implementación en la librería `conway.lib` para Singular, las optimizaciones incorporadas, y las decisiones de diseño que permiten calcular polinomios de Conway de grados que van desde los triviales hasta los miles (en el caso de exponentes de Mersenne).

La librería completa se incluye en el Apéndice B. Las instrucciones de instalación de Singular se encuentran en el Apéndice A.

4.1. Cuatro caminos según la estructura de n

La implementación distingue cuatro casos según la factorización $n = q_1^{e_1} \cdots q_s^{e_s}$, que se resumen en la siguiente tabla:

Caso	Condición	Método	Función
Base	$n = 1$	$C_{p,1}$	`conwayDeg1`
Compuesto	$s \geq 2$	Heath-Loehr (MCD + raíces g-ésimas)	`caseComposite`
Pot. primo	$s = 1$, $e \geq 2$	Heath-Loehr o exhaustiva según g	`casePrimePower`
Primo	$s = 1$, $e = 1$	Exhaustiva en orden de Parker	`casePrimeExhaust`

La selección del caso la realiza `conwayRec`, que además gestiona la recursión sobre los divisores maximales de n (calculando primero los C_{p,d_i} necesarios) y la caché en RAM para evitar volver a calcular dentro de una misma sesión.

A continuación describimos cada caso.

4.2. Caso compuesto ($s \geq 2$): el algoritmo de Heath-Loehr

Este es el caso ideal para reducir el número de candidatos. Cuando n tiene al menos dos factores primos distintos, el Teorema 3.4.1 garantiza que $r(z) = \mathrm{mcd}\{C_{p,d_i}(z^{m_i/g})\}$ es irreducible de grado n, y que solo hay g candidatos compatibles.

4.2.1. Construcción de $r(z)$

La primera fase del algoritmo construye $r(z)$. Cuando $s = 2$, nuestra implementación (`caseComposite`) incorpora una optimización importante descrita en la publicación de Heath-Loehr [4]: en lugar de calcular el MCD de los dos polinomios $C_{p,d_1}(z^{m_1/g})$ y $C_{p,d_2}(z^{m_2/g})$ (que pueden tener grados muy dispares), se factoriza el de menor grado y se filtran sus factores irreducibles de grado n por compatibilidad con el otro. Esto evita construir en memoria el polinomio grande. El umbral para activar esta optimización es que el grado del polinomio menor sea inferior a 50 000; para grados mayores o para $s \geq 3$, se calcula el MCD directamente.

4.2.2. Iteración sobre los g candidatos

Una vez obtenido $r(z)$, se representa $\mathbb{F}_{p^n} = \mathbb{F}_p[z]/(r(z))$ y se calculan la raíz g-ésima α de $z_0 = \overline{z}$ y una raíz primitiva g-ésima de la unidad ζ. El bucle principal (`iterateCandidates`) recorre los g candidatos $\beta_k = \alpha \cdot \zeta^k$ con las siguientes optimizaciones:

- **Multiplicación incremental**: $\beta_{k+1} = \beta_k \cdot \zeta$ en lugar de $\alpha \cdot \zeta^k$, reemplazando una exponenciación por una multiplicación (propuesta en [4]).
- **Test de primitividad primero**: se comprueba $\beta_k^{(p^n-1)/q} \neq 1$ para cada primo $q \mid p^n - 1$ antes de calcular el polinomio mínimo, que es más costoso.
- **Poda lexicográfica**: la función `minPolyPruned` aborta el cálculo del polinomio mínimo en cuanto detecta que va a ser peor que el mejor candidato actual en el orden de Parker.
- **Reducción modular precalculada**: la preparación del polinomio $r(z)$ para la reducción modular se realiza una sola vez al inicio del bucle y se reutiliza en todas las operaciones (mediante `fastPow` en lugar de `polyPow`).

4.2.3. Limitación por el orden lexicográfico

El algoritmo debe recorrer los g candidatos *completos* para garantizar la minimalidad lexicográfica. A diferencia de la búsqueda exhaustiva (que recorre polinomios en orden de Parker y puede parar al primer hit), aquí el índice k no tiene relación con el orden de Parker de los polinomios mínimos: la correspondencia $k \mapsto \mathrm{minpoly}(\alpha\zeta^k)$ es imprevisible. Por tanto, si $g > 2 \times 10^7$ (umbral de nuestra implementación), el método es impracticable y se recurre a la búsqueda exhaustiva en orden de Parker. Esta búsqueda exhaustiva, aunque sea sobre más polinomios candidatos, nos da la mínima oportunidad de que si el grado es bajo sí se calcule, que nos da más esperanza que la imposibilidad de completar todos los candidatos de g.

4.3. Caso potencia de primo ($s = 1$, $e \geq 2$)

Cuando $n = q^e$ con $e \geq 2$, el polinomio $r(z) = C_{p,n/q}(z)$ tiene grado $n/q < n$ y no proporciona una representación de $\mathbb{F}_{p^n}$. La función `casePrimePower` necesita encontrar un polinomio irreducible auxiliar de grado n (mediante `findIrred`) y localizar una raíz de $C_{p,n/q}$ en $\mathbb{F}_{p^n}$ (mediante `findRoot`).

Si g es manejable ($\leq 2 \times 10^7$), se procede como en el caso compuesto. Si no, se pasa a la búsqueda exhaustiva.

Ejemplo 4.3.1. *Para $n = 4 = 2^2$ con $p = 2$: $g = (2^4 - 1)/(2^2 - 1) = 5$. Cinco candidatos, instantáneo.*

Para $n = 128 = 2^7$ con $p = 2$: $g = 2^{64} + 1 \approx 1{,}8 \times 10^{19}$. Completamente inviable por el método de raíces; se recurre a búsqueda exhaustiva.

4.4. Caso primo ($s = 1$, $e = 1$): búsqueda exhaustiva

Cuando n es primo, $g = (p^n - 1)/(p - 1)$ es del orden de p^n y el método de Heath-Loehr pierde toda su ventaja. Nuestra implementación usa la función `casePrimeExhaust`, que recorre los polinomios mónicos de grado n en **orden de Parker exacto**, con el término constante fijado por compatibilidad con $C_{p,1}$, y se detiene en el primer candidato que sea irreducible, primitivo y compatible.

4.4.1. ¿Por qué funciona bien?

Para n primo, la única condición de compatibilidad es con $C_{p,1}$ (que solo fija el término constante). Por tanto, la condición que realmente filtra es la irreducibilidad más la primitividad. La densidad de polinomios primitivos irreducibles de grado n es $\varphi(p^n - 1)/(n \cdot p^n)$, que para $p = 2$ y n moderado es del orden de $1/(2n)$. Esto significa que en promedio basta probar $\sim 2n$ candidatos, es decir, unos pocos cientos para $n \leq 200$.

4.4.2. El cuello de botella: factorización de $p^n - 1$

El test de primitividad (Proposición 2.4.1) requiere los factores primos de $p^n - 1$. Para valores moderados de n, el comando `factor` del sistema operativo los obtiene en segundos. Para n mayor (digamos $n > 200$ en $p = 2$), $2^n - 1$ tiene más de 60 dígitos y `factor` puede tardar minutos u horas, o directamente no terminar.

Nuestra implementación emplea tres estrategias para manejar esta factorización:

1. **Test de primalidad Miller-Rabin** (`millerRabinBigint`): si $p^n - 1$ resulta ser primo, todo irreducible de grado n es primitivo y no hace falta factorizar. Este test es determinista para $p^n - 1 < 3{,}3 \times 10^{24}$ (usando 12 bases) y probabilístico pero extremadamente fiable para valores mayores.

2. **Tabla de exponentes de Mersenne** (`isMersenneExponent`): para $p = 2$, la lista de n tales que $2^n - 1$ es primo está tabulada. Si n aparece en esta lista, se salta la factorización directamente.

3. **Factorización externa**: para los demás casos, se delega al comando `factor` del sistema operativo mediante una llamada al shell. Si este falla (número demasiado grande), se usa factorización por divisiones sucesivas hasta un límite, obteniendo una factorización posiblemente parcial que, no obstante, produce un test de primitividad válido (puede dar falsos negativos pero nunca falsos positivos). Si ese candidato fuera el menor lexicográficamente, el algoritmo devolvería un polinomio compatible y primitivo pero no necesariamente el menor, es decir, no el polinomio de Conway. En la práctica, esto no ha ocurrido en ninguno de nuestros cálculos: todos los resultados obtenidos con factorización parcial coinciden con las tablas de Lübeck [6].

Observación 12. *Esta es la razón por la que los primos $n > 200$ son costosos: no porque haya que probar muchos candidatos (se prueban pocos), sino porque cada test de primitividad requiere factorizar $2^n - 1$, que es un número de $\sim n/3{,}3$ dígitos.*

4.5. Primos de Mersenne: un caso excepcional

Cuando $p = 2$ y n es un exponente de Mersenne, las tres condiciones del polinomio de Conway se simplifican drásticamente:

1. **Compatibilidad**: como n es primo, la única condición es con $C_{2,1} = x + 1$, que fija el término constante a 1.
2. **Primitividad**: como $2^n - 1$ es primo, todo irreducible de grado n es automáticamente primitivo (Observación 10).
3. **Minimalidad**: hay que encontrar el menor irreducible en el orden de Parker.

El polinomio de Conway se reduce, por tanto, al **menor polinomio irreducible mónico de grado n con término constante 1 en el orden de Parker**. No hay test de primitividad ni de compatibilidad; solo irreducibilidad.

4.5.1. Test de irreducibilidad de Ben-Or

Para este propósito, la función `factorize` de Singular resulta excesiva: calcula todos los factores irreducibles de un polinomio, cuando solo necesitamos saber si *es* irreducible (una pregunta de sí/no). El test de Ben-Or [1], implementado en nuestra función `isIrreducibleBenOr`, responde a esta pregunta de forma más eficiente.

El test se basa en el Criterio de Rabin y opera íntegramente en el anillo cociente $\mathbb{F}_p[x]/(f(x))$:

1. **Condición 1**: verificar que $x^{p^n} \equiv x \pmod{f(x)}$, calculando n aplicaciones sucesivas del Frobenius $x \mapsto x^p \bmod f$.
2. **Condición 2**: para cada primo $q \mid n$, verificar que $\mathrm{mcd}(x^{p^{n/q}} - x,\ f(x)) = 1$.

Para n primo, la condición 2 se reduce a $\mathrm{mcd}(x^p - x, f) = 1$, que es trivialmente cierta si $\deg(f) > 1$ y $f(0) \neq 0$. Por tanto, **para n primo basta con la condición 1**: n aplicaciones del Frobenius, cada una de coste $O(n^2)$ operaciones en $\mathbb{F}_2$. El coste total es $O(n^3)$ por candidato.

4.5.2. La función `conwayPolyMersenne`

Para los exponentes de Mersenne, hemos creado una función dedicada `conwayPolyMersenne` que recorre la lista de exponentes conocidos y, para cada uno, busca el menor irreducible con `menorIrredParker`. Esta función:

1. Construye candidatos en orden de Parker (en $\mathbb{F}_2$, coincide con el lexicográfico estándar).
2. Verifica irreducibilidad con `isIrreducibleBenOr` (sin `factorize`).
3. Se detiene en el primer irreducible encontrado: ese es $C_{2,n}$.

Observación 13. *Para n primo sobre $\mathbb{F}_2$, el primer irreducible aparece típicamente entre los primeros $\sim n$ candidatos, ya que la densidad de irreducibles de grado n es $\sim 1/n$. Sin embargo, el cálculo de $C_{2,4423}$ fue más laborioso de lo esperable: se probaron* 11 058 *candidatos hasta encontrar el adecuado, más del doble de la media teórica. Cada candidato requiere un test de Ben-Or con* 4423 *cuadrados de polinomios de grado $\sim$ 4423 sobre $\mathbb{F}_2$. Singular maneja eficientemente esta aritmética (usando representación binaria internamente), lo que hace factible el cálculo a pesar del número elevado de intentos.*

Observación 14 (Por qué $C_{2,4423}$ es calculable pero $C_{2,128}$ no)**.** *Puede parecer paradójico que podamos calcular $C_{2,4423}$ (un polinomio de grado enorme) pero no $C_{2,128}$. La explicación se debe a la estructura aritmética:*

- *$n = 4423$ es primo y $C_{2,4423} - 1$ es primo de Mersenne. Todo irreducible es primitivo, la compatibilidad es trivial, y el primer irreducible en orden de Parker es el Conway. Solo se necesita un test de irreducibilidad por candidato.*
- *$n = 128 = 2^7$ es potencia de primo. Hay $g = 2^{64} + 1 \approx 1{,}8 \times 10^{19}$ candidatos compatibles, $2^{128} - 1$ no es primo (tiene 7 factores primos), y el test de primitividad es costoso. Además, la búsqueda exhaustiva necesita comprobar compatibilidad con $C_{2,64}$ (evaluación en $\mathbb{F}_{2^{128}}$) para cada candidato, y la densidad de compatibles es ínfima ($g/2^{128} \approx 2^{-64}$).*

En definitiva, la dificultad de calcular un polinomio de Conway no depende del grado n sino del parámetro g, de la factorizabilidad de $p^n - 1$, y de la densidad de candidatos compatibles.

4.6. Funciones auxiliares

4.6.1. Test de primitividad

La función `isPrimitivePoly` implementa el test de la Proposición 2.4.1: un polinomio irreducible f de grado n es primitivo si y solo si $\alpha^{(p^n-1)/q} \neq 1$ para cada primo $q \mid p^n - 1$, donde $\alpha = \overline{x}$ en $\mathbb{F}_p[x]/(f)$. Cada comprobación es una exponenciación modular de coste $O(n^2 \log(p^n))$.

Incorpora tres atajos para evitar la factorización de $p^n - 1$:

- Si n es un exponente de Mersenne conocido para $p = 2$: devuelve `true` directamente.
- Si $p^n - 1$ es primo (Miller-Rabin): devuelve `true` directamente.
- Si $p^n - 1 < 2^{31}$: factoriza por divisiones sucesivas internamente.
- En otro caso: delega al comando `factor` del sistema operativo.

4.6.2. Comparación en orden de Parker

La función `isSmallerConway` implementa la Definición 3.1.1: compara los coeficientes de x^{n-1} hacia x^0, multiplicando por $(-1)^{n-i}$ y normalizando a $\{0, \ldots, p-1\}$. La función auxiliar `parkerCoef` realiza la conversión inversa: dado un índice de contador $k \in \{0, \ldots, p-1\}$ y una posición i, devuelve el coeficiente real que ocupa la posición k-ésima en el orden de Parker para esa posición. Esto es lo que permite recorrer candidatos en orden de Parker exacto en las búsquedas exhaustivas.

4.6.3. Extracción de raíces g-ésimas

La función `gthRoot` calcula α tal que $\alpha^g = z_0$ en $\mathbb{F}_{p^n}$. Distingue dos casos:

- Si $\operatorname{mcd}(g \operatorname{mód} \operatorname{ord}(z_0),\ \operatorname{ord}(z_0)) = 1$: calcula $\alpha = z_0^{g^{-1} \operatorname{mód} \operatorname{ord}(z_0)}$ por inversión modular (`modInverse`). Es instantáneo.
- En caso contrario: localiza un generador γ de $\mathbb{F}_{p^n}^*$ y recorre las potencias γ^{jg} hasta encontrar z_0. Más lento, pero siempre funciona.

4.6.4. Búsqueda de irreducibles

La función `findIrred` busca un polinomio irreducible de grado n sobre $\mathbb{F}_p$ (necesario para el caso $s = 1$). Emplea una estrategia progresiva:

1. Trinomios $x^n + x^i + a_0$: los más comunes, especialmente en característica 2.
2. Pentanomios $x^n + x^i + x^j + x^k + a_0$: para los pocos grados sin trinomios irreducibles.
3. Búsqueda exhaustiva: último recurso.

La irreducibilidad se verifica con `isIrreducibleBenOr` en lugar de `factorize`, lo que acelera considerablemente esta fase para grados grandes.

4.7. Implementación en Singular

La librería `conway.lib` se carga y utiliza del siguiente modo:

```
ring R = 2, x, dp;
LIB "conway.lib";
poly c6 = conwayPoly(6);
c6;                          // x6+x4+x3+x+1
```

El anillo base `R` fija la característica p y la variable x. La función principal `conwayPoly(n)` calcula $C_{p,n}$ recursivamente. Para los exponentes de Mersenne, se dispone además de:

```
conwayPolyMersenne();        // calcula todos los C_{2,n} de Mersenne
```

4.7.1. Estructura de la librería

La librería consta de diez secciones:

1. **Aritmética de enteros**: `mcdBigint`, `modInverse`, `factorizeSmall`, `millerRabinBigint`, `factorizeBigint`, `primeFactors`.
2. **Aritmética en $\mathbb{F}_{p^n}$**: `polyPow`, `fastPow`, `polyEval`, `minPoly`, `minPolyPruned`.
3. **Orden de Parker**: `isSmallerConway`, `parkerCoef`.
4. **Tests de primitividad y compatibilidad**: `isMersenneExponent`, `isPrimitivePoly`, `isCompatible`.

5. **Herramientas de campo finito**: `substPow`, `findGenerator`, `gthRoot`, `gthRootOfUnity`, `findIrred`, `findRoot`.
6. **Núcleo del algoritmo**: `caseComposite`, `casePrimePower`, `exhaustiveSearch`, `casePrimeExhaust`.
7. **Selección de candidatos**: `selectBest`, `selectBestWithIrr`, `iterateCandidates`.
8. **Entrada principal y recursión**: `conwayPoly`, `conwayRec`, `conwayDeg1`.
9. **Conteo de candidatos**: `candidatosConway`, `mostrarCandidatos`.
10. **Mersenne y test de irreducibilidad**: `conwayPolyMersenne`, `menorIrredParker`, `isIrreducibleBenOr`.

4.7.2. Representación de $\mathbb{F}_{p^n}$

Trabajamos en el anillo $\mathbb{F}_p[x]$ reduciendo módulo un polinomio irreducible $f(x)$ de grado n. Las operaciones se realizan con la función `reduce` de Singular. Cuando $s \geq 2$, el propio $r(z)$ sirve como polinomio irreducible y $z_0 = \texttt{var}(1)$ es su raíz (función `selectBest`). Cuando $s = 1$, se busca un irreducible auxiliar con `findIrred` y se localiza la raíz de $r(z)$ en $\mathbb{F}_{p^n}$ (función `selectBestWithIrr`).

4.7.3. Exponenciación modular

La operación que más se repite a lo largo del algoritmo es calcular α^e mód $f(x)$ en $\mathbb{F}_{p^n}$, donde e puede ser un número enorme (del orden de p^n). Hacerlo multiplicando α por sí mismo e veces sería inviable. En su lugar, usamos el método de *cuadrados repetidos* (*square-and-multiply*), que reduce el número de multiplicaciones de e a $\lfloor \log_2 e \rfloor$.

La idea es: se escribe el exponente e en binario y se recorre bit a bit de izquierda a derecha empezando por el segundo. En cada paso se eleva al cuadrado el acumulador (lo que equivale a duplicar el exponente), y cuando el bit actual es 1 se multiplica además por la base. Después de cada operación se reduce módulo $f(x)$ para mantener el grado acotado. Así, por ejemplo, $\alpha^{13} = \alpha^{1101_2}$ se calcula como:

$$\alpha \xrightarrow{\text{cuadrado}} \alpha^2 \xrightarrow{\times\alpha} \alpha^3 \xrightarrow{\text{cuadrado}} \alpha^6 \xrightarrow{\text{cuadrado}} \alpha^{12} \xrightarrow{\times\alpha} \alpha^{13},$$

con solo 5 operaciones en lugar de 12 multiplicaciones.

En nuestra librería, esta operación está implementada en dos variantes: `polyPow`, que prepara internamente el polinomio f para la reducción modular (mediante la función `std` de Singular), y `fastPow`, que recibe esta preparación como parámetro. La segunda se usa dentro de bucles como `iterateCandidates`, donde se realizan miles de exponenciaciones con el mismo polinomio f: prepararlo una sola vez al inicio del bucle en lugar de repetir la preparación en cada llamada supone un ahorro significativo.

4.7.4. Comparación con la implementación original de Heath y Loehr

Heath y Loehr [4] presentaron en 2004 los resultados de su implementación en Magma, ejecutada sobre un ordenador Sun Ultra Sparc 30. Su publicación recoge los tiempos de cálculo para diversos polinomios de Conway. La siguiente tabla compara algunos datos significativos:

$C_{p,n}$	g	Alcance
$C_{2,42}$	5 419	HL y `conway.lib`
$C_{2,48}$	65 281	HL y `conway.lib`
$C_{2,60}$	80 581	HL y `conway.lib`
$C_{2,56}$	15 790 321	Solo HL, tardaría unos 5-6 días en `conway.lib`
$C_{3,34}$	32 285 041	Solo HL, tardaría unos 10-11 días en `conway.lib`
$C_{2,211}$	—	Solo `conway.lib`
$C_{2,521}$	—	Solo `conway.lib`
$C_{2,4423}$	—	Solo `conway.lib`

Las tres últimas filas corresponden a casos que la implementación de Heath y Loehr no abordaba de forma óptima: grados primos grandes y exponentes de Mersenne. Para los grados compuestos con g moderado, ambas implementaciones son comparables en alcance, aunque los tiempos no son directamente comparables por la diferencia de hardware y software (usaron Magma).

La diferencia fundamental está en el alcance. Heath y Loehr se limitaron al algoritmo basado en polinomios, cuyo límite práctico se sitúa en $g \sim 10^8$. Nuestra implementación añade tres vías que ellos no consideraron: la búsqueda exhaustiva como fallback automático cuando g es inmanejable, el cálculo de grados primos mediante el test de Ben-Or, y la función dedicada para exponentes de Mersenne que elimina el test de primitividad. Estas tres adiciones son las que permiten obtener los ocho polinomios de Conway no registrados previamente. Además, dentro del propio bucle de Heath-Loehr, incorporamos optimizaciones no presentes en su implementación original: la poda lexicográfica en el cálculo del polinomio mínimo (que descarta candidatos a mitad de cómputo si ya son peores que el mejor conocido) y el precálculo de la reducción modular para evitar repetir esa preparación en cada una de las g iteraciones.

El algoritmo de Heath-Loehr sigue siendo el núcleo de nuestra implementación para el caso $s \geq 2$ con g manejable, y es el responsable de los cálculos más eficientes. Nuestras aportaciones no lo sustituyen sino que amplían el alcance del cálculo en el resto de casos.

4.8. Verificación de resultados

Un error en $C_{p,m}$ se propagaría a todos los grados múltiplos de m. Por ello, cada resultado se verifica por tres vías independientes:

4.8.1. Comprobación de primitividad

La función `isPrimitivePoly` verifica que $\alpha^{(p^n-1)/q} \neq 1$ para cada primo $q \mid p^n - 1$, donde $\alpha = \overline{x}$ en $\mathbb{F}_p[x]/(C_{p,n})$. Es independiente del método de cálculo.

4.8.2. Comprobación de compatibilidad

La función `isCompatible` calcula $\beta = \alpha^{(p^n-1)/(p^m-1)}$ y evalúa $C_{p,m}(\beta)$. Si el resultado es cero, la compatibilidad se cumple. Se realiza *a posteriori*.

4.8.3. Comparación con tablas conocidas

De manera manual para los polinomios resultantes de cada ejecución se ha contrastado el resultado con la tabla de Lübeck [6], y no se ha detectado ningún fallo en cientos de ejemplos diferentes.

Capítulo 5

Resultados computacionales

En este capítulo presentamos los resultados obtenidos con nuestra implementación en Singular de la librería `conway.lib`. Todos los cálculos se han realizado sobre un ordenador personal con procesador Intel Core i7, del que solo se aprovecha la frecuencia de un núcleo por el tipo de programación monohilo que utiliza Singular, ejecutando Singular 4.4.1 sobre Cygwin (Windows). Esto contrasta con los esfuerzos habituales en el cálculo de polinomios de Conway, que suelen recurrir a computación paralela en múltiples máquinas, como los descritos por Lübeck [6]. Los polinomios obtenidos se han verificado con la función `testConway` y contrastado con la base de datos de Frank Lübeck [6].

5.1. Polinomios de Conway sobre $\mathbb{F}_2$

Empezamos por la característica 2. Al ser la más utilizada en criptografía y teoría de códigos, es la más estudiada y la que nos ofrece más valores de referencia para comprobar que todo funciona correctamente.

5.1.1. Grados $n = 1, \ldots, 41$

Hemos calculado $C_{2,n}$ para todos los grados n de 1 a 41, incluyendo tanto grados primos (resueltos por búsqueda exhaustiva en orden de Parker) como grados compuestos (resueltos por el algoritmo de Heath-Loehr).. A modo ilustrativo, mostramos los primeros 15:

n	$C_{2,n}(x)$	Caso
1	$x+1$	base
2	x^2+x+1	$s=1$
3	x^3+x+1	$s=1$
4	x^4+x+1	$s=1$
5	x^5+x^2+1	$s=1$
6	$x^6+x^4+x^3+x+1$	$s=2$
7	x^7+x+1	$s=1$
8	$x^8+x^4+x^3+x^2+1$	$s=1$
9	x^9+x+1	$s=1$
10	$x^{10}+x^6+x^5+x^3+x^2+x+1$	$s=2$
11	$x^{11}+x^2+1$	$s=1$
12	$x^{12}+x^7+x^6+x^5+x^3+x+1$	$s=2$
13	$x^{13}+x^4+x^3+x+1$	$s=1$
14	$x^{14}+x^7+x^5+x^3+1$	$s=2$
15	$x^{15}+x^5+x^4+x^2+1$	$s=2$

La columna "Caso" indica si el algoritmo siguió la vía $s = 1$ (grado primo o potencia de primo, resuelto por búsqueda exhaustiva) o $s \geq 2$ (grado compuesto con al menos dos primos distintos, resuelto por Heath-Loehr). Todos los resultados coinciden exactamente con las tablas de Lübeck [6].

5.1.2. Grados compuestos mayores

Los grados compuestos son donde el algoritmo de Heath-Loehr muestra su verdadera ventaja. Hemos calculado también $C_{2,n}$ para grados como $n = 49$, 60 y 65:

n	Factorización	g	Tiempo aprox.	$C_{2,n}(x)$
30	$2 \cdot 3 \cdot 5$	331	$\sim 5\,\mathrm{s}$	(véase más abajo)
34	$2 \cdot 17$	131 071	$\sim 13\,\mathrm{min}$	(véase más abajo)
49	7^2	4 432 676 798 593	$\sim 1\,\mathrm{s}$	(búsqueda exhaustiva)
60	$2^2 \cdot 3 \cdot 5$	80 581	$\sim 10\,\mathrm{min}$	(véase más abajo)
65	$5 \cdot 13$	145 295 143 558 111	$\sim 3\,\mathrm{s}$	(búsqueda exhaustiva)

Los casos $n = 49$ y $n = 65$ son interesantes: a pesar de que g es enorme, la búsqueda exhaustiva en orden de Parker encuentra el polinomio de Conway en muy pocos intentos, porque la densidad de candidatos compatibles es alta.

Los polinomios de grado 30 y superior son demasiado largos para la tabla. Los listamos a continuación:

$C_{2,30}(x)$ ($g = 331$, $s = 3$):

$$x^{30}+x^{17}+x^{16}+x^{14}+x^{13}+x^{11}+x^7+x^5+x^3+x^2+x+1.$$

$C_{2,34}(x)$ ($g = 131\,071$, $s = 2$):

$$x^{34} + x^{16} + x^{15} + x^{13} + x^{11} + x^7 + x^5 + x^3 + 1.$$

$C_{2,60}(x)$ ($g = 80\,581$, $s = 3$):

$$\begin{aligned} x^{60} &+ x^{45} + x^{44} + x^{42} + x^{41} + x^{39} + x^{36} + x^{34} + x^{33} + x^{32} + x^{30} \\ &+ x^{26} + x^{25} + x^{22} + x^{19} + x^{17} + x^{12} + x^8 + x^5 + x^4 + x^3 + x^2 + 1. \end{aligned}$$

Observación 15. *El caso $n = 60$ es especialmente significativo. A pesar de que $2^{60} \approx 1{,}15 \times 10^{18}$ (lo que haría completamente inviable una búsqueda exhaustiva), el parámetro $g = 80\,581$ permite al algoritmo terminar en unos 8 minutos. Esto ilustra la ventaja del método: una reducción del espacio de búsqueda en un factor superior a 10^{13}.*

5.1.3. Grados primos hasta 211

Para grados primos, el algoritmo de Heath-Loehr no es aplicable ($s = 1$, g no optimizado). Nuestra implementación recurre entonces a la búsqueda exhaustiva en orden de Parker, que funciona bien porque para n primo la única condición de compatibilidad es con $C_{2,1}$ (que fija el término constante a 1), entonces el primer polinomio primitivo en orden lexicográfico de Parker (que para p=2 el orden lexicográfico estándar) será el polinomio de Conway, y la densidad de polinomios primitivos es suficientemente alta.

Hemos calculado $C_{2,n}$ para todos los primos $n \leq 211$. Todos los resultados coinciden con las tablas de Lübeck [6].

5.1.4. Polinomios de Conway para exponentes de Mersenne

Un caso especial particularmente favorable para nuestros algoritmos, porque los hemos preparado para este caso, se da cuando n es un **exponente de Mersenne**, es decir, cuando $2^n - 1$ es un número primo (un primo de Mersenne). En este caso, el grupo multiplicativo $\mathbb{F}_{2^n}^*$ tiene orden primo, lo que implica que *todo* polinomio irreducible de grado n sobre $\mathbb{F}_2$ es automáticamente primitivo. El polinomio de Conway se reduce entonces al menor polinomio irreducible de grado n con término constante 1 en el orden de Parker.

Esta simplificación, combinada con el test de irreducibilidad de Ben-Or, que evita la factorización completa y opera en tiempo $O(n^2)$ por candidato, permite calcular polinomios de Conway de grados que serían completamente inalcanzables por otros métodos.

Hemos calculado primeros ocho polinomios de Conway para exponentes de Mersenne no registrados anteriormente.

n	Primo de Mersenne $2^n - 1$	$C_{2,n}(x)$
521	$\sim 6{,}9 \times 10^{156}$	$x^{521} + x^9 + x^6 + x^5 + x^3 + x + 1$
607	$\sim 5{,}3 \times 10^{182}$	$x^{607} + x^9 + x^7 + x^6 + x^4 + x + 1$
1279	$\sim 1{,}0 \times 10^{385}$	$x^{1279} + x^{11} + x^9 + x^8 + x^5 + x^3 + x^2 + x + 1$
2203	$\sim 1{,}5 \times 10^{663}$	$x^{2203} + x^{11} + x^{10} + x^6 + x^4 + x + 1$
2281	$\sim 4{,}5 \times 10^{686}$	$x^{2281} + x^9 + x^8 + x^7 + x^6 + x^2 + 1$
3217	$\sim 2{,}6 \times 10^{969}$	$x^{3217} + x^{11} + x^{10} + x^9 + x^8 + x^6 + x^5 + x^4 + x^3 + x^2 + 1$
4253	$\sim 4{,}5 \times 10^{1281}$	$x^{4253} + x^{12} + x^{10} + x^7 + x^5 + x^4 + 1$
4423	$\sim 2{,}8 \times 10^{1332}$	$x^{4423} + x^{14} + x^{12} + x^{10} + x^9 + x^5 + x + 1$

Estos ocho polinomios son, hasta donde sabemos, **resultados nuevos** que no figuran en la base de datos de Lübeck [6] ni en las tablas internas de GAP o Magma. Los exponentes de Mersenne $n = 2, 3, 5, 7, 13, 17, 19, 31, 61, 89, 107$ y 127 sí aparecen en dichas bases de datos y nuestros resultados coinciden con los tabulados.

Observación 16. *Es destacable el reducido número de términos no nulos presentes en los polinomios obtenidos. Este hecho es una consecuencia de la interacción entre la estructura de los cuerpos finitos para exponentes de Mersenne y el orden de Parker. Al ser n un exponente de Mersenne, el orden del grupo multiplicativo $\mathbb{F}_{2^n}^*$, dado por $2^n - 1$, es primo. Esta propiedad garantiza que todo polinomio irreducible de grado n sobre $\mathbb{F}_2$ sea automáticamente primitivo.*

Dado que el algoritmo explora los candidatos siguiendo el orden lexicográfico de Parker, el primer candidato que supera el test de irreducibilidad es también primitivo y resulta ser el polinomio de Conway buscado.

5.2. Polinomios de Conway sobre $\mathbb{F}_3$, $\mathbb{F}_5$, $\mathbb{F}_7$, $\mathbb{F}_{11}$ y $\mathbb{F}_{13}$

Para primos $p > 2$, los cálculos son generalmente más costosos porque las operaciones en $\mathbb{F}_p[x]$ involucran coeficientes no binarios y los tamaños de los grupos $p^n - 1$ crecen más rápido. No obstante, nuestro programa ha podido procesar rangos amplios.

5.2.1. Característica $p = 3$

Hemos completado los cálculos para todos los grados de $n = 1$ a $n = 20$, pero se pueden calcular muchos más, como los primos hasta grado 100. Mostramos los diez primeros:

n	g	Tiempo aprox.	$C_{3,n}(x)$
1	—	$< 1\,\mathrm{s}$	$x+2$
2	4	$< 1\,\mathrm{s}$	x^2+2x+2
3	13	$< 1\,\mathrm{s}$	x^3+2x+1
4	10	$< 1\,\mathrm{s}$	x^4+2x^3+2
5	121	$< 1\,\mathrm{s}$	x^5+2x+1
6	7	$< 1\,\mathrm{s}$	$x^6+x^4+x^2+2x+2$
7	1093	$< 1\,\mathrm{s}$	x^7+2x^2+1
8	82	$< 1\,\mathrm{s}$	$x^8+2x^5+x^4+2x^2+2x+2$
9	757	$3\,\mathrm{s}$	$x^9+2x^3+2x^2+x+1$
10	61	$< 1\,\mathrm{s}$	$x^{10}+2x^6+2x^5+x^4+x+2$

5.2.2. Característica $p = 5$

Para $\mathbb{F}_5$ hemos calculado hasta el grado $n = 15$, aquí están los 9 primeros:

n	g	Tiempo aprox.	$C_{5,n}(x)$
1	—	$< 1\,\mathrm{s}$	$x+3$
2	6	$< 1\,\mathrm{s}$	x^2+4x+2
3	31	$< 1\,\mathrm{s}$	x^3+3x+3
4	26	$< 1\,\mathrm{s}$	x^4+4x^2+4x+2
5	781	$< 1\,\mathrm{s}$	x^5+4x+3
6	21	$< 1\,\mathrm{s}$	$x^6+x^4+4x^3+4x+2$
7	19 531	$< 1\,\mathrm{s}$	x^7+3x+3
8	626	$1\,\mathrm{s}$	$x^8+x^4+3x^2+4x+2$
9	15 751	$15\,\mathrm{s}$	x^9+2x^3+x+3
10	521	$2\,\mathrm{s}$	$x^{10}+3x^5+3x^4+2x^3+4x^2+x+2$

5.2.3. Característica $p = 7$

Para $\mathbb{F}_7$, grados 1 a 13:

n	g	Tiempo aprox.	$C_{7,n}(x)$
1	—	$< 1\,\mathrm{s}$	$x+4$
2	8	$< 1\,\mathrm{s}$	x^2+6x+3
3	57	$< 1\,\mathrm{s}$	x^3+6x^2+4
4	50	$< 1\,\mathrm{s}$	x^4+5x^2+4x+3
5	2801	$< 1\,\mathrm{s}$	x^5+x+4
6	43	$< 1\,\mathrm{s}$	$x^6+x^4+5x^3+4x^2+6x+3$
7	137 257	$< 1\,\mathrm{s}$	x^7+6x+4
8	2402	$7\,\mathrm{s}$	$x^8+4x^3+6x^2+2x+3$
9	2 101	$5\,\mathrm{s}$	$x^9+6x^4+x^3+6x+4$
10	117 993	$\sim 2\,\mathrm{min}$	$x^{10}+x^6+x^5+4x^4+x^3+2x^2+3x+3$

5.2.4. Características $p = 11$ y $p = 13$

Para esos dos primos, hemos calculado los once primeros de p=11 y los ocho primeros de p=13 aquí están:

p	n	g	Tiempo aprox.	$C_{p,n}(x)$
11	1	—	$< 1\,\mathrm{s}$	$x+9$
11	2	12	$< 1\,\mathrm{s}$	x^2+7x+2
11	3	133	$< 1\,\mathrm{s}$	x^3+2x+9
11	4	122	$< 1\,\mathrm{s}$	$x^4+8x^2+10x+2$
11	5	16 105	$< 1\,\mathrm{s}$	x^5+10x^2+9
11	6	111	$< 1\,\mathrm{s}$	$x^6+3x^4+4x^3+6x^2+7x+2$
11	7	1 948 717	$< 1\,\mathrm{s}$	x^7+4x+9
11	8	14 642	$1\,\mathrm{min}$	$x^8+7x^4+7x^3+x^2+7x+2$
11	9	1 772 893	$1\,\mathrm{h}$	x^9+9x^2+8x+9
11	10	13 421	$1\,\mathrm{min}$	$x^{10}+7x^5+8x^4+10x^3+6x^2+6x+2$
11	11	28 531 167 061	$< 1\,\mathrm{s}$	$x^{11}+10x+9$
13	1	—	$< 1\,\mathrm{s}$	$x+11$
13	2	14	$< 1\,\mathrm{s}$	$x^2+12x+2$
13	3	183	$< 1\,\mathrm{s}$	$x^3+2x+11$
13	4	170	$< 1\,\mathrm{s}$	$x^4+3x^2+12x+2$
13	5	30 941	$< 1\,\mathrm{s}$	$x^5+4x+11$
13	6	157	$< 1\,\mathrm{s}$	$x^6+10x^3+11x^2+11x+2$
13	7	5 229 043	$< 1\,\mathrm{s}$	$x^7+3x+11$
13	8	28 562	$3\,\mathrm{min}$	$x^8+8x^4+12x^3+2x^2+3x+2$

Podíamos haber calculado más polinomios de cada uno de esos valores p sin problema, pero no es especialmente interesante, esto es suficiente para comprobar que funcionan todos los tipos de algoritmo para distintas características.

5.3. Análisis del rendimiento

5.3.1. El caso compuesto: dependencia con g

Para grados con $s \geq 2$ primos distintos, donde se aplica el algoritmo de Heath-Loehr, el tiempo de ejecución depende fundamentalmente del parámetro g:

n	s	g	Tiempo	Observación
6	2	3	$< 1\,\mathrm{s}$	trivial
10	2	11	$< 1\,\mathrm{s}$	trivial
12	2	13	$< 1\,\mathrm{s}$	trivial
30	3	331	$\sim 5\,\mathrm{s}$	rápido
34	2	43 691	$\sim 4\,\mathrm{min}$	moderado
60	3	80 581	$\sim 8\,\mathrm{min}$	moderado

El caso $n = 34$ ($g \approx 1{,}3 \times 10^5$) tarda más que $n = 60$ ($g \approx 8 \times 10^4$) a pesar de tener un grado menor, porque g es mayor. Esto confirma que g, y no n ni p^n, es el factor determinante del rendimiento en el caso compuesto en el caso de Heath-Loehr.

5.3.2. El caso primo: el papel de la factorización de $p^n - 1$

Para grados primos, la búsqueda exhaustiva encuentra el primer candidato válido en pocos intentos. Sin embargo, el cuello de botella puede ser la **factorización de** $p^n - 1$, necesaria para el test de primitividad. Nuestra implementación emplea tres estrategias según el caso:

- Si $p^n - 1$ es primo (detectado por Miller-Rabin), todo irreducible es primitivo y no hace falta factorizar. Esto cubre los exponentes de Mersenne.
- Si $p^n - 1$ cabe en el rango del comando `factor` del sistema operativo ($\lesssim 40$ dígitos), la factorización es inmediata.
- Para valores mayores, se emplea factorización parcial por divisiones sucesivas, que puede no ser completa pero proporciona un test de primitividad válido (véase Capítulo 4).

5.3.3. Comparación con la búsqueda exhaustiva

Para grados compuestos, la ventaja del algoritmo de Heath-Loehr frente a la fuerza bruta es contundente:

n	g (Heath-Loehr)	$2^n/g$ (fuerza bruta)	Factor de mejora
6	3	21	$\times 7$
12	13	315	$\times 24$
30	331	3 243 933	$\times 9\,800$
34	131 071	131 073	$\times 1$
60	80 581	$\approx 1{,}4 \times 10^{13}$	$\times 1{,}8 \times 10^8$

Para $n = 60$, nuestro algoritmo es del orden de cien millones de veces más rápido que la fuerza bruta. El caso $n = 34$ ilustra la complementariedad: como $34 = 2 \times 17$ y 17 es grande, $g = 2^{17} - 1$ coincide con $2^n/g$, de modo que ambos enfoques son comparables.

5.3.4. Los exponentes de Mersenne: un caso límite favorable

Los resultados de la Sección 5.1.4 representan el extremo opuesto: grados enormes (n hasta 3217) donde el cálculo es posible gracias a la confluencia de tres factores:

1. La primalidad de $2^n - 1$ elimina el test de primitividad: todo irreducible de grado n es automáticamente primitivo.
2. Al ser n primo, la compatibilidad se reduce a una única condición trivial (el término constante igual a 1).
3. El test de irreducibilidad de Ben-Or (Teorema 2.6.1) opera en tiempo $O(n^2)$ por candidato, sin necesidad de factorizar el polinomio.

Esto muestra que el alcance de una implementación no depende solo del grado n, sino de la estructura aritmética de n y de $p^n - 1$.

5.4. Resumen

Hemos calculado como ejemplo polinomios de Conway para:

- $\mathbb{F}_2$: todos los grados de 1 a 41, todos los primos hasta 211, grados compuestos selectos (49, 60, 65...), y los exponentes de Mersenne 521, 607, 1279, 2203, 2281, 3217, 4253 y 4423.
- $\mathbb{F}_3$: grados 1 a 24.
- $\mathbb{F}_5$: grados 1 a 14.
- $\mathbb{F}_7$: grados 1 a 13.
- $\mathbb{F}_{11}$: grados 1 a 11.
- $\mathbb{F}_{13}$: grados 1 a 9.

Menos los polinomios que no habían sido calculados antes, todos los resultados han sido verificados por tres vías independientes: coincidencia con las tablas de Lübeck, primitividad del polinomio, y compatibilidad con los polinomios de grados divisores.

El mayor cálculo por el método de Heath-Loehr fue $C_{2,60}$ ($g = 80\,581$, ~ 10 minutos). El mayor cálculo por búsqueda exhaustiva con el test de Ben-Or fue $C_{2,4423}$, posible gracias a la primalidad de $2^{4423} - 1$. Ambos resultados confirman la viabilidad de nuestra implementación y la complementariedad de los distintos enfoques algorítmicos.

Capítulo 6

Conclusiones

6.1. Resumen de aportaciones

En este trabajo hemos explicado los fundamentos teóricos del problema del cálculo de polinomios de Conway sobre cuerpos finitos y hemos trabajado en la obtención de resultados computacionales nuevos. Las aportaciones principales son las siguientes:

Desarrollo teórico. Hemos presentado una exposición autocontenida de la teoría necesaria para definir y calcular los polinomios de Conway: la estructura de los cuerpos finitos y sus extensiones, la norma y su transitividad, los criterios de irreducibilidad de Rabin y Ben-Or, y la teoría de sistemas compatibles de generadores en grupos cíclicos de Heath y Loehr [4]. En particular, hemos demostrado la existencia de los polinomios de Conway (Teorema 3.2.2), la reducción a divisores maximales (Proposición 3.3.1) y la caracterización del número de candidatos compatibles mediante el parámetro g (Teorema 3.4.1).

Implementación. Hemos desarrollado la librería `conway.lib` para Singular, que implementa el algoritmo de Heath-Loehr junto con varias optimizaciones que lo hacen práctico para un rango amplio de parámetros:

- Cuatro vías de cálculo según la estructura de n: el caso trivial n=1, el algoritmo de Heath-Loehr para grados compuestos ($s \geq 2$) y para potencias de primo con g manejable, la búsqueda exhaustiva en orden de Parker para grados primos y para cuando el algoritmo de Heath-Loehr nos da un g demasiado grande (con comprobación de compatibilidad en este último caso), y una función dedicada para exponentes de Mersenne aprovechando sus propiedades.

- Optimizaciones en el bucle de Heath-Loehr (`iterateCandidates`): multiplicación incremental $\beta_{k+1} = \beta_k \cdot \zeta$ (propuesta en [4]), precálculo del ideal estándar para evitar recalcularlo en cada exponenciación modular, test de primitividad antes del cálculo del polinomio mínimo (descarte rápido de no primitivos), y poda lexicográfica de Parker en `minPolyPruned` (aborta si el candidato ya es peor que el mejor conocido).

- Optimización en la construcción de $r(z)$ para $s = 2$: factorización del polinomio de menor grado y filtro por compatibilidad, evitando construir en memoria el polinomio grande (propuesta en la Sección 4.1 de [4]).

- Test de irreducibilidad de Ben-Or [1] en lugar de factorización completa en todas las búsquedas exhaustivas y en `findIrred`, lo que permite verificar candidatos de grado grande

(hasta varios miles) en tiempo razonable. Esta es una aportación propia de nuestra implementación.

- Detección de primos de Mersenne por dos vías complementarias: una tabla de exponentes conocidos (`isMersenneExponent`) para descarte instantáneo, y un test de primalidad de Miller-Rabin (`millerRabinBigint`) como método alternativo general para cualquier p y n. Cuando $p^n - 1$ es primo, se elimina por completo el test de primitividad. Ambas son aportaciones propias.

Resultados computacionales. Hemos calculado polinomios de Conway para:

- $\mathbb{F}_2$: todos los grados de 1 a 41, todos los grados primos hasta 211, grados compuestos selectos (34, 49, 60, 65), y los exponentes de Mersenne 521, 607, 1279, 2203, 2281, 3217, 4253 y 4423.
- $\mathbb{F}_3$: grados 1 a 24.
- $\mathbb{F}_5$: grados 1 a 14.
- $\mathbb{F}_7$: grados 1 a 13.
- $\mathbb{F}_{11}$: grados 1 a 11.
- $\mathbb{F}_{13}$: grados 1 a 9.

Todos los resultados previamente existentes han sido verificados. Los seis polinomios de Conway para los exponentes de Mersenne $n = 521, 607, 1279, 2203$, 2281, 3217, 4253 y 4423 son, hasta donde hemos podido comprobar, resultados nuevos no presentes en la base de datos de Lübeck [6] ni en las tablas internas de GAP o Magma.

6.2. Reflexiones

A lo largo del trabajo hemos comprobado que la dificultad de calcular un polinomio de Conway no depende tanto del grado n como de la estructura aritmética de n y de $p^n - 1$. Esta observación, ya señalada por Heath y Loehr [4], se manifiesta de forma particularmente clara en dos contrastes:

- $C_{2,4423}$ (grado 4423) se calcula en veinte horas aproximadamente, mientras que $C_{2,128}$ (grado 128) es incomputable con los métodos actuales. La razón es que 4423 es primo y $2^{4423} - 1$ es primo de Mersenne, lo que simplifica radicalmente las condiciones; mientras que $128 = 2^7$ es potencia de primo y $g = 2^{64} + 1$ está fuera de nuestro alcance.
- $C_{2,60}$ (grado 60, compuesto con tres factores primos, $g = 80\,581$) se calcula en aproximadamente 10 minutos, mientras que el espacio total tiene $2^{60} \approx 10^{18}$ polinomios. El algoritmo de Heath-Loehr reduce la búsqueda en un factor de 10^{13}.
- $C_{2,55}$ (grado $55 = 5 \times 11$, compuesto con $s = 2$) tiene $g \approx 5{,}7 \times 10^{11}$, un número de candidatos que no podemos manejar computacionalmente. Sin embargo, la implementación detecta esta situación y recurre automáticamente a la búsqueda exhaustiva en orden de Parker, que solo necesita probar hasta términos de grado 11 para encontrar el primero

válido. Esto ilustra cómo la búsqueda exhaustiva actúa como complemento eficaz cuando el parámetro g es grande: aunque Heath-Loehr no puede recorrer los g candidatos, el orden lexicográfico bajo de ese caso hace que la fuerza bruta encuentre uno rápidamente.

- Todos los primos de p=2 hasta n=199 son calculables de forma casi instantánea debido a que basta con encontrar el primer polinomio primitivo en orden lexicográfico de Parker porque no tiene ninguna restricción de compatibilidad con otros subcuerpos al ser primo. Lo que nos limita es la comprobación de primitividad, a diferencia del caso de los polinomios de Mersenne que sólo requiere irreducibilidad.

La combinación de estas observaciones con las optimizaciones implementadas (test de Ben-Or, detección de Mersenne, búsqueda exhaustiva como fallback) ha permitido a nuestra librería calcular polinomios de Conway que no figuran en las bases de datos existentes, incluyendo las de Lübeck [6] y las integradas en GAP y Magma. Estos sistemas disponen de implementaciones propias del algoritmo de Heath-Loehr, pero no contemplan el caso particular de los exponentes de Mersenne como vía de cálculo específica, lo que explica que los polinomios $C_{2,521}$, $C_{2,607}$, $C_{2,1279}$, $C_{2,2203}$, $C_{2,2281}$, $C_{2,3217}$, $C_{2,4253}$ y $C_{2,4423}$ no hubieran sido calculados hasta ahora.

6.3. Líneas de trabajo futuro

El trabajo realizado deja abiertas varias direcciones de investigación y desarrollo:

1. **Ampliación de los cálculos.** La idea natural para extender el grado de polinomios calculados es la paralelización del bucle de `iterateCandidates`, donde cada candidato β_k se evalúa de forma independiente, y que permitiría multiplicar el umbral actual de 2×10^7 candidatos de forma proporcional al número de hilos o máquinas disponibles, haciendo abordables grados compuestos con g del orden de 10^9 o superior con las suficientes máquinas. Por otro lado, el cálculo de $C_{2,n}$ para exponentes de Mersenne mayores ($n = 9689, 9941, \dots$) sería posible en principio con nuestra implementación, aunque requiere más tiempo y memoria; en este caso, el cuello de botella es exclusivamente el test de irreducibilidad de Ben-Or de cada candidato, y cualquier mejora en su eficiencia se traduciría directamente en grados mayores alcanzables.

2. **Registro de los nuevos polinomios.** Los ocho polinomios de Conway correspondientes a los exponentes de Mersenne 521, 607, 1279, 2203, 2281, 3217, 4253 y 4423 deberían incorporarse a las bases de datos de referencia: la tabla de Frank Lübeck [6] y las librerías internas de GAP, Magma y SageMath. Esto requiere contactar con los mantenedores de dichas bases de datos y proporcionarles los polinomios junto con la verificación de irreducibilidad y compatibilidad.

3. **Factorización de $p^n - 1$.** El cuello de botella para grados primos grandes no es la búsqueda del polinomio sino la factorización de $p^n - 1$. Como la factorización de enteros es un problema para el que no se conoce algoritmo eficiente en computación clásica, métodos como el algoritmo de Shor en un ordenador cuántico con suficientes qubits permitirían extender el rango de grados primos calculables más allá de $n = 200$. No obstante, el estado actual del hardware cuántico está lejos de este objetivo: el mayor número factorizado de forma verificable por un ordenador cuántico es 35 [12].

4. **Características mayores.** Nuestra implementación funciona para cualquier primo p, pero las pruebas se han centrado en $p \leq 13$ por escasez de tiempo y limitación de hardware. Para primos grandes ($p > 100$), el orden de Parker se vuelve más complejo (el factor $(-1)^{n-i}$ produce un orden no trivial) y los tamaños de los grupos $p^n - 1$ crecen rápidamente. Sería interesante estudiar hasta qué punto las optimizaciones actuales siguen siendo efectivas, o si alguno de los métodos implementados se ralentiza y requiere optimizaciones adicionales. Además, esas características menos estudiadas tienen mucho potencial de tener polinomios realizables por este algoritmo aún no calculados.

5. **Cálculo sistemático de grados semiprimos.** Un grado $n = q_1 \cdot q_2$ con q_1, q_2 primos distintos tiene $s = 2$ y solo dos divisores maximales. Esto minimiza las condiciones de compatibilidad, lo que favorece que el candidato lexicográficamente menor aparezca pronto. Podríamos ejecutar un cálculo de $C_{p,n}$ para todos los semiprimos $n = q_1 q_2$ en un rango determinado que no estén calculados, y si g está fuera de alcance (podría ser distinto al establecido actualmente de 2×10^7), y utilizar el algoritmo en orden de fijando un presupuesto de intentos de candidatos por grado, y ese número de intentos vendrá determinado por la paralelización, el tiempo que estamos dispuestos a invertir y el hardware. Los que terminen se añaden a las tablas y los que no se registran como intentos para no repetir el esfuerzo. Es probable que para $p = 2$ o $p = 3$ la mayoría de semiprimos pequeños ya estén calculados, pero para otras características puede que no.

6. **Caché persistente de resultados.** Nuestra implementación intentó incluir funciones de caché en disco que guardaban cada polinomio calculado en un archivo individual para reutilizarlo en sesiones posteriores. Sin embargo, durante el desarrollo encontramos problemas de compatibilidad con el sistema de archivos de Cygwin en Windows (bloqueos en las operaciones de lectura/escritura al arrancar Singular), por lo que la funcionalidad tuvo que desecharse. Una línea de trabajo futuro es estudiar cómo implementar esta caché persistente de modo que los polinomios calculados se acumulen progresivamente en una tabla local, evitando recalcular los de grados divisores en ejecuciones sucesivas.

Apéndice A

Instalación de Singular

Singular es un sistema de álgebra computacional libre (licencia GNU GPL) desarrollado en la Universidad de Kaiserslautern, especializado en álgebra conmutativa, geometría algebraica y teoría de singularidades.

En este apéndice se recogen las instrucciones de instalación para los tres sistemas operativos principales, siguiendo la documentación oficial (`singular.uni-kl.de`). Si solo quieres probar Singular sin instalar nada, existe una versión online en `https://www.singular.uni-kl.de:8003/`.

A.1. Linux / Unix

Es la plataforma nativa de Singular y donde mejor funciona. Hay varias opciones según la distribución.

A.1.1. Ubuntu (20.04, 22.04, 24.04)

La forma más rápida es instalar directamente desde los repositorios oficiales de Ubuntu (aunque no siempre tendrá la última versión):

```
sudo apt-get update
sudo apt-get install singular
```

Para obtener la última versión (paquete `singular41`), hay que añadir el repositorio propio de Singular:

```
# 1. Descargar e instalar la clave GPG
wget ftp://jim.mathematik.uni-kl.de/repo/extra/gpg
sudo apt-key add gpg

# 2. Anadir el repositorio (elegir la linea de tu version):
# Ubuntu 20.04:
#   Anadir a /etc/apt/sources.list:
#   deb https://www.singular.uni-kl.de/ftp/repo/ubuntu20 focal main
# Ubuntu 22.04:
#   deb https://www.singular.uni-kl.de/ftp/repo/ubuntu22 jammy main
# Ubuntu 24.04:
#   deb https://www.singular.uni-kl.de/ftp/repo/ubuntu24 noble main
```

```
# 3. Instalar
sudo apt-get update
sudo apt-get install singular41
```

Nota: los paquetes `singular` y `singular41` son incompatibles entre sí. Si tienes uno instalado, desinstala antes de poner el otro.

A.1.2. Debian (11, 12, 13)

El procedimiento es análogo al de Ubuntu, pero con repositorios específicos:

```
# Descargar clave GPG
wget ftp://jim.mathematik.uni-kl.de/repo/extra/gpg
sudo apt-key add gpg

# Anadir repositorio (elegir segun version):
# Debian 11: deb https://www.singular.uni-kl.de/ftp/repo/debian11 bullseye main
# Debian 12: deb https://www.singular.uni-kl.de/ftp/repo/debian12 bookworm main
# Debian 13: deb https://www.singular.uni-kl.de/ftp/repo/debian13 trixie main

sudo apt-get update
sudo apt-get install singular41
```

A.1.3. Fedora y otras distribuciones

Para Fedora y distribuciones sin soporte APT, la instalación se hace manualmente descargando el archivo TGZ desde la web de Singular. Las instrucciones detalladas están en: `https://www.singular.uni-kl.de/index.php/singular-download/109.html`

A.1.4. Gentoo

En Gentoo simplemente:

```
emerge singular
```

A.2. Windows

Singular no tiene una versión nativa para Windows, pero se puede usar a través de Cygwin o del Subsistema de Linux para Windows (WSL). Hay dos opciones:

A.2.1. Opción A: Cygwin (método clásico)

1. Descargar e instalar Cygwin desde `https://cygwin.com/setup-x86_64.exe` con la configuración por defecto.

2. Durante la instalación, en la pantalla "Select packages", cambiar "View.ª "Full", buscar "wgetz cambiar la columna "New"de "Skip.ª alguna versión disponible.

3. Completar la instalación.

4. Abrir la terminal de Cygwin y ejecutar:

```
wget ftp://jim.mathematik.uni-kl.de/repo/cygwin/64/build.tar
tar xf build.tar
bash build_cygwin
```

Después de esto, Singular se puede iniciar desde la terminal de Cygwin escribiendo `Singular`.

A.2.2. Opción B: WSL (más cómodo, recomendado)

Si tienes Windows 10/11, la opción más cómoda es usar el Subsistema de Linux para Windows (WSL):

1. Instalar WSL con Ubuntu siguiendo las instrucciones de Microsoft: `https://learn.microsoft.com/en-us/windows/wsl/install`
2. Abrir la terminal de Ubuntu (buscar "bash.en el menú de inicio, o escribir "bash.en el cmd).
3. Instalar Singular como en cualquier Ubuntu:

```
sudo apt-get update
sudo apt-get install singular
```

Esta opción es en general más estable y cómoda que Cygwin. Además, nuestra librería `conway.lib` usa el comando `factor` de Linux internamente, que funciona nativamente en WSL pero puede dar problemas en Cygwin.

A.3. macOS

Hay tres formas de instalar Singular en Mac:

A.3.1. Homebrew (recomendado)

La forma más sencilla con diferencia. Si ya tienes Homebrew instalado:

```
brew install Singular
```

Si no tienes Homebrew, instálalo primero desde `https://brew.sh`

A.3.2. Archivo DMG (instalación manual)

Descargar el archivo DMG correspondiente a tu procesador:

- **Intel (x86_64):** `Singular-4-3-2_64.dmg`
- **Apple Silicon (M1/M2/M3):** `Singular-4-4-0_M1.dmg`

Los archivos se descargan desde: `ftp://jim.mathematik.uni-kl.de/pub/Math/Singular/UNIX/` Montar la imagen y arrastrar el contenido a la carpeta Aplicaciones. Si macOS bloquea la aplicación por ser de un "desarrollador no identificado": ir a Ajustes del Sistema → Privacidad y Seguridad, y en "Permitir apps descargadas de"seleccionar .App Store y desarrolladores identificados".

A.3.3. Fink (alternativa)

Si usas el gestor de paquetes Fink:

```
fink selfupdate
fink install singular singular-doc
```

A.4. Primer uso y carga de conway.lib

Una vez instalado Singular, abrimos una terminal y escribimos:

```
Singular
```

Debería aparecer algo como:

```
SINGULAR                                 /
A Computer Algebra System for Polynomial Computations       /   version 4.x.x
0<
by: W. Decker, G.-M. Greuel, G. Pfister, H. Schoenemann     \
Copyright (c) 1986-2024 - Univ. Kaiserslautern-Landau \
>
```

Para cargar nuestra librería y calcular un polinomio de Conway:

```
// Cargar la libreria (el archivo debe estar en el directorio actual
// o en el path de librerias de Singular)
LIB "conway.lib";

// Definir el anillo base: caracteristica p, variable x, orden dp
ring R = 2, x, dp;

// Calcular C_{2,6}
conwayPoly(6);
// Deberia devolver: x6+x4+x3+x+1
```

A.5. Recursos adicionales

- **Manual online de Singular:** `https://www.singular.uni-kl.de/index.php/singular-manual.html`
- **Probar Singular online (sin instalar):** `https://www.singular.uni-kl.de:8003/`
- **Base de datos de polinomios de Conway (Lübeck):** `http://www.math.rwth-aachen.de/~Frank.Luebeck/data/ConwayPol/`

Apéndice B

Código fuente de conway.lib

A continuación se incluye el listado completo de la librería `conway.lib` para Singular, que implementa el algoritmo de Heath-Loehr para el cálculo de polinomios de Conway. [4] El código está comentado en detalle y organizado en 9 partes lógicas (véase la estructura interna del propio archivo).

Para usar esta librería, copiar el texto en un archivo `conway.lib` en el directorio de trabajo o descargarlo desde el enlace: `https://github.com/miguels-b/libreria-Singular-Polinomios-de-Conway.git`

```
////////////////////////////////////////////////////////////////////////////
version="2.0";
category="Finite␣fields";
info="
LIBRARY:␣␣conway.lib␣␣␣Calculo␣de␣polinomios␣de␣Conway␣sobre␣cuerpos␣finitos

AUTHOR:␣␣␣Miguel,␣TFG␣Matematicas

OVERVIEW:
␣␣Implementacion␣del␣algoritmo␣de␣Heath-Loehr␣(2004)␣para␣calcular␣el
␣␣polinomio␣de␣Conway␣C_{p,n}␣sobre␣F_p,␣con␣las␣ideas␣de␣Ruano-Laplagne
␣␣(2007)␣para␣Singular.

␣␣Dado␣un␣primo␣p␣y␣un␣grado␣n␣=␣q1^e1␣*␣...␣*␣qs^es,␣el␣algoritmo
␣␣reduce␣el␣espacio␣de␣busqueda␣de␣p^n␣candidatos␣a␣exactamente
␣␣g␣=␣mcd{␣(p^n-1)/(p^{n/qi}-1)␣}␣candidatos␣mediante␣un␣MCD␣de
␣␣polinomios,␣evitando␣la␣busqueda␣exhaustiva␣para␣n␣compuesto.

␣␣Para␣n␣primo␣(s=1,␣e=1)␣se␣usa␣busqueda␣exhaustiva␣en␣orden␣de␣Parker,
␣␣que␣termina␣al␣primer␣candidato␣valido␣(h␣=␣p^n/tau␣~␣1).

␣␣Para␣n␣potencia␣de␣primo␣(s=1,␣e>1)␣se␣usa␣el␣algoritmo␣de␣Heath-Loehr
␣␣con␣r(z)␣=␣C_{p,n/q}(z)␣como␣polinomio␣auxiliar.

CASOS␣DEL␣ALGORITMO:
␣␣n␣=␣1␣␣␣␣␣␣->␣␣menor␣primitivo␣de␣F_p␣(trivial)
␣␣s␣>=␣2␣␣␣␣␣->␣␣r(z)␣=␣mcd{␣C_{p,di}(z^{mi/g})␣}␣es␣irreducible␣grado␣n
␣␣s␣=␣1,␣e=1␣->␣␣n␣primo:␣busqueda␣exhaustiva␣Parker␣con␣parada␣inmediata
␣␣s␣=␣1,␣e>1␣->␣␣n␣=␣q^e:␣r(z)␣=␣C_{p,n/q}(z),␣busqueda␣entre␣g␣raices

OPTIMIZACIONES␣IMPLEMENTADAS:
```

```
  - Termino constante fijo por compatibilidad con C_{p,1} (reduce busqueda /p)
  - Orden de Parker exacto en el bucle exhaustivo (parada al primer valido)
  - Poda lexicografica en iterateCandidates (abortar minpoly si ya es peor)
  - Multiplicacion incremental: beta_{k+1} = beta_k * zeta (evita polyPow)
  - Test de primitividad antes de calcular minpoly (descarte rapido)
  - Ideal precalculado una sola vez por llamada
  - Factorizacion de p^n-1 precalculada y reutilizada en todo el bucle
  - Busqueda de irreducibles: trinomios -> pentanomios -> sistematica

REFERENCIAS:
  [HL04] Heath, Loehr: New algorithms for generating Conway polynomials
         over finite fields. J. Symbolic Comput. 38 (2004), 1003-1024.
  [RL07] Ruano, Laplagne: Conway Polynomials and Singular. Segovia, 2007.
  [Lub]  Frank Luebeck: tablas en
         https://www.math.rwth-aachen.de/~Frank.Luebeck/data/ConwayPol/

PROCEDURES (publicas):
  conwayPoly(n [,known])     Calcula C_{p,n}
  isSmallerConway(f,g,n)     Comparacion en orden de Parker
  isPrimitivePoly(f,n)       Test de primitividad
  isCompatible(fn,fm,n,m)    Test de compatibilidad
  candidatosConway(n)        Cuenta g y phi(p^n-1)/n para C_{p,n}
  mostrarCandidatos(n)       Muestra g y phi(p^n-1)/n por pantalla
";

LIB "polylib.lib";


////////////////////////////////////////////////////////////////////////////////
//
//  SECCION 1: ARITMETICA DE ENTEROS GRANDES
//
//  Operaciones sobre bigint que Singular no tiene de serie:
//  mcd, inverso modular, extraccion de coeficientes, factorizacion.
//
////////////////////////////////////////////////////////////////////////////////

// Algoritmo de Euclides para bigint.
proc mcdBigint(bigint a, bigint b)
"USAGE:   mcdBigint(a,b);  a,b bigint
RETURN:   mcd(a,b)
"
{
    if (a < 0) { a = -a; }
    if (b < 0) { b = -b; }
    bigint t;
    while (b != 0) { t = b; b = a mod b; a = t; }
    return(a);
}

// Inverso modular por el algoritmo extendido de Euclides.
// Precondicion: mcd(a,m) = 1.
proc modInverse(bigint a, bigint m)
"USAGE:   modInverse(a,m);  a,m bigint con mcd(a,m)=1
RETURN:   x tal que a*x = 1 mod m
"
```

```
{
    bigint m0 = m; bigint x0 = 0; bigint x1 = 1;
    bigint q; bigint t;
    a = ((a mod m) + m) mod m;
    while (a > 1)
    {
        q = a div m; t = m; m = a mod m; a = t;
        t = x0; x0 = x1 - q * x0; x1 = t;
    }
    return(((x1 mod m0) + m0) mod m0);
}

// Devuelve el coeficiente de x^i en f como entero.
proc coeffAt(poly f, int i)
{
    if (deg(f) < i) { return(0); }
    poly term = coeffs(f, var(1)^i);
    if (term == 0) { return(0); }
    return(int(leadcoef(term)));
}

// Factorizacion por division de prueba para n entero pequenyo.
// Devuelve lista de pares [primo, exponente].
proc factorizeSmall(int n)
{
    list L; int idx = 0; int d = 2; int e;
    while (d * d <= n)
    {
        if (n mod d == 0)
        {
            e = 0;
            while (n mod d == 0) { n = n div d; e = e + 1; }
            idx = idx + 1; L[idx] = list(d, e);
        }
        d = d + 1;
    }
    if (n > 1) { idx = idx + 1; L[idx] = list(n, 1); }
    return(L);
}


// --- Test de primalidad Miller-Rabin determinista para bigint ---
// Determinista para n < 3.3 * 10^24 (12 bases).
// Para n mayor es probabilistico pero con error < 4^{-12}.
static proc millerRabinBigint(bigint n)
{
    if (n < 2) { return(0); }
    if (n == 2 || n == 3) { return(1); }
    if ((n mod 2) == 0) { return(0); }

    bigint d = n - 1;
    int r = 0;
    while ((d mod 2) == 0) { d = d div 2; r = r + 1; }

    list bases = list(bigint(2), bigint(3), bigint(5), bigint(7),
                      bigint(11), bigint(13), bigint(17), bigint(19),
```

```
                        bigint(23), bigint(29), bigint(31), bigint(37));
    int i; int j;
    for (i = 1; i <= size(bases); i++)
    {
        bigint a = bases[i];
        if (a >= n) { break; }
        bigint x = 1;
        bigint bv = a mod n;
        bigint exp = d;
        while (exp > 0)
        {
            if ((exp mod 2) == 1) { x = (x * bv) mod n; }
            exp = exp div 2;
            if (exp > 0) { bv = (bv * bv) mod n; }
        }
        if (x == 1 || x == n - 1) { continue; }
        int found = 0;
        for (j = 1; j < r; j++)
        {
            x = (x * x) mod n;
            if (x == n - 1) { found = 1; break; }
        }
        if (!found) { return(0); }
    }
    return(1);
}

// Factorizacion para bigint.
// Para numeros grandes usa la llamada al sistema 'factor' de Linux/Cygwin (
    instantaneo).
// Devuelve lista de pares [primo, exponente].
proc factorizeBigint(bigint n)
{
    list L;

    // 1. Usar el comando 'factor' del SO y formatearlo a codigo de Singular
    string cmd = "factor␣" + string(n) + "␣|␣cut␣-d:␣-f2␣|␣tr␣-s␣'␣'␣'\\n'␣|␣
        grep␣-v␣'^$'␣|␣uniq␣-c␣|␣awk␣'{print␣\"L[\"␣NR␣\"]=list(bigint(\"␣$2␣\")
        ,\"␣$1␣\");\"}'␣>␣tmp_fac.txt";
    int dummy = system("sh", cmd);

    // 2. Leer el resultado
    string output = read("tmp_fac.txt");

    // 3. Si funciona, lo ejecutamos y devolvemos la lista
    if (size(output) > 0)
    {
        execute(output);
        return(L);
    }

    // 4. FALLBACK: Si el comando falla por algun motivo, usamos trial division
        (lento)
    int idx = 0; bigint d = 2; int e;
    while (d * d <= n)
    {
```

```
        if (n mod d == 0)
        {
            e = 0;
            while (n mod d == 0) { n = n div d; e = e + 1; }
            idx = idx + 1; L[idx] = list(d, e);
        }
        d = d + 1;
    }
    if (n > 1) { idx = idx + 1; L[idx] = list(n, 1); }
    return(L);
}

// Extrae la lista de factores primos distintos de M.
// Usado para el test de primitividad y para gthRoot.
static proc primeFactors(bigint M, int p, int n)
{
    list facs = factorizeBigint(M);
    list primes;
    int i;
    for (i = 1; i <= size(facs); i++)
    {
        primes[i] = bigint(facs[i][1]);
    }
    if (size(primes) == 0)
    {
        ERROR("primeFactors:␣no␣se␣pudo␣factorizar␣" + string(M));
    }
    return(primes);
}


////////////////////////////////////////////////////////////////////////////////
//
//  SECCION 2: ARITMETICA EN F_p[x]/(f)
//
//  Exponenciacion modular, evaluacion de polinomios y polinomio minimo.
//  La funcion nuclear es fastPow, que recibe el ideal ya precalculado
//  para evitar recalcular std() en cada llamada dentro de un bucle.
//
////////////////////////////////////////////////////////////////////////////////

// Exponenciacion modular base^e mod modp en F_p[x].
// Interfaz publica que precalcula el ideal internamente.
proc polyPow(poly base, bigint e, poly modp)
"USAGE:␣␣␣polyPow(base,␣e,␣modp);
RETURN:␣␣base^e␣mod␣modp␣en␣F_p[x]
NOTE:␣␣␣␣Square-and-multiply␣con␣reduccion␣en␣cada␣paso.
"
{
    if (e == 0) { return(poly(1)); }
    ideal I = std(ideal(modp));
    poly r = 1;
    base = reduce(base, I);
    while (e > 0)
    {
        if ((e mod 2) == 1) { r = reduce(r * base, I); }
```

```
        e = e div 2;
        if (e > 0) { base = reduce(base * base, I); }
    }
    return(r);
}

// Version interna de polyPow que recibe el ideal ya calculado.
// Usarla dentro de bucles para no recalcular std() en cada iteracion.
static proc fastPow(poly base, bigint e, ideal I)
{
    if (e == 0) { return(poly(1)); }
    poly r = 1;
    base = reduce(base, I);
    while (e > 0)
    {
        if ((e mod 2) == 1) { r = reduce(r * base, I); }
        e = e div 2;
        if (e > 0) { base = reduce(base * base, I); }
    }
    return(r);
}

// Evalua f(alpha) en F_p[x]/(modp) usando fastPow para cada monomio.
proc polyEval(poly f, poly alpha, poly modp)
{
    ideal I = std(ideal(modp));
    poly r = 0;
    int d; number c;
    while (f != 0)
    {
        d = deg(lead(f));
        c = leadcoef(f);
        r = reduce(r + c * fastPow(alpha, bigint(d), I), I);
        f = f - lead(f);
    }
    return(r);
}

// Polinomio minimo de alpha en F_p[x]/(modp) sobre F_p.
// Calcula los conjugados de Frobenius: alpha, alpha^p, alpha^{p^2}, ...
// hasta que vuelva al primero, y expande el producto (x-alpha_i).
proc minPoly(poly alpha, int p, int n, poly modp)
{
    ideal I = std(ideal(modp));

    // Paso 1: calcular los conjugados de Frobenius
    list conj;
    conj[1] = reduce(alpha, I);
    poly curr = conj[1];
    int d = 1; int i;
    for (i = 2; i <= n; i++)
    {
        curr = fastPow(curr, bigint(p), I);
        if (curr == conj[1]) { break; }
        d = d + 1; conj[d] = curr;
    }
```

```
    // Paso 2: expandir (x - conj[1])*(x - conj[2])*...*(x - conj[d])
    // usando los coeficientes elementales simetricos de forma incremental
    list C; C[1] = poly(1); C[2] = reduce(-conj[1], I);
    int j; int len; list Cnew;
    for (i = 2; i <= d; i++)
    {
        len = size(C); Cnew = list(); Cnew[1] = C[1];
        for (j = 2; j <= len; j++)
        {
            Cnew[j] = reduce(C[j] - conj[i]*C[j-1], I);
        }
        Cnew[len+1] = reduce(-conj[i]*C[len], I);
        C = Cnew;
    }

    // Paso 3: reconstruir el polinomio desde los coeficientes
    poly result = 0; number c;
    for (i = 1; i <= d+1; i++)
    {
        c = leadcoef(C[i]);
        result = result + c * var(1)^(d-i+1);
    }
    return(result);
}

// Version con poda lexicografica de Parker.
// Construye el minpoly y aborta (devuelve poly(0)) en cuanto detecta
// que el resultado ya seria mayor que 'best' en el orden de Parker.
// Si best == 0 no hay con que comparar y devuelve el minpoly completo.
static proc minPolyPruned(poly beta, int p, int n, poly modp, poly best)
{
    ideal I = std(ideal(modp));

    // Conjugados de Frobenius
    list conj; conj[1] = reduce(beta, I);
    poly curr = conj[1];
    int d = 1; int i;
    for (i = 2; i <= n; i++)
    {
        curr = fastPow(curr, bigint(p), I);
        if (curr == conj[1]) { break; }
        d = d + 1; conj[d] = curr;
    }
    // Si el grado del minpoly no es n, este beta no da un Conway de grado n
    if (d != n) { return(poly(0)); }

    // Expandir el producto incremental
    list C; C[1] = poly(1); C[2] = reduce(-conj[1], I);
    int j; int len; list Cnew;
    for (i = 2; i <= d; i++)
    {
        len = size(C); Cnew = list(); Cnew[1] = C[1];
        for (j = 2; j <= len; j++)
        {
            Cnew[j] = reduce(C[j] - conj[i]*C[j-1], I);
```

```
        }
        Cnew[len+1] = reduce(-conj[i]*C[len], I);
        C = Cnew;
    }

    // Reconstruir el polinomio
    poly result = 0; number c;
    for (i = 1; i <= d+1; i++)
    {
        c = leadcoef(C[i]);
        result = result + c * var(1)^(d-i+1);
    }

    // Sin referencia: devolver directamente
    if (best == 0) { return(result); }

    // Poda de Parker: comparar result contra best coeficiente a coeficiente
    // desde i = n-1 hacia 0. El valor efectivo de Parker en posicion i es
    //   (-1)^(n-i) * a_i  mod p  con 0 < 1 < ... < p-1.
    int ai; int bi; poly tf; poly tg;
    for (i = n-1; i >= 0; i = i-1)
    {
        tf = jet(result, i); tg = jet(best, i);
        if (deg(tf) == i) { ai = int(leadcoef(tf)); } else { ai = 0; }
        if (deg(tg) == i) { bi = int(leadcoef(tg)); } else { bi = 0; }
        if ((n-i) mod 2 != 0)
        {
            ai = ((-ai) mod p + p) mod p;
            bi = ((-bi) mod p + p) mod p;
        }
        else
        {
            ai = (ai mod p + p) mod p;
            bi = (bi mod p + p) mod p;
        }
        if (ai > bi) { return(poly(0)); } // peor -> podar
        if (ai < bi) { return(result);  } // mejor -> aceptar
    }
    return(poly(0)); // iguales -> no reemplazar
}


////////////////////////////////////////////////////////////////////////////////
//
//  SECCION 3: ORDEN LEXICOGRAFICO DE PARKER
//
//  El orden de Conway (<lex de Parker) compara los coeficientes de
//  f = x^n + a_{n-1}x^{n-1} + ... + a_1 x + a_0
//  desde a_0 hacia arriba usando el valor efectivo (-1)^(n-i) * a_i mod p
//  con el orden 0 < 1 < ... < p-1 en Z_p.
//
//  Para p=2 coincide con el orden lexicografico normal porque -1 = 1.
//  Para p > 2 los coeficientes en posiciones de exponente impar se comparan
//  con signo negativo, lo que invierte su orden relativo.
//
////////////////////////////////////////////////////////////////////////////////
```

```
// Comparacion en orden de Parker: devuelve 1 si f <_Parker g, 0 si no.
proc isSmallerConway(poly f, poly g, int n)
{
    int p = char(basering);
    int i; int ai; int bi; poly tf; poly tg;
    for (i = n-1; i >= 0; i = i-1)
    {
        tf = jet(f, i); tg = jet(g, i);
        if (deg(tf) == i) { ai = int(leadcoef(tf)); } else { ai = 0; }
        if (deg(tg) == i) { bi = int(leadcoef(tg)); } else { bi = 0; }
        if ((n-i) mod 2 != 0) { ai = -ai; bi = -bi; }
        ai = ((ai mod p) + p) mod p;
        bi = ((bi mod p) + p) mod p;
        if (ai < bi) { return(1); }
        if (ai > bi) { return(0); }
    }
    return(0);
}

// Convierte un indice de contador (0..p-1) al coeficiente real a_i
// segun el orden de Parker para la posicion i en un polinomio de grado n.
//
// La idea: queremos que al incrementar el contador de 0 a p-1,
// el valor efectivo Parker (-1)^(n-i) * a_i crezca de 0 a p-1.
//
//   Si (n-i) es par:   valor_Parker = a_i           -> a_i en orden 0,1,...,p-1
//   Si (n-i) es impar: valor_Parker = (-a_i mod p)  -> para que crezca,
//                      a_i debe ir 0, p-1, p-2, ..., 1
static proc parkerCoef(int idx, int i, int n, int p)
{
    if ((n - i) mod 2 == 0)
    {
        return(idx);          // orden directo
    }
    else
    {
        if (idx == 0) { return(0); }
        return(p - idx);      // orden invertido: 0, p-1, p-2, ..., 1
    }
}


////////////////////////////////////////////////////////////////////////////////
//
//  SECCION 4: TEST DE PRIMITIVIDAD Y COMPATIBILIDAD
//  isMersenneExponent: comprueba si n es un primo de mersenne.
//  isPrimitivePoly: comprueba que alpha = var(1) genera GF(p^n)*.
//  isCompatible:   comprueba la condicion de anidamiento C_{p,n} | C_{p,m}(x^{e
    }).
//
////////////////////////////////////////////////////////////////////////////////

// Funcion auxiliar que guarda la lista de exponentes
static proc isMersenneExponent(int n)
{
```

```
    list mersennes = 2, 3, 5, 7, 13, 17, 19, 31, 61, 89, 107, 127, 521, 607,
        1279, 2203, 2281, 3217, 4253, 4423, 9689, 9941, 11213, 19937, 21701,
        23209, 44497, 86243, 110503, 132049, 216091, 756839, 859433, 1257787,
        1398269, 2976221, 3021377, 6972593, 13466917, 20996011, 24036583,
        25964951, 30402457, 32582657, 37156667, 42643801, 43112609, 57885161,
        74207281, 77232917, 82589933, 136279841;
    int i;
    for (i = 1; i <= size(mersennes); i++)
    {
        if (n == mersennes[i]) { return(1); }
    }
    return(0);
}


// Test de primitividad: f es primitivo sii alpha^{(p^n-1)/q} != 1
// para todo primo q | p^n-1.
proc isPrimitivePoly(poly f, int n, list #)
"USAGE:␣␣␣isPrimitivePoly(f,␣n␣[,␣factoresDePnMenos1]);
RETURN:␣␣1␣si␣f␣es␣primitivo␣de␣grado␣n␣sobre␣F_p,␣0␣si␣no.
NOTE:␣␣␣␣Pasar␣la␣lista␣de␣factores␣primos␣de␣p^n-1␣evita␣refactorizar.
EXAMPLE:␣example␣isPrimitivePoly;␣shows␣an␣example
"
{
    int p = char(basering);

    // --- ATAJO PARA PRIMOS DE MERSENNE ---
    // Si p=2 y n es un exponente de Mersenne conocido,
    // sabemos 100% que todo irreducible es primitivo.
    if (p == 2 && isMersenneExponent(n))
    {
        return(1);
    }
    // -------------------------------------

    bigint M = bigint(p)^n - 1;

    list primes;
    if (size(#) > 0) { primes = #[1]; }

    // Si no nos pasaron los factores precalculados, tenemos que encontrarlos
    if (size(primes) == 0)
    {
        // Test rapido de primalidad general (por si no es base 2 o no esta en
            la lista)
        if (millerRabinBigint(M))
        {
            return(1);
        }

        // Si no es primo, intentamos factorizar por trial division si es
            pequenyo
        if (M < bigint(2)^31)
        {
            list facs = factorizeSmall(int(M));
            int i;
```

```
            for (i = 1; i <= size(facs); i++) { primes[i] = bigint(facs[i][1]);
                }
        }
        else
        {
            primes = primeFactors(M, p, n);
        }
    }

    if (size(primes) == 0) { ERROR("isPrimitivePoly:␣no␣se␣pudo␣factorizar␣p^n
        -1."); }

    // Bucle normal de comprobacion de primitividad
    int i;
    bigint e;
    poly r;

    for (i = 1; i <= size(primes); i++)
    {
        e = M div primes[i];
        r = polyPow(var(1), e, f);
        if (r == 1) { return(0); }
    }
    return(1);
}


// Test de compatibilidad: fn es compatible con fm sii
// fn(x) | fm(x^{(p^n-1)/(p^m-1)}), comprobado evaluando fm en alpha^e.
proc isCompatible(poly fn, poly fm, int n, int m)
"USAGE:␣␣␣isCompatible(fn,␣fm,␣n,␣m);␣␣con␣m␣|␣n
RETURN:␣␣1␣si␣C_{p,n}␣=␣fn␣es␣compatible␣con␣C_{p,m}␣=␣fm.
NOTE:␣␣␣␣Evalua␣fm(alpha^{(p^n-1)/(p^m-1)})␣donde␣alpha␣es␣raiz␣de␣fn.
"
{
    int p = char(basering);
    bigint Mn = bigint(p)^n - 1;
    bigint Mm = bigint(p)^m - 1;
    bigint e = Mn div Mm;
    poly alpha_e = polyPow(var(1), e, fn);
    poly val = polyEval(fm, alpha_e, fn);
    return(int(val == 0));
}


////////////////////////////////////////////////////////////////////////////////
//
//  SECCION 5: HERRAMIENTAS DE CAMPO FINITO
//
//  Funciones para trabajar en GF(p^n): encontrar generadores, raices
//  g-esimas, raices de polinomios, e irreducibles auxiliares.
//
////////////////////////////////////////////////////////////////////////////////

// Sustitucion x -> x^k en un polinomio f(x), devuelve f(x^k).
static proc substPow(poly f, bigint k)
```

```
{
    poly r = 0; int d; number c;
    while (f != 0)
    {
        d = deg(lead(f)); c = leadcoef(f);
        r = r + c * var(1)^int(bigint(d) * k);
        f = f - lead(f);
    }
    return(r);
}

// Busca un generador de GF(p^n)* = F_p[x]/(modp)* por busqueda sistematica.
// Un elemento gamma genera el grupo sii gamma^{M/q} != 1 para todo primo q|M.
static proc findGenerator(poly modp, int p, int n, bigint M, list primesM)
{
    ideal I = std(ideal(modp));
    poly one = poly(1);
    int c; poly gamma; int is_gen; int j; int rem; int coef_val; int i;
    for (c = 2; c < 2000000; c++)
    {
        // Construir el elemento c en base p como polinomio en F_p[x]
        gamma = 0; rem = c;
        for (i = 0; rem > 0; i++)
        {
            coef_val = rem mod p;
            if (coef_val != 0) { gamma = gamma + number(coef_val) * var(1)^i; }
            rem = rem div p;
        }
        // Test de generador
        is_gen = 1;
        for (j = 1; j <= size(primesM); j++)
        {
            if (fastPow(gamma, M div primesM[j], I) == one) { is_gen = 0; break;
                }
        }
        if (is_gen) { return(gamma); }
    }
    ERROR("findGenerator:␣no␣encontrado␣en␣2000000␣intentos␣para␣grado␣" +
        string(n));
}

// Extrae una raiz g-esima de z0 en GF(p^n).
// Metodo: si mcd(g, M/g) = 1 usamos inversion modular (O(log g)).
// En caso contrario usamos busqueda directa con multiplicacion incremental.
static proc gthRoot(poly z0, bigint g, poly modp, int p, int n)
{
    bigint M = bigint(p)^n - 1;
    bigint ordz = M div g;
    bigint gmod = g mod ordz;
    if (gmod == 0) { gmod = g; }
    bigint mcd_val = mcdBigint(gmod, ordz);
    ideal I = std(ideal(modp));

    // Caso facil: inversion modular directa
    if (mcd_val == 1)
    {
```

```
        bigint e = modInverse(gmod, ordz);
        return(fastPow(z0, e, I));
    }

    // Caso general: busqueda entre las M/g potencias de gamma
    list primesM = primeFactors(M, p, n);
    poly gamma = findGenerator(modp, p, n, M, primesM);
    poly step = fastPow(gamma, g, I);
    poly cur = 1;
    poly z0_red = reduce(z0, I);
    bigint j; bigint limit = M div g;
    for (j = 0; j <= limit; j = j + 1)
    {
        if (cur == z0_red) { return(fastPow(gamma, j, I)); }
        cur = reduce(cur * step, I);
    }
    ERROR("gthRoot: raiz no encontrada para g = " + string(g));
}

// Devuelve una raiz g-esima primitiva de la unidad: zeta = gamma^{M/g}.
static proc gthRootOfUnity(bigint g, poly modp, int p, int n, bigint M, list
    primesM)
{
    if (M mod g != 0) { ERROR("gthRootOfUnity: g no divide p^n-1."); }
    poly gamma = findGenerator(modp, p, n, M, primesM);
    ideal I = std(ideal(modp));
    return(fastPow(gamma, M div g, I));
}

// Busca un polinomio irreducible de grado n sobre F_p.
// Estrategia: primero trinomios x^n + x^i + a0 (los mas comunes),
// luego pentanomios, y finalmente busqueda sistematica como ultimo recurso.
static proc findIrred(int p, int n)
{
    poly candidate; int k; int is_irred; list facs;
    int i; int j; int m; int a0;

    // Intento 1: trinomios x^n + x^i + a0
    for (i = 1; i < n; i++)
    {
        for (m = 1; m < p; m++)
        {
            candidate = var(1)^n + var(1)^i + number(m);
            is_irred = isIrreducibleBenOr(candidate, n);
            if (is_irred && deg(candidate) == n) { return(candidate); }
        }
    }

    // Intento 2: pentanomios x^n + x^i + x^j + x^m + a0
    for (i = n-1; i > 2; i = i-1)
    {
        for (j = i-1; j > 1; j = j-1)
        {
            for (m = j-1; m > 0; m = m-1)
            {
                for (a0 = 1; a0 < p; a0++)
```

```
                {
                    candidate = var(1)^n + var(1)^i + var(1)^j + var(1)^m +
                        number(a0);
                    is_irred = isIrreducibleBenOr(candidate, n);
                    if (is_irred && deg(candidate) == n) { return(candidate); }
                }
            }
        }
    }

    // Intento 3: busqueda sistematica (lento para n y p grandes)
    int max_val = p^(n-1); int c; int coef_val; int rem;
    for (c = 1; c < max_val; c++)
    {
        if ((c mod p) == 0) { continue; } // termino independiente cero ->
            reducible
        candidate = var(1)^n; rem = c;
        for (i = 0; i < n; i++)
        {
            coef_val = rem mod p;
            if (coef_val != 0) { candidate = candidate + number(coef_val) * var
                (1)^i; }
            rem = rem div p;
        }
        is_irred = isIrreducibleBenOr(candidate, n);
        if (is_irred && deg(candidate) == n) { return(candidate); }
    }
    ERROR("findIrred: no encontrado para p=" + string(p) + ", n=" + string(n));
}

// Busca una raiz de f(x) en GF(p^n) = F_p[x]/(modp)
// probando las potencias gamma^{step}, gamma^{2*step}, ...
// donde step = (p^n-1)/(p^d-1) hace que solo visitemos el subgrupo
// correcto de orden p^d-1.
static proc findRoot(poly f, poly modp, int p, int n)
{
    int d = deg(f);
    bigint Mn = bigint(p)^n - 1;
    bigint Md = bigint(p)^d - 1;
    list primesM = primeFactors(Mn, p, n);
    poly gamma = findGenerator(modp, p, n, Mn, primesM);
    bigint step = Mn div Md;
    ideal I = std(ideal(modp));
    poly gamma_step = fastPow(gamma, step, I);
    poly cur = 1; poly val; bigint j;
    for (j = 0; j < Md; j = j + 1)
    {
        val = polyEval(f, cur, modp);
        if (val == 0) { return(cur); }
        cur = reduce(cur * gamma_step, I);
    }
    ERROR("findRoot: raiz de " + string(f) + " no encontrada en GF(" + string(p)
        + "^" + string(n) + ")");
}
```

```
////////////////////////////////////////////////////////////////////////////
//
//  SECCION 6: NUCLEO DEL ALGORITMO HEATH-LOEHR
//
//  Tres casos segun la factorizacion de n:
//    caseComposite    -> s >= 2 primos distintos
//    casePrimePower   -> s = 1, n = q^e con e >= 2
//    casePrimeExhaust -> s = 1, e = 1, n primo
//
//  En todos los casos se llama finalmente a iterateCandidates, que
//  recorre las g raices g-esimas de z0 y devuelve el menor minpoly
//  primitivo en orden de Parker.
//
////////////////////////////////////////////////////////////////////////////

// Caso n = q1^e1 * ... * qs^es con s >= 2.
//
// El teorema 9 de Heath-Loehr garantiza que el polinomio
//   r(z) = mcd{ C_{p,di}(z^{mi/g}) }
// es irreducible de grado n. Tomamos z0 = var(1) (raiz de r) y buscamos
// entre las g raices g-esimas de z0 el menor primitivo.
//
// Caso compuesto (s >= 2 primos distintos)
static proc caseComposite(int n, int p, int s, list di, list qi,
                          list mi, list Cpdi, bigint g)
{
    // --- NUEVO: Abortar temprano si g es inmanejable ---
    if (g > bigint(20000000))
    {
        "//␣␣␣g␣INMANEJABLE␣(" + string(g) + ")␣->␣Abortando␣Heath-Loehr.␣
            Loteria␣Parker...";
        return(exhaustiveSearch(n, p, Cpdi, di));
    }
    // ---------------------------------------------------

    // Identificar el indice del divisor di con menor grado de sustitucion
    bigint minDeg = bigint(di[1]) * (mi[1] div g);
    int minIdx = 1; int i;
    for (i = 2; i <= s; i++)
    {
        bigint deg_i = bigint(di[i]) * (mi[i] div g);
        if (deg_i < minDeg) { minDeg = deg_i; minIdx = i; }
    }
    "//␣␣␣[COMPUESTO]␣Grado␣del␣polinomio␣menor␣en␣MCD:␣" + string(minDeg);
    poly rz;
    if (s == 2 && minDeg < 50000)
    {
        "//␣␣␣Factorizando␣polinomio␣menor␣(grado␣" + string(minDeg) + ")...";
        poly small_poly = substPow(Cpdi[minIdx], mi[minIdx] div g);
        list facs = factorize(small_poly);
        int otherIdx = 3 - minIdx;
        int k; poly factor_k; poly z0_pow; poly val;
        rz = 0;
        for (k = 1; k <= size(facs[1]); k++)
        {
            if (deg(facs[1][k]) == n)
```

```
            {
                factor_k = facs[1][k];
                z0_pow = polyPow(var(1), mi[otherIdx] div g, factor_k);
                val = polyEval(Cpdi[otherIdx], z0_pow, factor_k);
                if (val == 0) { rz = factor_k; break; }
            }
        }
        if (rz == 0) { ERROR("caseComposite: no se encontro factor compatible.")
            ; }
    }
    else
    {
        "//   Calculando MCD de polinomios de sustitucion...";
        poly G = substPow(Cpdi[1], mi[1] div g);
        for (i = 2; i <= s; i++)
        {
            poly Hi = substPow(Cpdi[i], mi[i] div g);
            G = gcd(G, Hi);
        }
        rz = G;
    }

    "//   r(z) obtenido, grado " + string(deg(rz));
    return(selectBest(rz, p, n, g));
}

// Caso n = potencia de primo (s = 1, e >= 2)
static proc casePrimePower(int n, int p, int q1, int ndivq,
                           poly Cpndivq, bigint g)
{
    "//   [POTENCIA DE PRIMO] n = " + string(q1) + "^" + string(factorizeSmall(n
        )[1][2]);
    "//   Candidatos g = " + string(g);

    if (g <= bigint(20000000))
    {
        "//   g es manejable -> metodo de raices g-esimas";
        poly irr = findIrred(p, n);
        poly z0 = findRoot(Cpndivq, irr, p, n);
        return(selectBestWithIrr(z0, irr, p, n, g));
    }

    "//   g INMANEJABLE -> Abortando Heath-Loehr. Loteria Parker...";
    // Se pasa como lista para unificar con el caso compuesto
    return(exhaustiveSearch(n, p, list(Cpndivq), list(ndivq)));
}

// --- Busqueda exhaustiva en orden de Parker (Loteria Unificada) ---
// Ahora acepta una lista de divisores y comprueba compatibilidad con TODOS.
static proc exhaustiveSearch(int n, int p, list Cpdi, list di)
{
    "//   [EXHAUSTIVA] Busqueda en orden Parker para p=" + string(p)
        + ", n=" + string(n);

    // --- Termino constante fijado por compatibilidad con C_{p,1} ---
    int a0_int;
```

```
if (p == 2)
{
    a0_int = 1;
}
else
{
    poly cp1 = conwayDeg1(p);
    int a0_C1 = int(leadcoef(jet(cp1, 0)));
    int gamma_val = ((-a0_C1) mod p + p) mod p;
    if (n mod 2 == 0) { a0_int = gamma_val mod p; }
    else              { a0_int = ((-gamma_val) mod p + p) mod p; }
}
"//␣␣␣Termino␣constante␣fijado:␣a_0␣=␣" + string(a0_int);

// --- Factores de p^n - 1 para el test de primitividad ---
bigint M = bigint(p)^n - 1;
list primesM;
int M_is_prime = millerRabinBigint(M);
if (M_is_prime)
{
    "//␣␣␣p^n␣-␣1␣es␣primo␣->␣todo␣irreducible␣es␣primitivo";
    primesM[1] = M;
}
else
{
    primesM = primeFactors(M, p, n);
    "//␣␣␣" + string(size(primesM)) + "␣factores␣primos␣de␣p^n-1";
}

// --- Exponentes de compatibilidad (precalculados una sola vez) ---
int s = size(di);
list compat_exp;
int m_idx;
for (m_idx = 1; m_idx <= s; m_idx++)
{
    compat_exp[m_idx] = M div (bigint(p)^di[m_idx] - 1);
}

// --- Contadores Parker: cnt[i] in {0,...,p-1} para a_i, i=1..n-1 ---
// cnt[1] corresponde a a_1 (varia mas rapido en Parker tras a_0).
int i; list cnt;
for (i = 1; i <= n-1; i = i+1) { cnt[i] = 0; }

poly candidate;
int coef_val; int is_irred; int k; list facs; int done = 0;
int total = 0;
poly alpha_e; poly val; int is_comp; int is_prim; int j;
bigint e; poly r;

while (!done)
{
    total = total + 1;

    // --- Construir candidato con coeficientes Parker ---
    candidate = var(1)^n + number(a0_int);
    for (i = 1; i <= n-1; i = i+1)
```

```
{
    coef_val = parkerCoef(cnt[i], i, n, p);
    if (coef_val != 0)
    {
        candidate = candidate + number(coef_val) * var(1)^i;
    }
}

// --- Test 1: irreducibilidad ---
is_irred = isIrreducibleBenOr(candidate, n);

if (is_irred && deg(candidate) == n)
{
    // --- Test 2: compatibilidad con TODOS los divisores maximales ---
    is_comp = 1;
    for (m_idx = 1; m_idx <= s; m_idx++)
    {
        alpha_e = polyPow(var(1), compat_exp[m_idx], candidate);
        val = polyEval(Cpdi[m_idx], alpha_e, candidate);
        if (val != 0) { is_comp = 0; break; }
    }

    if (is_comp)
    {
        // --- Test 3: primitividad ---
        is_prim = 1;
        if (!M_is_prime)
        {
            for (j = 1; j <= size(primesM); j++)
            {
                e = M div primesM[j];
                r = polyPow(var(1), e, candidate);
                if (r == 1) { is_prim = 0; break; }
            }
        }
        if (is_prim)
        {
            "//␣␣␣Conway␣encontrado␣tras␣" + string(total)
                + "␣candidatos:␣" + string(candidate);
            return(candidate);
        }
    }
}

// --- Avanzar al siguiente candidato en orden Parker ---
// Incrementar cnt[1], propagando acarreo hacia cnt[2], cnt[3], ...
i = 1;
while (i <= n-1)
{
    cnt[i] = cnt[i] + 1;
    if (cnt[i] < p) { break; }
    cnt[i] = 0; i = i + 1;
}
if (i > n-1) { done = 1; }

// Progreso
```

```
        if (total mod 5000 == 0)
        {
            "//␣␣␣Probados␣" + string(total) + "␣candidatos...";
        }
    }
    ERROR("exhaustiveSearch:␣espacio␣agotado␣para␣p="
        + string(p) + ",␣n=" + string(n));
}



// Para n primo, g = (p^n-1)/(p-1) es enorme pero h = p^n/tau ~ 1,
// lo que significa que el Conway aparece muy pronto en la lista
// lexicografica de Parker. Por eso iteramos en orden de Parker exacto
// y paramos al primer candidato valido.
//
// El termino constante a_0 = (-1)^n * gamma (donde gamma es la raiz
// de C_{p,1}) esta fijado de antemano por la condicion de compatibilidad,
// lo que reduce el espacio de busqueda en un factor p.

static proc casePrimeExhaust(int n, int p, poly C1)
{
    "//␣␣␣[PRIMO]␣Busqueda␣exhaustiva␣en␣orden␣Parker␣para␣n=" + string(n);

    // Calcular el termino constante obligatorio por compatibilidad con C_{p,1}:
    // Si alpha es raiz de C_{p,n} entonces alpha^{(p^n-1)/(p-1)} = gamma
    // donde gamma es la raiz de C_{p,1}. El producto de las raices de C_{p,n}
    // es (-1)^n * gamma, que es exactamente el termino independiente.
    int a0_C1 = int(leadcoef(jet(C1, 0)));
    int gamma_val = ((-a0_C1) mod p + p) mod p;
    int const_term;
    if (n mod 2 == 0) { const_term = gamma_val mod p; }
    else              { const_term = ((-gamma_val) mod p + p) mod p; }
    "//␣␣␣Termino␣constante␣fijado:␣a_0␣=␣" + string(const_term);

    // Precalcular factores de p^n-1 una sola vez para todo el bucle
    bigint M = bigint(p)^n - 1;
    list primesM;

    // --- ATAJO PARA PRIMOS DE MERSENNE EN EL BUCLE EXHAUSTIVO ---
    if (p == 2 && isMersenneExponent(n))
    {
        primesM[1] = M; // Si es Mersenne, el unico factor primo es el mismo
    }
    else
    {
        primesM = primeFactors(M, p, n);
    }

    // Contadores Parker: cnt[i] in {0,...,p-1} para la posicion a_i, i=1..n-1.
    // cnt[1] varia mas rapido (comparado primero por Parker tras a_0).
    int i; list cnt;
    for (i = 1; i <= n-1; i = i+1) { cnt[i] = 0; }

    poly candidate;
    int coef_val; int is_irred; int k; list facs; int done = 0;
```

```
    int debug_contador = 0;

    while (!done)
    {
        debug_contador = debug_contador + 1;
        if (debug_contador mod 100 == 0)
        {
            "// Buscando... probados " + string(debug_contador) + " candidatos
                de Parker.";
        }

        // Construir candidato con coeficientes en orden de Parker
        candidate = var(1)^n + number(const_term);
        // Construir candidato con coeficientes en orden de Parker
        candidate = var(1)^n + number(const_term);
        for (i = 1; i <= n-1; i = i+1)
        {
            coef_val = parkerCoef(cnt[i], i, n, p);
            if (coef_val != 0) { candidate = candidate + number(coef_val) * var
                (1)^i; }
        }

        // Test 1: irreducibilidad (test de Ben-Or)
        is_irred = isIrreducibleBenOr(candidate, n);

        if (is_irred)
        {
            // Test 2: primitividad (usa primesM precalculado)
            if (isPrimitivePoly(candidate, n, list(primesM)))
            {
                // Test 3: compatibilidad con C_{p,1}
                if (isCompatible(candidate, C1, n, 1))
                {
                    // PARADA INMEDIATA: el primero valido en orden Parker es el
                        Conway
                    "//   Conway encontrado: " + string(candidate);
                    return(candidate);
                }
            }
        }

        // Avanzar al siguiente candidato en orden Parker:
        // incrementar cnt[1], propagando el acarreo hacia cnt[2], cnt[3], ...
        i = 1;
        while (i <= n-1)
        {
            cnt[i] = cnt[i] + 1;
            if (cnt[i] < p) { break; }
            cnt[i] = 0; i = i + 1;
        }
        if (i > n-1) { done = 1; }
    }
    ERROR("casePrimeExhaust: no encontrado para n=" + string(n) + ", p=" +
        string(p));
}
```

```
////////////////////////////////////////////////////////////////////////////////
//
//  SECCION 7: SELECCION DEL MEJOR CANDIDATO
//
//  Una vez obtenida la raiz z0 del polinomio auxiliar r(z), hay exactamente
//  g raices g-esimas de z0 en GF(p^n), dadas por alpha * zeta^k para
//  k = 0, ..., g-1, donde alpha es una raiz g-esima de z0 y zeta es una
//  raiz g-esima primitiva de la unidad.
//
//  iterateCandidates recorre estas g raices con las optimizaciones:
//    1. Test de primitividad antes de calcular el minpoly.
//    2. Multiplicacion incremental beta_{k+1} = beta_k * zeta.
//    3. Poda lexicografica de Parker en el calculo del minpoly.
//
////////////////////////////////////////////////////////////////////////////////

// Punto de entrada para caseComposite: z0 = var(1) en GF(p^n) = F_p[z]/(modp).
static proc selectBest(poly modp, int p, int n, bigint g)
{
    bigint M = bigint(p)^n - 1;
    list primesM = primeFactors(M, p, n);
    "//␣␣␣Calculando␣raiz␣g-esima␣de␣z0...";
    poly alpha = gthRoot(var(1), g, modp, p, n);
    poly zeta  = gthRootOfUnity(g, modp, p, n, M, primesM);
    "//␣␣␣Iterando␣" + string(g) + "␣candidatos␣con␣poda␣Parker...";
    return(iterateCandidates(alpha, zeta, modp, p, n, g, M, primesM));
}

// Punto de entrada para casePrimePower: z0 es una raiz de r(z) ya calculada.
static proc selectBestWithIrr(poly z0, poly modp, int p, int n, bigint g)
{
    bigint M = bigint(p)^n - 1;
    list primesM = primeFactors(M, p, n);
    "//␣␣␣Calculando␣raiz␣g-esima␣de␣z0...";
    poly alpha = gthRoot(z0, g, modp, p, n);
    poly zeta  = gthRootOfUnity(g, modp, p, n, M, primesM);
    "//␣␣␣Iterando␣" + string(g) + "␣candidatos␣con␣poda␣Parker...";
    return(iterateCandidates(alpha, zeta, modp, p, n, g, M, primesM));
}

// Bucle principal sobre los g candidatos.
// Para cada beta = alpha * zeta^k:
//   - descarta los no primitivos rapidamente
//   - calcula el minpoly con poda Parker
//   - actualiza 'best' si el nuevo minpoly es mejor
static proc iterateCandidates(poly alpha, poly zeta, poly modp,
                              int p, int n, bigint g, bigint M,
                              list primesM)
{
    ideal I = std(ideal(modp));
    poly best = 0; int found = 0;
    bigint k; bigint count_prim = 0;
    poly beta = reduce(alpha, I);
    int is_prim; int j; poly one = poly(1); poly mp;
```

```
    for (k = 0; k < g; k = k+1)
    {
        // Test de primitividad: beta^{M/q} != 1 para todo primo q | M
        is_prim = 1;
        for (j = 1; j <= size(primesM); j++)
        {
            if (fastPow(beta, M div primesM[j], I) == one) { is_prim = 0; break;
                }
        }

        if (is_prim)
        {
            count_prim = count_prim + 1;
            // Calcular minpoly con poda lexicografica de Parker
            mp = minPolyPruned(beta, p, n, modp, best);
            if (mp != 0)
            {
                best = mp; found = 1;
                "//␣␣␣Nuevo␣mejor␣en␣k=" + string(k) + ":␣" + string(best);
            }
        }

        // Multiplicacion incremental: evita recalcular beta desde alpha cada
            vez
        beta = reduce(beta * zeta, I);

        // Progreso cada 5000 iteraciones
        if (k > 0 && (k mod 5000 == 0))
        {
            "//␣␣␣Progreso:␣k=" + string(k) + "/" + string(g)
                + "␣␣primitivos=" + string(count_prim);
        }
    }

    if (!found) { ERROR("iterateCandidates:␣ningun␣candidato␣primitivo␣
        encontrado."); }
    "//␣␣␣Completado:␣" + string(count_prim) + "␣primitivos␣de␣" + string(g) + "
        ␣candidatos.";
    return(best);
}


////////////////////////////////////////////////////////////////////////////////
//
//   SECCION 8: ENTRADA PRINCIPAL Y RECURSION
//
////////////////////////////////////////////////////////////////////////////////


// Calcula el polinomio de Conway C_{p,n} donde p = char(basering).
// Se puede pasar una lista 'known' con Conway ya calculados para reutilizar.
proc conwayPoly(int n, list #)
"USAGE:␣␣␣conwayPoly(n␣[,␣known]);
RETURN:␣␣poly,␣el␣polinomio␣de␣Conway␣C_{p,n}
ASSUME:␣␣ring␣R␣=␣p,␣x,␣dp;␣con␣p␣primo.
```

```
EXAMPLE: example conwayPoly; shows an example
"
{
    int p = char(basering);
    if (p == 0) { ERROR("char(basering) debe ser primo > 0."); }

    list known;
    if (size(#) > 0) { known = #[1]; }

    poly result = conwayRec(n, p, known);
    return(result);
}
example
{
    "EXAMPLE:"; echo = 2;
    ring R = 2, x, dp;
    conwayPoly(1);
    conwayPoly(3);
    conwayPoly(6);
}

// Recursion interna. Calcula C_{p,n} recursivamente sobre los divisores de n.
static proc conwayRec(int n, int p, list known)
{
    // --- BASE DE DATOS INTERNA (Cero bloqueos, cero archivos) ---
    // Si le pides un grado imposible, lo devuelve al instante.
    if (p == 2 && n == 77)
    {
        "// [BASE DE DATOS] Rescatado C_{2,77} de la memoria interna!";
        return(var(1)^77 + var(1)^15 + var(1)^14 + var(1)^11 + var(1)^4 + var(1)
            ^2 + 1);
    }

    if (p == 2 && n == 32)
    {
        "// [BASE DE DATOS] Rescatado C_{2,32} de la memoria interna!";
        return(var(1)^32 + var(1)^15 + var(1)^9 + var(1)^7 + var(1)^4 + var(1)^3
             + 1);
    }

    if (p == 2 && n == 64)
    {
        "// [BASE DE DATOS] Rescatado C_{2,64} de la memoria interna!";
        return(var(1)^64 + var(1)^33 + var(1)^30 + var(1)^26 + var(1)^25 + var
            (1)^24 + var(1)^23 + var(1)^22 + var(1)^21 + var(1)^20 + var(1)^18 +
             var(1)^13 + var(1)^12 + var(1)^11 + var(1)^10 + var(1)^7 + var(1)^5
             + var(1)^4 + var(1)^2 + var(1) + 1);
    }
    // ------------------------------------------------------------

    // 1. Cache en memoria RAM (lista known)
    if (size(known) >= n)
    {
        if (typeof(known[n]) == "poly")
        {
            if (known[n] != 0)
```

```
            {
                "// [RAM] Rescatado C_{" + string(p) + "," + string(n) + "} de
                    la memoria rapida!";
                return(known[n]);
            }
        }
    }

    // 2. Caso base
    if (n == 1)
    {
        poly c1 = conwayDeg1(p);
        return(c1);
    }

    // 3. Factorizar n
    list facN = factorizeSmall(n);
    int s = size(facN);
    list di; list qi;
    int i;
    for (i = 1; i <= s; i++)
    {
        qi[i] = facN[i][1];
        di[i] = n div qi[i];
    }

    // 4. Calcular recursivamente los Conway de divisores maximales
    list Cpdi;
    int sz;
    for (i = 1; i <= s; i++)
    {
        Cpdi[i] = conwayRec(di[i], p, known);
        sz = size(known);
        if (sz < di[i])
        {
            int kk;
            for (kk = sz + 1; kk <= di[i]; kk++) { known[kk] = poly(0); }
        }
        known[di[i]] = Cpdi[i];
    }

    // 5. Parametros
    list mi;
    for (i = 1; i <= s; i++)
    {
        mi[i] = (bigint(p)^n - 1) div (bigint(p)^di[i] - 1);
    }
    bigint g = mi[1];
    for (i = 2; i <= s; i++) { g = mcdBigint(g, mi[i]); }

    "// ==============================================";
    "// C_{" + string(p) + "," + string(n) + "}: s=" + string(s)
        + ", g=" + string(g);
    "// ==============================================";

    // 6. Delegar al caso
```

```
    poly result;
    if (s >= 2)
    {
        // Caso compuesto (varios primos distintos)
        result = caseComposite(n, p, s, di, qi, mi, Cpdi, g);
    }
    else
    {
        if (facN[1][2] == 1)
        {
            // Caso primo puro (exponente = 1). Va directo a busqueda de Parker.
            result = casePrimeExhaust(n, p, Cpdi[1]);
        }
        else
        {
            // Caso potencia de primo (exponente >= 2). Va a Heath-Loehr o
                Loteria.
            result = casePrimePower(n, p, qi[1], di[1], Cpdi[1], g);
        }
    }

    return(result);
}

// C_{p,1}: menor elemento primitivo de F_p, expresado como x - gamma.
// Los casos frecuentes estan hardcodeados para evitar el bucle.
static proc conwayDeg1(int p)
{
    if (p == 2)  { return(var(1) + 1); }
    if (p == 3)  { return(var(1) + 1); }  // gamma=2, x-2 = x+1 en F_3
    if (p == 5)  { return(var(1) + 3); }  // gamma=2, x-2 = x+3 en F_5
    if (p == 7)  { return(var(1) + 4); }  // gamma=3, x-3 = x+4 en F_7
    if (p == 11) { return(var(1) + 9); }  // gamma=2, x-2 = x+9 en F_11
    if (p == 13) { return(var(1) + 11); } // gamma=2, x-2 = x+11 en F_13

    // Para otros primos: buscar el menor primitivo por fuerza bruta
    list fp1 = factorizeSmall(p - 1);
    int g;
    for (g = 2; g < p; g++)
    {
        int ok = 1;
        int j;
        for (j = 1; j <= size(fp1); j++)
        {
            bigint e = bigint(p-1) div bigint(fp1[j][1]);
            bigint r = 1; bigint base = bigint(g); bigint ee = e;
            while (ee > 0)
            {
                if ((ee mod 2) == 1) { r = (r * base) mod bigint(p); }
                ee = ee div 2;
                if (ee > 0) { base = (base * base) mod bigint(p); }
            }
            if (r == 1) { ok = 0; break; }
        }
        if (ok) { return(var(1) - number(g)); }
    }
```

```
    ERROR("conwayDeg1:␣no␣se␣encontro␣primitivo␣en␣F_" + string(p));
}

// Calcula C_{2,n} para todos los exponentes de Mersenne.
// Para estos n, 2^n - 1 es primo, asi que todo irreducible de grado n
// con termino constante 1 que sea el menor en Parker es el Conway.
// No necesita factorizar 2^n-1 ni test de primitividad.
//
// REQUIERE: ring R = 2, x, dp;
proc conwayPolyMersenne()
"USAGE:␣␣␣conwayPolyMersenne();
RETURN:␣␣nada␣(imprime␣resultados␣por␣pantalla)
ASSUME:␣␣ring␣R␣=␣2,␣x,␣dp;
NOTE:␣␣␣␣Calcula␣C_{2,n}␣para␣todos␣los␣exponentes␣de␣Mersenne␣conocidos,
␣␣␣␣␣␣␣␣deteniendose␣cuando␣el␣calculo␣se␣vuelve␣inviable.
␣␣␣␣␣␣␣␣Para␣estos␣n,␣2^n-1␣es␣primo,␣todo␣irreducible␣es␣primitivo,
␣␣␣␣␣␣␣␣y␣el␣Conway␣es␣el␣menor␣irreducible␣monico␣con␣a_0=1␣en␣Parker.
"
{
    if (char(basering) != 2) { ERROR("Se␣requiere␣caracteristica␣2."); }

    list mersennes = 2, 3, 5, 7, 13, 17, 19, 31, 61, 89, 107, 127,
                     521, 607, 1279, 2203, 2281, 3217, 4253, 4423,
                     9689, 9941, 11213, 19937, 21701, 23209, 44497,
                     86243, 110503, 132049, 216091, 756839, 859433,
                     1257787, 1398269, 2976221, 3021377, 6972593,
                     13466917, 20996011, 24036583, 25964951, 30402457,
                     32582657, 37156667, 42643801, 43112609, 57885161,
                     74207281, 77232917, 82589933, 136279841;

    "";
    "============================================================";
    "␣␣Polinomios␣de␣Conway␣para␣exponentes␣de␣Mersenne␣(p=2)";
    "␣␣2^n␣-␣1␣primo␣=>␣todo␣irreducible␣es␣primitivo";
    "␣␣Solo␣hay␣que␣buscar␣el␣menor␣irreducible␣con␣a_0␣=␣1";
    "============================================================";
    "";

    int n, t, elapsed, i;
    poly result;
    for (i = 1; i <= size(mersennes); i++)
    {
        n = mersennes[i];
        "//␣---␣C_{2," + string(n) + "}␣(Mersenne␣M_" + string(n) + ")␣---";

        t = timer;
        result = menorIrredParker(n);
        elapsed = timer - t;

        if (result != 0)
        {
            "//␣␣␣C_{2," + string(n) + "}␣=␣" + string(result);
            "//␣␣␣Tiempo:␣" + string(elapsed) + "␣ms";
            "";
        }
        else
```

```
        {
            "//␣␣␣No␣se␣pudo␣calcular␣(grado␣demasiado␣grande).";
            "//␣␣␣Deteniendo␣aqui.";
            return();
        }
    }
}

// Busca el menor polinomio irreducible monico de grado n sobre F_2
// con termino constante 1, en orden de Parker.
// En caracteristica 2, Parker = lexicografico estandar.
//
// Estrategia: recorrer polinomios x^n + a_{n-1}x^{n-1} + ... + a_1*x + 1
// en orden lexicografico (a_1 varia mas rapido) y devolver el primero
// que sea irreducible.
//
// Optimizacion: test de irreducibilidad de Ben-Or.
// Para F_2 y n primo basta comprobar x^{2^n} = x mod f(x),
// evitando factorizar el polinomio.
static proc menorIrredParker(int n)
{
    // Caso trivial
    if (n == 2) { return(var(1)^2 + var(1) + 1); }
    if (n == 3) { return(var(1)^3 + var(1) + 1); }

    // Para grados muy grandes, avisamos y paramos
    // (el test de irreducibilidad se vuelve muy lento)
    if (n > 50000)
    {
        "//␣␣␣AVISO:␣grado␣" + string(n) + "␣demasiado␣grande␣para"
            + "␣test␣de␣irreducibilidad␣en␣tiempo␣razonable.";
        return(poly(0));
    }

    // Contadores Parker: cnt[i] para a_i, i = 1..n-1
    // En char 2, Parker = lex estandar, asi que cnt[1] varia mas rapido.
    // Todos los coeficientes son 0 o 1 (estamos en F_2).
    list cnt;
    int i;
    for (i = 1; i <= n-1; i++) { cnt[i] = 0; }

    int total = 0;
    int done = 0;
    poly candidate;

    while (!done)
    {
        total = total + 1;

        // Construir candidato: x^n + sum(cnt[i]*x^i) + 1
        candidate = var(1)^n + 1;
        for (i = 1; i <= n-1; i++)
        {
            if (cnt[i] != 0) { candidate = candidate + var(1)^i; }
        }
```

```
        // Test de irreducibilidad.
        // Para F_2 y n primo: f es irreducible de grado n
        // sii x^{2^n} = x mod f. Esto evita factorize().
        if (isIrredBen(candidate, n))
        {
            "//␣␣␣Encontrado␣tras␣" + string(total) + "␣candidatos";
            return(candidate);
        }

        // Avanzar al siguiente en orden Parker (= lex para p=2)
        i = 1;
        while (i <= n-1)
        {
            cnt[i] = cnt[i] + 1;
            if (cnt[i] < 2) { break; }
            cnt[i] = 0;
            i = i + 1;
        }
        if (i > n-1) { done = 1; }

        if (total mod 1000 == 0)
        {
            "//␣␣␣Probados␣" + string(total) + "␣candidatos...";
        }
    }

    return(poly(0));
}

// Test de irreducibilidad de Ben-Or (1981).
//
// Un polinomio f de grado n sobre F_p es irreducible sii:
//   (1) x^{p^n} = x mod f
//   (2) gcd(x^{p^d} - x, f) = 1 para todo 1 <= d <= n/2
//
// La condicion (1) verifica que las raices de f estan en F_{p^n}.
// La condicion (2) verifica que ninguna raiz esta en un subcuerpo.
//
// Para n primo, (2) se reduce a gcd(x^p - x, f) = 1, que es
// trivialmente cierto si f tiene grado > 1 y termino constante != 0.
// Asi que para n primo basta con (1).
//
// Para n compuesto, hay que comprobar (2) para cada d que divida a n
// con d < n. En la practica, basta con los d = n/q para cada primo q | n.
//
// Coste: O(n^2 * log(p^n)) operaciones en F_p, frente al O(n^3) o peor
// de los algoritmos de factorizacion completa.
static proc isIrreducibleBenOr(poly f, int n)
{
    int p = char(basering);
    ideal I = std(ideal(f));

    // Condicion (1): x^{p^n} = x mod f
    // Calculamos x^{p^n} haciendo n veces x -> x^p mod f
    poly cur = var(1);
    int i;
```

```
    for (i = 1; i <= n; i++)
    {
        cur = fastPow(cur, bigint(p), I);
    }
    if (cur != var(1)) { return(0); }

    // Condicion (2): gcd(x^{p^d} - x, f) = 1 para d | n, d < n
    // Solo necesitamos comprobar d = n/q para cada primo q | n.
    list facN = factorizeSmall(n);
    int j,d,k;
    poly xpd, h, g;
    for (j = 1; j <= size(facN); j++)
    {
        d = n div facN[j][1];
        if (d < n)
        {
            // Calcular x^{p^d} mod f haciendo d cuadrados/potencias
            xpd = var(1);
            for (k = 1; k <= d; k++)
            {
                xpd = fastPow(xpd, bigint(p), I);
            }
            // gcd(x^{p^d} - x, f) debe ser 1
            h = xpd - var(1);
            if (h == 0) { return(0); }
            g = gcd(h, f);
            if (deg(g) > 0) { return(0); }  // factor comun no trivial
        }
    }

    return(1);
}



////////////////////////////////////////////////////////////////////////////////
//
//  SECCION 9: CONTEO DE CANDIDATOS
//
//  candidatosConway calcula g y phi(p^n-1)/n para cualquier (p,n),
//  sin limite de precision porque usa bigint en Singular.
//
////////////////////////////////////////////////////////////////////////////////

// Calcula g (candidatos compatibles) y phi(p^n-1)/n (irreducibles primitivos)
// para el polinomio de Conway C_{p,n}.
proc candidatosConway(int n)
"USAGE:␣␣␣candidatosConway(n);␣␣n␣entero␣positivo
RETURN:␣␣list(g,␣phi(p^n-1)/n)
ASSUME:␣␣ring␣R␣=␣p,␣x,␣dp;␣con␣p␣primo.
NOTE:␣␣␣␣g␣=␣mcd{␣(p^n-1)/(p^{n/qi}-1)␣}␣segun␣Heath-Loehr.
␣␣␣␣␣␣␣␣phi(p^n-1)/n␣=␣numero␣de␣polinomios␣irreducibles␣primitivos␣de␣grado␣n.
"
{
    int p = char(basering);
```

```
    bigint Mn = bigint(p)^n - 1;

    // Calcular g
    bigint g;
    if (n == 1)
    {
        g = 1;
    }
    else
    {
        list facN = factorizeSmall(n);
        int s = size(facN); int i;
        list mi;
        for (i = 1; i <= s; i++)
        {
            int qi = facN[i][1];
            mi[i] = Mn div (bigint(p)^(n div qi) - 1);
        }
        g = mi[1];
        for (i = 2; i <= s; i++) { g = mcdBigint(g, mi[i]); }
    }

    // Calcular phi(Mn) = Mn * prod_{q primo, q|Mn} (1 - 1/q)
    // usando phi(Mn) = prod q^(e-1)*(q-1) para cada q^e || Mn
    list facs_raw = primeFactors(Mn, p, n);
    bigint phi_Mn = Mn;
    int j;
    for (j = 1; j <= size(facs_raw); j++)
    {
        bigint q = facs_raw[j];
        phi_Mn = phi_Mn div q * (q - 1);
    }

    bigint irred_prim = phi_Mn div bigint(n);

    list result;
    result[1] = g;
    result[2] = irred_prim;
    return(result);
}

// Muestra por pantalla el resumen de candidatos para C_{p,n}.
proc mostrarCandidatos(int n)
"USAGE:␣␣␣mostrarCandidatos(n);␣␣n␣entero␣positivo
ASSUME:␣␣ring␣R␣=␣p,␣x,␣dp;␣con␣p␣primo.
"
{
    int p = char(basering);
    list res = candidatosConway(n);
    bigint g        = res[1];
    bigint phi_n    = res[2];
    "";
    "===␣Candidatos␣para␣C_{" + string(p) + "," + string(n) + "}␣===";
    "␣␣g␣␣␣(candidatos␣compatibles␣con␣Conway␣de␣grado␣inferior):␣" + string(g);
    "␣␣phi(p^n-1)/n␣␣(irreducibles␣primitivos␣de␣grado␣n␣totales):␣" + string(
        phi_n);
```

```
    "␣␣El␣Conway␣C_{p,n}␣es␣el␣menor␣en␣orden␣Parker␣entre␣los␣g␣compatibles.";
    "";}
```

Bibliografía

[1] Michael Ben-Or. «Probabilistic algorithms in finite fields». En: *Proceedings of the 22nd Annual Symposium on Foundations of Computer Science (FOCS)* (1981), págs. 394-398. DOI: 10.1109/SFCS.1981.37.

[2] Wolfram Decker, Gert-Martin Greuel, Gerhard Pfister y Hans Schönemann. SINGULAR *4-4-0 — A computer algebra system for polynomial computations.* http://www.singular.uni-kl.de. 2024.

[3] John B. Fraleigh. *A First Course in Abstract Algebra.* Pearson, 1999.

[4] Lenwood S. Heath y Nicholas A. Loehr. «New algorithms for generating Conway polynomials over finite fields». En: *Journal of Symbolic Computation* 38 (2004), págs. 1003-1024.

[5] Rudolf Lidl y Harald Niederreiter. *Introduction to Finite Fields and Their Applications.* Cambridge: Cambridge University Press, 1994.

[6] Frank Lüebeck. *Conway Polynomials for Finite Fields.* RWTH Aachen University. 2026. URL: https://www.math.rwth-aachen.de/~Frank.Luebeck/data/ConwayPol/index.html.

[7] Gary L. Mullen y Daniel Panario. *Handbook of Finite Fields.* CRC Press, 2013.

[8] Richard A. Parker. *Finite Fields and Conway Polynomials.* Talk given at IBM Heidelberg Scientific Center. 1990.

[9] Michael O. Rabin. «Probabilistic Algorithms in Finite Fields». En: *SIAM Journal on Computing* 9 (2 1980), págs. 273-280.

[10] Diego Ruano y Santiago Laplagne. *Conway Polynomials and Singular.* Presentación en Segovia, 10 de octubre de 2007. 2007.

[11] A. Scheerhorn. «Trace- and norm-compatible extensions of finite fields». En: *Applicable Algebra in Engineering, Communication and Computing* 3 (1992), págs. 199-209. DOI: 10.1007/BF01268660. URL: https://doi.org/10.1007/BF01268660.

[12] Monit Sharma. *Sometimes It's Too Slow... for Shor!!* Mar. de 2026. URL: https://medium.com/@_monitsharma/sometimes-its-too-slow-for-shor-c01677032324.